

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is a solution to this [latex.org forum query](#).

Contents

1 Installation

1.1 Package manager

When not already available, **beanoves** package may be installed using a TEX distribution's package manager, either from the graphical user interface, or with the relevant command (`tlmgr` for **T_EX Live** and `mpm` for **MiK_TE_X**). This should install files **beanoves.sty** and its debug version **beanoves-debug.sty** as well as **beanoves-doc.pdf** documentation.

1.2 Manual installation

The **beanoves** source files are available from the [source repository](#). They can also be fetched from the [CTAN repository](#).

1.3 Usage

The **beanoves** package is imported by putting `\RequirePackage{beanoves}` in the preamble of a **L^AT_EX** document that uses the **beamer** class. Should the package cause problems, its features can be temporarily deactivated with simple commands `\BeanovesOff` and `\BeanovesOn`.

2 Minimal example

The L^AT_EX document below is a contrived example to show how the **beamer** overlay specifications have been extended. More demonstration files are available from the [beanoves source repository](#).

```
1 \documentclass{beamer}
2 \RequirePackage{beanoves}
3 \begin{document}
4 \Beanoves {
5   A = 1:4,
6   B = A.last::3,
7   C = B.next,
8 }
9 \begin{frame}
10  {\Large Frame \insertframenum}
11  {\Large Slide \insertslidenum}
12 -- \visible<?(A.1)> {Only on slide 1}\\
13 -- \visible<?(B.range)> {Only on slides 4 to 6}\\
14 -- \visible<?(C.1)> {Only on slide 7}\\
15 -- \visible<?(A.2)> {Only on slide 2}\\
16 -- \visible<?(B.2:B.last)> {Only on slides 5 to 6}\\
17 -- \visible<?(C.2)> {Only on slide 8}\\
18 -- \visible<?(A.next)-> {From slide 5}\\
19 -- \visible<?(B.3:B.last)> {Only on slide 6}\\
20 -- \visible<?(C.3)> {Only on slide 9}\\
21 \end{frame}
22 \end{document}
```

On line 4, we use the `\Beanoves` command to declare *named overlay sets*. On line 5, we declare an overlay set named ‘A’, which is a range starting at slide 1 and ending at slide 4. On line 12, the extended *named overlay specification* `?(A.1)` stands for 1 because 1 is the first index of the overlay set named A. On line 15, `?(A.2)` stands for 2 whereas on line 18, `?(A.next)` stands for 5. On line 6, we declare a second overlay set named ‘B’, starting after the 3 slides of ‘A’ namely 4. Its length is 3 meaning that its last slide number is 6, thus each `?(B.last)` is replaced by 6. The next slide number after slide range ‘B’ is 7 which is also the start of the third slide range due to line 7.

3 Named overlay sets

3.1 Presentation

Within a **beamer** frame, there are different slides that appear in turn according to overlay specifications. The main overlay set is a range of integers covering all the slide numbers, from one to the total amount of slides. In general, an overlay set is a range of positive integers identified by a unique name. The main practical interest is that such sets may be defined relative to one another, we can even have lists of overlay sets. Finally, we can use these lists to build and organize **beamer** overlay specifications logically.

3.2 Named overlay reference

A.1, C.2 are *named overlay references*, as well as A and X!A.B.C.2. More precisely, they are string identifiers, each one referencing a well defined static integer or range to be used in **beamer** overlay specifications. They have 3 components:

1. the *frame identifier* $\langle id \rangle!$, like X!, optional
2. the *path* $\langle c_1 \rangle \dots \langle c_j \rangle$ like A.B.C, required ($j > 0$).
3. the *attributes* $\langle attr_1 \rangle \dots \langle attr_k \rangle$ like .1, .2, .reset.first...

The *full path* is $\langle id \rangle! \langle c_1 \rangle \dots \langle c_j \rangle$. The *frame ids* and path components $\langle c \rangle$'s are alphanumeric case sensitive identifiers, with possible underscores but with no space. Unicode symbols above U+00A0 are allowed if the underlying T_EX engine supports it. Only the *frame id* is allowed to be empty, in which case it may apply to any common frame. The *path components* must not consist of only lowercase letters¹. The attributes $\langle attr \rangle$'s may be signed integers of one of the keywords defined below.

The mapping from *named overlay references* to sets of integers is defined at the global T_EX level to allow its use in `\begin{frame}<...>` and to share the same overlay sets between different frames. Hence the *frame id* due to the need to possibly target a particular frame.

3.3 Defining named overlay sets

In order to define *named overlay sets*, we can either execute the next `\Beanoves` command before a **beamer** frame environment, or use the custom `beanoves` option of this environment.

<code>\Beanoves</code>	<code>\Beanoves{\langle ref_1 \rangle=\langle spec_1 \rangle, \dots, \langle ref_j \rangle=\langle spec_j \rangle}</code>
------------------------	---

<code>\Beanoves*</code>	Each $\langle ref \rangle$ key is a <i>named overlay reference</i> whereas each $\langle spec \rangle$ is an <i>overlay set specifier</i> . When the same $\langle ref \rangle$ key is used multiple times, only the last one is taken into account.
-------------------------	--

When performed at the document level, the `\Beanoves` command starts by cleaning what was set by previous calls. When performed inside L^AT_EX environments, each new call cumulates with the previous one. Notice that the argument of this function can contain macros: they will be exhaustively expanded at resolution time².

<code>beanoves</code>	<code>beanoves = {\langle ref_1 \rangle=\langle spec_1 \rangle, \dots, \langle ref_j \rangle=\langle spec_j \rangle}</code>
-----------------------	---

The `\Beanoves` arguments take precedence over both the `\Beanoves*` arguments and the `beanoves` options. This allows to provide an overlay name only when not already defined, which is helpfull when the very same frame source is included multiple times in different contexts. Notice that $\langle ref \rangle=1$ can be shortened to $\langle ref \rangle$.

3.3.1 Value specifiers

Hereafter $\langle value \rangle$ denotes a numerical expression.

¹This will avoid collisions with the `fp` module of `expl3`.

²Precision is needed about the exact time when the expansion occurs.

Standalone

$\langle \text{ref} \rangle = \langle \text{value} \rangle$, a *value specifier* for a single number. When $= \langle \text{value} \rangle$ is omitted it defaults to $=1$.

The numerical expressions are evaluated and then rounded using `\fp_eval:n`. They can contain mathematical functions and *named overlay references* defined above. If this reference concerns a set of values, only the first one is considered.

The corresponding overlay set can be seen as a *value counter*.

3.3.2 Colon delimited range specifiers

In beamer, integer ranges are denoted by $\langle \text{first} \rangle - \langle \text{last} \rangle$. In order to authorize algebraic expressions for $\langle \text{first} \rangle$ and $\langle \text{last} \rangle$, the dash character $-$ is replaced by a colon $:$ in beanoes. Hereafter $\langle \text{first} \rangle$, $\langle \text{last} \rangle$ and $\langle \text{length} \rangle$ are placeholders for *value specifiers*.

Standalone

$\langle \text{ref} \rangle = \langle \text{first} \rangle :$,
 $\langle \text{ref} \rangle = \langle \text{first} \rangle ::$, for the infinite range of signed integers starting at and including $\langle \text{first} \rangle$.

$\langle \text{ref} \rangle = \langle \text{first} \rangle : \langle \text{last} \rangle$,
 $\langle \text{ref} \rangle = \langle \text{first} \rangle :: \langle \text{length} \rangle$,
 $\langle \text{ref} \rangle = : \langle \text{last} \rangle :: \langle \text{length} \rangle$,
 $\langle \text{ref} \rangle = :: \langle \text{length} \rangle : \langle \text{last} \rangle$, are variants for the same finite range of signed integers starting at and including $\langle \text{first} \rangle$, ending at and including $\langle \text{last} \rangle$, provided $\langle \text{first} \rangle + \langle \text{length} \rangle = \langle \text{last} \rangle + 1$. $\langle \text{first} \rangle$ can be omitted, in which case it defaults to 1. Additionally $: \langle \text{last} \rangle$ and $:: \langle \text{length} \rangle$ are then equivalent.

$\langle \text{ref} \rangle = : \langle \text{last} \rangle$,
 $\langle \text{ref} \rangle = :: \langle \text{length} \rangle$, are syntactic sugar when $\langle \text{first} \rangle$ is 1.

3.3.3 List specifiers

$\langle \text{ref} \rangle = [\langle \text{def}_1 \rangle, \dots, \langle \text{def}_j \rangle]$, where $\langle \text{def}_k \rangle$, $1 \leq k \leq j$, is one of

- $\langle \text{index} \rangle = \langle \text{value} \rangle$,
- $\langle \text{value} \rangle$, a shortcut for $\langle i \rangle = \langle \text{value} \rangle$, $\langle i \rangle$ being the smallest positive integer such that $\langle \text{ref} \rangle . \langle i \rangle$ is not already defined.

The first step is to remove previous $\langle \text{ref} \rangle$ related definitions, then execute the various $\langle \text{ref} \rangle . \langle i \rangle = \langle \text{value} \rangle$ definitions in the order given. Here is the **implementation**.

$\langle \text{ref} \rangle = \{ \langle \text{def}_1 \rangle, \dots, \langle \text{def}_j \rangle \}$, where $\langle \text{def}_k \rangle$, $1 \leq k \leq j$, is one of

- $\langle \text{name} \rangle = \langle \text{spec} \rangle$,
- $\langle \text{name} \rangle$ for $\langle \text{name} \rangle = 1$,

The first step is to remove previous $\langle \text{ref} \rangle$ related definitions, then execute the various $\langle \text{ref} \rangle . \langle \text{name} \rangle = \langle \text{spec} \rangle$ definitions in the order given. In a final step, the $\langle \text{name} \rangle$'s are collected in a comma separed list to initialize $\langle \text{ref} \rangle$ with. $\langle \text{spec} \rangle$ is any specifier. Here is the **implementation**.

4 Resolution of $\langle \dots \rangle$ query expressions

This is the key feature of the `beanoves` package, extending `beamer overlay specifications` normally included between pointed brackets. Before the *overlay specifications* are processed by the `beamer` class, the `beanoves` package scans them for any occurrence of $\langle \langle \text{queries} \rangle \rangle$. Each one is then evaluated and replaced by its resolved static counterpart. The overall result is finally forwarded to the `beamer` class.

The $\langle \text{queries} \rangle$ argument is a comma separated list of individual $\langle \text{query} \rangle$'s processed from left to right as explained below. Notice that nesting a $\langle \dots \rangle$ query expression inside another query expression is supported, embedded expressions being resolved first.

The named overlay sets defined above are queried for integer numerical values that will be passed to `beamer`. Turning an *overlay query* into the static expression it represents, as when above $\langle A.1 \rangle$ was replaced by 1, is denoted by *overlay query resolution* or simply *resolution*. The process starts by replacing any *query reference* by its value as explained below until obtaining numerical expressions that are evaluated and finally rounded to the nearest integer to feed `beamer` with either ranges or numbers. When the *query reference* is a previously declared $\langle \text{ref} \rangle$, like **A** after **A=1**, it is simply replaced by the corresponding declared $\langle \text{value} \rangle$, here 1. Otherwise, we use *implicit overlay queries* and their *resolution rules* depending on the definition of the named overlay set. Hereafter $\langle i \rangle$ denotes a signed integer whereas $\langle \text{value} \rangle$, $\langle \text{first} \rangle$, $\langle \text{last} \rangle$, $\langle \text{length} \rangle$ and $\langle i \text{ expr} \rangle$ stand for raw integers or more general numerical expressions that are evaluated beforehand.

Resolution occurs only when requested and the result is cached for performance reason.

4.1 Range overlay queries

We give queries after named overlay sets definitions.

$\langle \text{ref} \rangle = \langle \text{first} \rangle$: as well as $\langle \text{first} \rangle ::$ define a range limited from below:

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{first} \rangle -$
$\langle \text{ref} \rangle.1$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{first} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{first} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{first} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{first} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{first} \rangle$

Notice that $\langle \text{ref} \rangle.\text{previous}$ and $\langle \text{ref} \rangle.0$ are most of the time synonyms.

$\langle \text{ref} \rangle = \langle \text{first} \rangle : \langle \text{last} \rangle$ as well as variants $\langle \text{first} \rangle :: \langle \text{length} \rangle$, $:: \langle \text{length} \rangle : \langle \text{last} \rangle$ or $: \langle \text{last} \rangle :: \langle \text{length} \rangle$, which are equivalent provided $\langle \text{first} \rangle + \langle \text{length} \rangle = \langle \text{last} \rangle + 1$.

Define a range limited from both above and below:

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{first} \rangle - \langle \text{last} \rangle$
$\langle \text{ref} \rangle.1$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{first} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{first} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{first} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{first} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.\text{last}$	$\langle \text{last} \rangle$
$\langle \text{ref} \rangle.\text{next}$	$\langle \text{last} \rangle + 1$
$\langle \text{ref} \rangle.\text{length}$	$\langle \text{length} \rangle$

Notice that the resolution of the $\langle \text{ref} \rangle$ overlay query is a beamer range and not an algebraic difference, negative integers do not make sense there while in `beamer` context.

In the frame example below, we use the `\BeanovesResolve` command for the demonstration. It is mainly used for debugging and testing purposes.

```

1 \Beanoves {
2   A = 3:8, % or similarly A = 3::6, A = ::6:8 and A = :8::6
3 }
4 \begin{frame} {Frame \insertframenum} {Slide \insertslidenumber}
5 \ttfamily
6 \BeanovesResolve[show] (A)          == 3-8,
7 \BeanovesResolve[show] (A.1)       == 3,
8 \BeanovesResolve[show] (A.-1)      == 1,
9 \BeanovesResolve[show] (A.previous) == 2,
10 \BeanovesResolve[show] (A.first)   == 3,
11 \BeanovesResolve[show] (A.last)    == 8,
12 \BeanovesResolve[show] (A.next)    == 9,
13 \BeanovesResolve[show] (A.length)  == 6,
14 \end{frame}

```

$\langle \text{ref} \rangle = [\dots]$

$\langle \text{ref} \rangle = \{\dots\}$ See the list range specifiers in section 3.3.3.

4.2 Value counter queries

$\langle \text{ref} \rangle = \langle \text{value} \rangle$ defines a counter value.

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.1$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{value} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{value} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{value} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{value} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.\text{last}$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.\text{next}$	$\langle \text{value} \rangle + 1$

Additionally, resolution rules are provided for dedicated *overlay queries*. Here, $\langle ref \rangle$ is considered a standard programming variable:

$\langle ref \rangle = \langle set\ expression \rangle$, resolve $\langle set\ expression \rangle$, assign the result to the $\langle ref \rangle$ and use it. It defines $\langle ref \rangle$ globally if not already done. Here $\langle set\ expression \rangle$ is the longest character sequence with no space³. To remove this limitation, use an embedded query expression or enclose the `clist` item within a pair of braces.

$\langle ref \rangle += \langle integer\ expression \rangle$, resolve $\langle integer\ expression \rangle$ into $\langle integer \rangle$, advance $\langle ref \rangle$ by $\langle integer \rangle$ and use the result.

$++\langle ref \rangle$, increment $\langle ref \rangle$ by 1 and use it.

$\langle ref \rangle ++$, use $\langle ref \rangle$ and then increment it by 1.

This can be used for an indirection.

IN PROGRESS

```

1 \Beanoves {
2   A = 1,
3   B = [ 10 = 100 ],
4   C = 10,
5 }
6 \begin{frame} {Frame \insertframenum} {Slide \insertslidenumber}
7 \ttfamily
8 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.10) == 10,
9 \BeanovesResolve[show] (B.C)      == \BeanovesResolve[show] (B.10) == 100,
10 \BeanovesResolve[show] (A.C+=10) == \BeanovesResolve[show] (A.20) == 20,
11 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.20) == 20,
12 \BeanovesResolve[show] (A.C+=10) == \BeanovesResolve[show] (A.20) == 20+10,
13 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.20) == 30,
14 \BeanovesResolve[show] (B.C)      == \BeanovesResolve[show] (B.20) == 20,
15 \end{frame}

```

In order to decrement a counter, one can increment with a negative value, no dedicated syntax is provided yet.

For each new frame, these counters are reset to the value they were initialized with. Sometimes, resetting the counter manually is necessary, for example when managing `tikz` overlay material.

`\BeanovesReset` `\BeanovesReset` [*options*] [$\langle ref_1 \rangle [= \langle spec_1 \rangle]$, ..., $\langle ref_j \rangle [= \langle spec_j \rangle]$]

This command is very similar to `\Beanoves`, except that a standalone $\langle ref_i \rangle$ resets the counter to the last value it was initialized with, and that it is meant to be used inside a `frame` environment. Each $= \langle spec_j \rangle$ is optional and defaults to `=1`. When the `all` option is provided, some internals that were cached for performance reasons are cleared as well.

4.3 The beamer counters

While inside a `frame` environment, it is possible to save the current value of the `beamerpauses` counter that controls whether elements should appear on the current slide. For that, we can execute one of `\Beanoves{\langle ref \rangle=pauses}` or in a query

³The parser for algebraic expression is very rudimentary.

?(...($\langle ref \rangle$ =pauses)...). Then later on, we can use ?(...($\langle ref \rangle$)...) to refer to this saved value in the same frame⁴. Next frame source is an example of usage.

```

1 \begin{frame}
2 \visible<+>{A}\\
3 \visible<+>{B\Beanoves{afterB=pauses}}\\
4 \visible<+>{C}\\
5 \visible<?(afterB)>{other C}\\
6 \visible<?(afterB.previous)>{other B}\\
7 \end{frame}

```

“A” first appears on slide 1, “B” on slide 2 and “C” on slide 3. On line 2, `afterB` takes the value of the `beamerpauses` counter once updated, *id est* 3. “B” and “other B” as well as “C” and “other C” appear at the same time. If the `beamerpauses` counter is not suitable, we can execute instead one of `\Beanoves{ $\langle ref \rangle$ =slideinframe}` or inside a query ?(...)($\langle ref \rangle$ =slideinframe)(...). It uses the numerical value of `\insertslideinframe`.

4.4 Multiple queries

It is possible to replace the comma separated list of queries ?($\langle query_1 \rangle$),...,?($\langle query_j \rangle$) with the shorter single query ?($\langle query_1 \rangle$),..., $\langle query_j \rangle$).

4.5 Frame id

Except for very special situations, the *frame ids* can be left unspecified. When no *frame id* was explicitly provided, `beanoves` uses the *last frame id* and, if the resolution fails, an empty *frame id*. At the beginning of each frame, the *last frame id* is set to the *frame id* of the current frame, which is denoted *current frame id* and is empty by default. Then it gets updated after each named reference resolution where a *frame id* is explicitly given. For example, the first time `A.1` reference is resolved within a given frame, it is first translated to $\langle last\ frame\ id \rangle!A.1$, but when used just after `Y!C.2`, for example, it becomes a shortcut to `Y!A.1` because the *last frame id* is then `Y`.

In order to set the *frame id* of the current frame to frame $\langle id \rangle$, use the new `beanoves id` option of the `beamer` frame environment.

`beanoves id` `beanoves id= $\langle frame\ id \rangle$ ,`

We can use the same $\langle frame\ id \rangle$ for different frames to share named overlay sets. Another possibility offered by the `beanoves` package to share named overlay sets is a fall back mechanism, for example when `X!A` cannot be resolved, `!A` is resolved instead.

⁴See [stackexchange](#) for an alternative that needs at least two passes.

4.6 Resolution command

`\BeanovesResolve` `\BeanovesResolve [setup] {queries}`

This function resolves the `<queries>`, which are like the argument of `?(<...>)` instructions: a comma separated list of single `<query>`'s. It is mainly used for debugging purposes. The optional `<setup>` is a key–value list:

`show` the result is left into the input stream

`in:N=<command>` the result is stored into `<command>`.

5 Support

See the [source repository](#). One can report issues there.

6 Implementation

Identify the internal prefix (L^AT_EX3 DocStrip convention).

1 `<@@=bnvs>`

6.1 Package declarations

```
2 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
3 \ProvidesExplPackage
4   {beanoves}
5   {2025/06/03}
6   {1.0}
7   {Named overlay specifications for beamer}
```

6.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into `beamer` specifications based on this data model.

The development is made easier due to a facility layer over `expl`.

6.3 Facility layer: definitions and naming

In order to make the code shorter and easier to read during development, we add a layer over L^AT_EX3. The `c` and `v` argument specifiers take a slightly different meaning when used in a function which name contains `bnvs` or `BNVS`. Where L^AT_EX3 would transform `l__bnvs_ref_tl` into `\l__bnvs_ref_tl`, `bnvs` will directly transform `ref` into `\l__bnvs_ref_tl`. The type of the local variable used depends on the context and may be `seq` or `int` for example. There are however a pair of exceptions mentioned below. For a better reading experience, “`ref`” will generally stand for `\l__bnvs_ref_tl`, whereas

“path sequence” will generally stand for `\l__bnvs_path_seq`. Other similar shortcuts are used as well.

Functions with BNVS in their names are management functions. They belong to a deeper layer and do not really contain any logic specific to the `beanoves` package.

<code>\BNVS:c</code>	<code>\BNVS:c {⟨cs core name⟩}</code>
<code>\BNVS_l:cn</code>	<code>\BNVS_l:cn {⟨local variable core name⟩} {⟨type⟩}</code>
<code>\BNVS_g:cn</code>	<code>\BNVS_g:cn {⟨global variable core name⟩} {⟨type⟩}</code>

These are naming functions internally used to focus on the discriminating part of variable or function names.

```

8 \cs_new:Npn \BNVS:c #1 { __bnvs_#1 }
9 \cs_new:Npn \BNVS_l:cn #1 #2 { l__bnvs_#1_#2 }
10 \cs_new:Npn \BNVS_g:cn #1 #2 { g__bnvs_#1_#2 }

```

<code>\BNVS_use_raw:N</code>	<code>\BNVS_use_raw:N ⟨cs⟩</code>
<code>\BNVS_use_raw:c</code>	<code>\BNVS_use_raw:c {⟨cs name⟩}</code>
<code>\BNVS_use_raw:Nc</code>	<code>\BNVS_use_raw:Nc ⟨function⟩ {⟨cs name⟩}</code>
<code>\BNVS_use_raw:nc</code>	<code>\BNVS_use_raw:nc {⟨tokens⟩} {⟨cs name⟩}</code>
<code>\BNVS_use:c</code>	<code>\BNVS_use:c {⟨cs core⟩}</code>
<code>\BNVS_use:Nc</code>	<code>\BNVS_use:Nc ⟨function⟩ {⟨cs core⟩}</code>
<code>\BNVS_use:nc</code>	<code>\BNVS_use:nc {⟨tokens⟩} {⟨cs core⟩}</code>

`\BNVS_use_raw:c` is a convenient wrapper over `\use:c`. possibly prepended with some code, for debugging and testing. It needs 3 expansion steps just like `\BNVS_use:c`. The other are used to expand `\use:c` enough before usage by `⟨function⟩` or `⟨tokens⟩`. The first argument of `⟨function⟩` has type N. The next token after `⟨tokens⟩` will have type N too. `⟨cs name⟩` is a full cs name whereas `⟨cs core⟩` will be prepended with the appropriate prefix specific to the `beanoves` package.

```

11 \cs_new:Npn \BNVS_use_raw:N #1 { #1 }
12 \cs_new:Npn \BNVS_use_raw:c #1 {
13   \exp_last_unbraced:No
14   \BNVS_use_raw:N { \cs:w #1 \cs_end: }
15 }

16 \cs_new:Npn \BNVS_use:c #1 {
17   \BNVS_use_raw:c { \BNVS:c { #1 } }
18 }

19 \cs_new:Npn \BNVS_use_raw:NN #1 #2 {
20   #1 #2
21 }

22 \cs_new:Npn \BNVS_use_raw:nN #1 #2 {
23   #1 #2
24 }

25 \cs_new:Npn \BNVS_use_raw:Nc #1 #2 {
26   \exp_args:NNc \BNVS_use_raw:NN #1 { #2 }
27 }

```

```

28 \cs_new:Npn \BNVS_use_raw:nc #1 #2 {
29   \exp_last_unbraced:Nno
30   \BNVS_use_raw:nN { #1 } { \cs:w #2 \cs_end: }
31 }

32 \cs_new:Npn \BNVS_use:Nc #1 #2 {
33   \BNVS_use_raw:Nc #1 { \BNVS:c { #2 } }
34 }

35 \cs_new:Npn \BNVS_use:nc #1 #2 {
36   \BNVS_use_raw:nc { #1 } { \BNVS:c { #2 } }
37 }

38 \tl_new:N \l__bnvs_last_unbraced_tl
39 \cs_new:Npn \BNVS_tl_last_unbraced:nv #1 {
40   \tl_set:Nn \l__bnvs_last_unbraced_tl { #1 }
41   \BNVS_tl_use:nc { \exp_last_unbraced:NV \l__bnvs_last_unbraced_tl }
42 }
43 \cs_new:Npn \BNVS_tl_use:nvv #1 #2 {
44   \BNVS_tl_use:nv { \BNVS_tl_use:nv { #1 } { #2 } }
45 }
46 \cs_new:Npn \BNVS_tl_use:nvvv #1 #2 {
47   \BNVS_tl_use:nvv { \BNVS_tl_use:nv { #1 } { #2 } }
48 }

49 \cs_new:Npn \BNVS_log:n #1 { }
50 \cs_generate_variant:Nn \BNVS_log:n { x }

```

6.3.1 Debug

\BNVS_DEBUG_ON:	\BNVS_DEBUG_ON:
\BNVS_DEBUG_OFF:	\BNVS_DEBUG_OFF:
\BNVS_DEBUG_push:nn	\BNVS_DEBUG_push:nn {<types on>} {<types off>}
\BNVS_DEBUG_pop:	\BNVS_DEBUG_pop:

These functions activate or deactivate the debug mode. They are only active in the `beanoves-debug` package. Manage debug messaging for one given *<type>* or *<types>*. The implementation is not publicly exposed. The *<type>* is a single letter or ******.

6.3.2 Facility layer: Functions

\BNVS_new:cpn	\BNVS_new:cpn is like \cs_new:cpn except that the name argument is tagged for beanoves
\BNVS_set:cpn	package. Similarly for \BNVS_set:cpn.

```

51 \cs_new:Npn \BNVS_new:cpn #1 {
52   \cs_new:cpn { \BNVS:c { #1 } }
53 }

54 \cs_new:Npn \BNVS_set:cpn #1 {
55   \cs_set:cpn { \BNVS:c { #1 } }
56 }

```

```

57 \cs_generate_variant:Nn \cs_generate_variant:Nn { c }
58 \cs_new:Npn \BNVS_generate_variant:cn #1 {
59   \cs_generate_variant:cn { \BNVS:c { #1 } }
60 }

```

6.3.3 logging

\BNVS_warning:n	\BNVS_warning:n {<message>}
\BNVS_warning:x	\BNVS_error:n {<message>}
\BNVS_error:n	\BNVS_fatal:n {<message>}
\BNVS_error:x	Very rudimentary. No long message available for error recovery.
\BNVS_fatal:n	
\BNVS_fatal:x	

```

61 \msg_new:nnn { beanoves } { :n } { #1 }
62 \msg_new:nnn { beanoves } { :nn } { #1~(#2) }

63 \cs_new:Npn \BNVS_warning:n {
64   \msg_warning:nnn { beanoves } { :n }
65 }
66 \cs_new:Npn \BNVS_warning:x {
67   \msg_warning:nnx { beanoves } { :n }
68 }

69 \cs_new:Npn \BNVS_error:n {
70   \msg_error:nnn { beanoves } { :n }
71 }
72 \cs_generate_variant:Nn \BNVS_error:n { x }

73 \cs_new:Npn \BNVS_fatal:n {
74   \msg_fatal:nnn { beanoves } { :n }
75 }
76 \cs_new:Npn \BNVS_fatal:x {
77   \msg_fatal:nnx { beanoves } { :n }
78 }
79 \cs_new:Npn \BNVS_fatal_unreachable: {
80   \BNVS_fatal:n { Unreachable }
81 }
82 \cs_new:Npn \BNVS_fatal_unreachable: { }

```

6.3.4 Facility layer: Variables

\BNVS_N_new:c	\BNVS_N_new:c {<type>}
\BNVS_v_new:c	\BNVS_v_new:c {<type>}

Creates a collection of typed utility functions, see usage below. These functions are undefined when no longer used. <type> is one of `tl`, `seq`...

```

83 \cs_new:Npn \BNVS_N_new:c #1 {

```

```

84  \cs_new:cpn { BNVS_#1:c } ##1 {
85    1 \BNVS:c{ ##1 } \tl_if_empty:nF { ##1 } { _ } #1
86  }
87  \cs_new:cpn { BNVS_#1_new:c } ##1 {
88    \use:c { #1_new:c } { \use:c { BNVS_#1:c } { ##1 } }
89  }
90  \cs_new:cpn { BNVS_#1_use:c } ##1 {
91    \use:c { \cs:w BNVS_#1:c \cs_end: { ##1 } }
92  }
93  \cs_new:cpn { BNVS_#1_use:Nc } ##1 ##2 {
94    \BNVS_use_raw:Nc
95    ##1 { \cs:w BNVS_#1:c \cs_end: { ##2 } }
96  }
97  \cs_new:cpn { BNVS_#1_use:nc } ##1 ##2 {
98    \BNVS_use_raw:nc
99    { ##1 } { \cs:w BNVS_#1:c \cs_end: { ##2 } }
100  }
101 }

102 \cs_new:Npn \BNVS_v_new:c #1 {
103   \cs_new:cpn { BNVS_#1_use:Nv } ##1 ##2 {
104     \BNVS_use_raw:nc
105     { \exp_args:Nv ##1 }
106     { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
107   }
108   \cs_new:cpn { BNVS_#1_use:cv } ##1 ##2 {
109     \BNVS_use_raw:nc
110     { \exp_args:NnV \BNVS_use:c { ##1 } }
111     { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
112   }
113   \cs_new:cpn { BNVS_#1_use:nv } ##1 ##2 {
114     \BNVS_use_raw:nc
115     { \exp_args:NnV \use:n { ##1 } }
116     { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
117   }
118 }

119 \BNVS_N_new:c { bool }
120 \BNVS_N_new:c { int }
121 \BNVS_v_new:c { int }
122 \BNVS_N_new:c { tl }
123 \BNVS_v_new:c { tl }
124 \cs_new:Npn \BNVS_tl_use:Nvv #1 #2 {
125   \BNVS_tl_use:nv { \BNVS_tl_use:Nv #1 { #2 } }
126 }

127 \cs_new:Npn \BNVS_tl_use:Nvvv #1 #2 {
128   \BNVS_tl_use:nvv { \BNVS_tl_use:Nv #1 { #2 } }
129 }

130 \cs_new:Npn \BNVS_tl_use:Nvvvv #1 #2 {
131   \BNVS_tl_use:nvvv { \BNVS_tl_use:Nv #1 { #2 } }
132 }

133 \BNVS_N_new:c { str }
134 \BNVS_v_new:c { str }

```

```

135 \BNVS_N_new:c { seq }
136 \BNVS_v_new:c { seq }
137 \cs_undefine:N \BNVS_N_new:c

```

\BNVS_use:Ncn \BNVS_use:Ncn *<function>* {*<core name>*} {*<type>*}

```

138 \cs_new:Npn \BNVS_use:Ncn #1 #2 #3 {
139   \BNVS_use_raw:c { BNVS_#3_use:Nc }   #1   { #2 }
140 }

141 \cs_new:Npn \BNVS_use:ncn #1 #2 #3 {
142   \BNVS_use_raw:c { BNVS_#3_use:nc } { #1 } { #2 }
143 }

144 \cs_new:Npn \BNVS_use:Nvn #1 #2 #3 {
145   \BNVS_use_raw:c { BNVS_#3_use:Nv }   #1   { #2 }
146 }

147 \cs_new:Npn \BNVS_use:nvn #1 #2 #3 {
148   \BNVS_use_raw:c { BNVS_#3_use:nv } { #1 } { #2 }
149 }

150 \cs_new:Npn \BNVS_use:Ncncn #1 #2 #3 {
151   \BNVS_use:ncn {
152     \BNVS_use:Ncn   #1   { #2 } { #3 }
153   }
154 }

155 \cs_new:Npn \BNVS_use:ncncn #1 #2 #3 {
156   \BNVS_use:ncn {
157     \BNVS_use:ncn { #1 } { #2 } { #3 }
158   }
159 }

160 \cs_new:Npn \BNVS_use:Nvncn #1 #2 #3 {
161   \BNVS_use:ncn {
162     \BNVS_use:Nvn   #1   { #2 } { #3 }
163   }
164 }

165 \cs_new:Npn \BNVS_use:nvncn #1 #2 #3 {
166   \BNVS_use:ncn {
167     \BNVS_use:nvn { #1 } { #2 } { #3 }
168   }
169 }

170 \cs_new:Npn \BNVS_use:Ncncncn #1 #2 #3 #4 #5 {
171   \BNVS_use:ncn {
172     \BNVS_use:Ncncn   #1   { #2 } { #3 } { #4 } { #5 }
173   }
174 }

175 \cs_new:Npn \BNVS_use:ncncncn #1 #2 #3 #4 #5 {
176   \BNVS_use:ncn {
177     \BNVS_use:ncncn { #1 } { #2 } { #3 } { #4 } { #5 }
178   }
179 }

```

\BNVS_new_c:cn \BNVS_new_c:nc {<type>} {<core name>}

```
180 \cs_new:Npn \BNVS_new_c:nc #1 #2 {
181   \BNVS_new_cpn { #1_#2:c } {
182     \BNVS_use_raw:c { BNVS_#1_use:nc } { \BNVS_use_raw:c { #1_#2:N } }
183   }
184 }

185 \cs_new:Npn \BNVS_new_cn:nc #1 #2 {
186   \BNVS_new_cpn { #1_#2:cn } ##1 {
187     \BNVS_use:ncn { \BNVS_use_raw:c { #1_#2:Nn } } { ##1 } { #1 }
188   }
189 }

190 \cs_new:Npn \BNVS_new_cnn:ncN #1 #2 #3 {
191   \BNVS_new_cpn { #2:cnn } ##1 {
192     \BNVS_use:Ncn { #3 } { ##1 } { #1 }
193   }
194 }

195 \cs_new:Npn \BNVS_new_cnn:nc #1 #2 {
196   \BNVS_use_raw:nc {
197     \BNVS_new_cnn:ncN { #1 } { #1_#2 }
198   } { #1_#2:Nnn }
199 }

200 \cs_new:Npn \BNVS_new_cnv:ncN #1 #2 #3 {
201   \BNVS_new_cpn { #2:cnv } ##1 ##2 {
202     \BNVS_tl_use:nv {
203       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
204     }
205   }
206 }

207 \cs_new:Npn \BNVS_new_cnv:nc #1 #2 {
208   \BNVS_use_raw:nc {
209     \BNVS_new_cnv:ncN { #1 } { #1_#2 }
210   } { #1_#2:Nnn }
211 }

212 \cs_new:Npn \BNVS_new_cnx:ncN #1 #2 #3 {
213   \BNVS_new_cpn { #2:cnx } ##1 ##2 {
214     \exp_args:Nnx \use:n {
215       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
216     }
217   }
218 }

219 \cs_new:Npn \BNVS_new_cnx:nc #1 #2 {
220   \BNVS_use_raw:nc {
221     \BNVS_new_cnx:ncN { #1 } { #1_#2 }
222   } { #1_#2:Nnn }
223 }
```

```

224 \cs_new:Npn \BNVS_new_cc:ncNn #1 #2 #3 #4 {
225   \BNVS_new:cpn { #2:cc } ##1 ##2 {
226     \BNVS_use:Ncncn #3 { ##1 } { #1 } { ##2 } { #4 }
227   }
228 }

229 \cs_new:Npn \BNVS_new_cc:ncn #1 #2 {
230   \BNVS_use_raw:nc {
231     \BNVS_new_cc:ncNn { #1 } { #1_#2 }
232   } { #1_#2:NN }
233 }

234 \cs_new:Npn \BNVS_new_cc:nc #1 #2 {
235   \BNVS_new_cc:ncn { #1 } { #2 } { #1 }
236 }

237 \cs_new:Npn \BNVS_new_cn:ncNn #1 #2 #3 #4 {
238   \BNVS_new:cpn { #2:cn } ##1 {
239     \BNVS_use:Ncn #3 { ##1 } { #1 }
240   }
241 }

242 \cs_new:Npn \BNVS_new_cn:ncn #1 #2 {
243   \BNVS_use_raw:nc {
244     \BNVS_new_cn:ncNn { #1 } { #1_#2 }
245   } { #1_#2:Nn }
246 }

247 \cs_new:Npn \BNVS_new_cv:ncNn #1 #2 #3 #4 {
248   \BNVS_new:cpn { #2:cv } ##1 ##2 {
249     \BNVS_use:nvn {
250       \BNVS_use:Ncn #3 { ##1 } { #1 }
251     } { ##2 } { #4 }
252   }
253 }

254 \cs_new:Npn \BNVS_new_cv:ncn #1 #2 {
255   \BNVS_use_raw:nc {
256     \BNVS_new_cv:ncNn { #1 } { #1_#2 }
257   } { #1_#2:Nn }
258 }

259 \cs_new:Npn \BNVS_new_cv:nc #1 #2 {
260   \BNVS_new_cv:ncn { #1 } { #2 } { #1 }
261 }

262 \cs_new:Npn \BNVS_l_use:Ncn #1 #2 #3 {
263   \BNVS_use_raw:Nc #1 { \BNVS_l:cn { #2 } { #3 } }
264 }

265 \cs_new:Npn \BNVS_l_use:ncn #1 #2 #3 {
266   \BNVS_use_raw:nc { #1 } { \BNVS_l:cn { #2 } { #3 } }
267 }

```



```

268 \cs_new:Npn \BNVS_g_use:Ncn #1 #2 #3 {
269   \BNVS_use_raw:Nc    #1    { \BNVS_g:cn { #2 } { #3 } }
270 }

271 \cs_new:Npn \BNVS_g_use:ncn #1 #2 #3 {
272   \BNVS_use_raw:nc { #1 } { \BNVS_g:cn { #2 } { #3 } }
273 }

```

```

\BNVS_new_conditional:cpnn \BNVS_new_conditional:cpnn {<core>} <parameter> {<conditions>} {<code>}

```

```

274 \cs_generate_variant:Nn \prg_new_conditional:Npnn { c }

275 \cs_new:Npn \BNVS_new_conditional:cpnn #1 {
276   \prg_new_conditional:cpnn { \BNVS:c { #1 } }
277 }
278 \cs_generate_variant:Nn \prg_generate_conditional_variant:Nnn { c }
279 \cs_new:Npn \BNVS_generate_conditional_variant:cnn #1 {
280   \prg_generate_conditional_variant:cnn { \BNVS:c { #1 } }
281 }

282 \cs_new:Npn \BNVS_new_conditional_vn:cNnn #1 #2 #3 #4 {
283   \BNVS_new_conditional:cpnn { #1:vn } ##1 ##2 { #4 } {
284     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
285       \prg_return_true:
286     } {
287       \prg_return_false:
288     }
289   }
290 }

291 \cs_new:Npn \BNVS_new_conditional_vn:cnn #1 #2 {
292   \BNVS_use:nc {
293     \BNVS_new_conditional_vn:cNnn { #1 }
294   } { #1:nn TF } { #2 }
295 }

296 \cs_new:Npn \BNVS_new_conditional_vc:cNnn #1 #2 #3 #4 {
297   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #4 } {
298     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
299       \prg_return_true:
300     } {
301       \prg_return_false:
302     }
303   }
304 }

305 \cs_new:Npn \BNVS_new_conditional_vc:cnn #1 {
306   \BNVS_use:nc {
307     \BNVS_new_conditional_vc:cNnn { #1 }
308   } { #1:ncTF }
309 }

```

```

310 \cs_new:Npn \BNVS_new_conditional_vvc:cNnnn #1 #2 #3 #4 #5 {
311   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #5 } {
312     \BNVS_use:nvn {
313       \BNVS_use:Nvn #2 { ##1 } { #3 }
314     } { ##2 } { #4 } { ##3 } {
315       \prg_return_true:
316     } {
317       \prg_return_false:
318     }
319   }
320 }

321 \cs_new:Npn \BNVS_new_conditional_vvc:cnnn #1 {
322   \BNVS_use:nc {
323     \BNVS_new_conditional_vvc:cNnnn { #1 }
324   } { #1:nncTF }
325 }

326 \cs_new:Npn \BNVS_new_conditional_vc:cNn #1 #2 #3 {
327   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #3 } {
328     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } {
329       \prg_return_true:
330     } {
331       \prg_return_false:
332     }
333   }
334 }

335 \cs_new:Npn \BNVS_new_conditional_vc:cn #1 {
336   \BNVS_use:nc {
337     \BNVS_new_conditional_vc:cNn { #1 }
338   } { #1:ncTF }
339 }

340 \cs_new:Npn \BNVS_new_conditional_vvc:cNn #1 #2 #3 {
341   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #3 } {
342     \BNVS_tl_use:nv {
343       \BNVS_tl_use:Nv #2 { ##1 }
344     } { ##2 } { ##3 } {
345       \prg_return_true:
346     } {
347       \prg_return_false:
348     }
349   }
350 }

351 \cs_new:Npn \BNVS_new_conditional_vvc:cn #1 {
352   \BNVS_use:nc {
353     \BNVS_new_conditional_vvc:cNn { #1 }
354   } { #1:nncTF }
355 }

```

6.3.5 Regex

```

356 \cs_new:Npn \BNVS_regex_use:Nc #1 #2 {
357   \BNVS_use_raw:Nc #1 { c \BNVS:c { #2 } _regex }
358 }

```

6.3.6 Token lists

<u>__bnvs_tl_clear:c</u>	__bnvs_tl_clear:c {<core key tl>}
<u>__bnvs_tl_use:c</u>	__bnvs_tl_use:c {<core>}
<u>__bnvs_tl_set_eq:cc</u>	__bnvs_tl_count:c {<core>}
<u>__bnvs_tl_set:cn</u>	__bnvs_tl_set_eq:cc {<lhs core name>} {<rhs core name>}
<u>__bnvs_tl_set:(cv cx ce)</u>	__bnvs_tl_set:cn {<core>} {<tl>}
<u>__bnvs_tl_put_left:cn</u>	__bnvs_tl_set:cv {<core>} {<value core name>}
<u>__bnvs_tl_put_right:cn</u>	__bnvs_tl_put_left:cn {<core>} {<tl>}
<u>__bnvs_tl_put_right:(cx ce cv)</u>	__bnvs_tl_put_right:cn {<core>} {<tl>}
	__bnvs_tl_put_right:cv {<core>} {<value core name>}

These are shortcuts to

- \tl_clear:c {l__bnvs_<core>_tl}
- \tl_use:c {l__bnvs_<core>_tl}
- \tl_set_eq:cc {l__bnvs_<lhs core>_tl}{l__bnvs_<rhs core>_tl}
- \tl_set:cv {l__bnvs_<core>_tl}{l__bnvs_<value core>_tl}
- \tl_set:cx {l__bnvs_<core>_tl}{<tl>}
- \tl_put_left:cn {l__bnvs_<core>_tl}{<tl>}
- \tl_put_right:cn {l__bnvs_<core>_tl}{<tl>}
- \tl_put_right:cv {l__bnvs_<core>_tl}{l__bnvs_<value core>_tl}

<u>\BNVS_new_conditional_vnc:cn</u>	\BNVS_new_conditional_vnc:cn {<core>} {<conditions>}
-------------------------------------	--

Creates <core>:vncTF from <core>:nncTF.

```

359 \cs_new:Npn \BNVS_new_conditional_vnc:cNn #1 #2 #3 {
360   \BNVS_new_conditional:cpnn { #1:vnc } ##1 ##2 ##3 { #3 } {
361     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } { ##3 } {
362       \prg_return_true:
363     } {
364       \prg_return_false:
365     }
366   }
367 }

```

```

368 \cs_new:Npn \BNVS_new_conditional_vnc:cn #1 {
369   \BNVS_use:nc {
370     \BNVS_new_conditional_vnc:cNn { #1 }
371   } { #1:nnctf }
372 }

```

\BNVS_new_conditional_vvnc:cn \BNVS_new_conditional_vvnc:cn {<core>} {<conditions>}

Creates <core>:vvncTF from <core>:nnctf.

```

373 \cs_new:Npn \BNVS_new_conditional_vvnc:cNn #1 #2 #3 {
374   \BNVS_new_conditional:cpnn { #1:vvnc } ##1 ##2 ##3 ##4 { #3 } {
375     \BNVS_tl_use:nv {
376       \BNVS_tl_use:Nv #2 { ##1 }
377     } { ##2 } { ##3 } { ##4 } {
378       \prg_return_true:
379     } {
380       \prg_return_false:
381     }
382   }
383 }

384 \cs_new:Npn \BNVS_new_conditional_vvnc:cn #1 {
385   \BNVS_use:nc {
386     \BNVS_new_conditional_vvnc:cNn { #1 }
387   } { #1:nnctf }
388 }

389 \cs_new:Npn \BNVS_new_conditional_vvvc:cNn #1 #2 #3 {
390   \BNVS_new_conditional:cpnn { #1:vvvc } ##1 ##2 ##3 ##4 { #3 } {
391     \BNVS_tl_use:nvv {
392       \BNVS_tl_use:Nv #2 { ##1 }
393     } { ##2 } { ##3 } { ##4 } {
394       \prg_return_true:
395     } {
396       \prg_return_false:
397     }
398   }
399 }

```

\BNVS_new_conditional_vvvc:cn \BNVS_new_conditional_vvvc:cn {<core>} {<conditions>}

Creates <core>:vvvcTF from <core>:nnctf.

```

400 \cs_new:Npn \BNVS_new_conditional_vvvc:cn #1 {
401   \BNVS_use:nc {
402     \BNVS_new_conditional_vvvc:cNn { #1 }
403   } { #1:nnctf }
404 }

```

```

405 \cs_new:Npn \BNVS_new_tl_c:c {
406   \BNVS_new_c:nc { tl }
407 }
408 \BNVS_new_tl_c:c { clear }
409 \BNVS_new_tl_c:c { use }
410 \BNVS_new_tl_c:c { count }

411 \BNVS_new:cpn { tl_set_eq:cc } #1 #2 {
412   \BNVS_use:ncncn { \tl_set_eq:NN } { #1 } { tl } { #2 } { tl }
413 }

414 \cs_new:Npn \BNVS_new_tl_cn:c {
415   \BNVS_new_cn:nc { tl }
416 }

417 \cs_new:Npn \BNVS_new_tl_cv:c #1 {
418   \BNVS_new_cv:ncn { tl } { #1 } { tl }
419 }
420 \BNVS_new_tl_cn:c { set }
421 \BNVS_new_tl_cv:c { set }

422 \BNVS_new:cpn { tl_set:cx } {
423   \exp_args:Nnx \__bnvs_tl_set:cn
424 }

425 \BNVS_new:cpn { tl_set:ce } {
426   \exp_args:Nne \__bnvs_tl_set:cn
427 }
428 \BNVS_new_tl_cn:c { put_right }
429 \BNVS_new_tl_cv:c { put_right }
430 % \BNVS_generate_variant:cn { tl_put_right:cn } { cx }

431 \BNVS_new:cpn { tl_put_right:cx } {
432   \exp_args:Nnnx \BNVS_use:c { tl_put_right:cn }
433 }

434 \BNVS_new:cpn { tl_put_right:ce } {
435   \exp_args:Nnne \BNVS_use:c { tl_put_right:cn }
436 }
437 \BNVS_new_tl_cn:c { put_left }
438 \BNVS_new_tl_cv:c { put_left }
439 % \BNVS_generate_variant:cn { tl_put_left:cn } { cx }

440 \BNVS_new:cpn { tl_put_left:cx } {
441   \exp_args:Nnnx \BNVS_use:c { tl_put_left:cn }
442 }

```

<u>__bnvs_tl_if_empty:cTF</u>	<u>__bnvs_tl_if_empty:cTF</u>	{\core}	{\yes:}	{\no:}
<u>__bnvs_tl_if_blank:vTF</u>	<u>__bnvs_tl_if_blank:vTF</u>	{\core}	{\yes:}	{\no:}
<u>__bnvs_tl_if_eq:cnTF</u>	<u>__bnvs_tl_if_eq:cnTF</u>	{\core}	{\tl}	{\yes:} {\no:}

These are shortcuts to

- \tl_if_empty:cTF {l__bnvs_<core>_tl} {\yes:} {\no:}
- \tl_if_eq:cnTF {l__bnvs_<core>_tl}{\tl} {\yes:} {\no:}

```

443 \cs_new:Npn \BNVS_new_conditional_c:ncNn #1 #2 #3 #4 {
444   \BNVS_new_conditional:cpnn { #2 } ##1 { #4 } {
445     \BNVS_use:Ncn #3 { ##1 } { #1 } {
446       \prg_return_true:
447     } {
448       \prg_return_false:
449     }
450   }
451 }

452 \cs_new:Npn \BNVS_new_conditional_c:ncn #1 #2 {
453   \BNVS_use_raw:nc {
454     \BNVS_new_conditional_c:ncNn { #1 } { #1_#2:c }
455   } { #1_#2:NTF }
456 }
457 \BNVS_new_conditional_c:ncn { t1 } { if_empty } { p, T, F, TF }

458 \BNVS_new_conditional:cpnn { t1_if_blank:v } #1 { T, F, TF } {
459   \BNVS_t1_use:Nv \t1_if_blank:nTF { #1 } {
460     \prg_return_true:
461   } {
462     \prg_return_false:
463   }
464 }

465 \cs_new:Npn \BNVS_new_conditional_cn:ncNn #1 #2 #3 #4 {
466   \BNVS_new_conditional:cpnn { #2:cn } ##1 ##2 { #4 } {
467     \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } {
468       \prg_return_true:
469     } {
470       \prg_return_false:
471     }
472   }
473 }

474 \cs_new:Npn \BNVS_new_conditional_cn:ncn #1 #2 {
475   \BNVS_use_raw:nc {
476     \BNVS_new_conditional_cn:ncNn { #1 } { #1_#2 }
477   } { #1_#2:NnTF }
478 }
479 \BNVS_new_conditional_cn:ncn { t1 } { if_eq } { T, F, TF }

480 \cs_new:Npn \BNVS_new_conditional_cv:ncNn #1 #2 #3 #4 {
481   \BNVS_new_conditional:cpnn { #2:cv } ##1 ##2 { #4 } {
482     \BNVS_use:nvn {
483       \BNVS_use:Ncn #3 { ##1 } { #1 }
484     } { ##2 } { #1 } {
485       \prg_return_true:
486     } {
487       \prg_return_false:
488     }
489   }
490 }

```

```

491 \cs_new:Npn \BNVS_new_conditional_cv:ncn #1 #2 {
492   \BNVS_use_raw:nc {
493     \BNVS_new_conditional_cv:ncNn { #1 } { #1_#2 }
494   } { #1_#2:NnTF }
495 }
496 \BNVS_new_conditional_cv:ncn { t1 } { if_eq } { T, F, TF }

497 \BNVS_new:cpn { gput_right:cccn } #1 #2 #3 {
498   \tl_gput_right:cn { \_bnvs_g_IPK:w { #1 } { #2 } { #3 } }
499 }

```

6.3.7 Strings

```

\_bnvs_str_if_eq:vvTF \_bnvs_str_if_eq:vvTF {<core>} {<t1>} {<yes:>} {<no:>}

```

These are shortcuts to

- \str_if_eq:ccTF {l_bnvs_<core>_t1}{<yes:>} {<no:>}

```

500 \cs_new:Npn \BNVS_new_conditional_vv:cNn #1 #2 #3 {
501   \BNVS_new_conditional:cpnn { #1:vv } ##1 ##2 { #3 } {
502     \BNVS_tl_use:nv {
503       \BNVS_tl_use:Nv #2 { ##1 }
504     } { ##2 } {
505       \prg_return_true:
506     } {
507       \prg_return_false:
508     }
509   }
510 }

511 \cs_new:Npn \BNVS_new_conditional_vv:cn #1 {
512   \BNVS_use:nc {
513     \BNVS_new_conditional_vvnc:cNn { #1 }
514   } { #1:nnTF }
515 }

516 \cs_new:Npn \BNVS_new_conditional_vn:ncNn #1 #2 #3 #4 {
517   \BNVS_new_conditional:cpnn { #2:vn } ##1 ##2 { #4 } {
518     \BNVS_use:Nvn #3 { ##1 } { #1 } { ##2 } {
519       \prg_return_true:
520     } {
521       \prg_return_false:
522     }
523   }
524 }

525 \cs_new:Npn \BNVS_new_conditional_vn:ncn #1 #2 {
526   \BNVS_use_raw:nc {
527     \BNVS_new_conditional_vn:ncNn { #1 } { #1_#2 }
528   } { #1_#2:nnTF }
529 }
530 \BNVS_new_conditional_vn:ncn { str } { if_eq } { T, F, TF }

```

```

531 \cs_new:Npn \BNVS_new_conditional_vv:ncNn #1 #2 #3 #4 {
532   \BNVS_new_conditional:cpnn { #2:vv } ##1 ##2 { #4 } {
533     \BNVS_use:nvn {
534       \BNVS_use:Nvn #3 { ##1 } { #1 }
535     } { ##2 } { #1 } {
536       \prg_return_true:
537     } {
538       \prg_return_false:
539     }
540   }
541 }

542 \cs_new:Npn \BNVS_new_conditional_vv:ncn #1 #2 {
543   \BNVS_use_raw:nc {
544     \BNVS_new_conditional_vv:ncNn { #1 } { #1_#2 }
545   } { #1_#2:nnTF }
546 }
547 \BNVS_new_conditional_vv:ncn { str } { if_eq } { T, F, TF }

```

6.3.8 Sequences

<code>__bnvs_seq_count:c</code>	<code>__bnvs_seq_new:c {<core>}</code>
<code>__bnvs_seq_clear:c</code>	<code>__bnvs_seq_count:c {<core>}</code>
<code>__bnvs_seq_set_eq:cc</code>	<code>__bnvs_seq_clear:c {<core>}</code>
<code>__bnvs_seq_gset_eq:cc</code>	<code>__bnvs_seq_set_eq:cc {<core₁>} {<core₂>}</code>
<code>__bnvs_seq_use:cn</code>	<code>__bnvs_seq_use:cn {<core>} {<separator>}</code>
<code>__bnvs_seq_item:cn</code>	<code>__bnvs_seq_item:cn {<core>} {<integer expression>}</code>
<code>__bnvs_seq_remove_all:cn</code>	<code>__bnvs_seq_remove_all:cn {<core>} {<tl>}</code>
<code>__bnvs_seq_put_left:cv</code>	<code>__bnvs_seq_put_right:cn {<seq core>} {<tl>}</code>
<code>__bnvs_seq_put_right:cn</code>	<code>__bnvs_seq_put_right:cv {<seq core>} {<tl core>}</code>
<code>__bnvs_seq_put_right:cv</code>	<code>__bnvs_seq_set_split:cnn {<seq core>} {<tl>} {<separator>}</code>
<code>__bnvs_seq_set_split:cnn</code>	<code>__bnvs_seq_pop_left:cc {<core₁>} {<core₂>}</code>
<code>__bnvs_seq_set_split:(cnv cnx)</code>	
<code>__bnvs_seq_pop_left:cc</code>	

These are shortcuts to

- `\seq_set_eq:cc {l__bnvs_<core1>_seq} {l__bnvs_<core2>_seq}`
- `\seq_count:c {l__bnvs_<core>_seq}`
- `\seq_use:cn {l__bnvs_<core>_seq}{<separator>}`
- `\seq_item:cn {l__bnvs_<core>_seq}{<integer expression>}`
- `\seq_remove_all:cn {l__bnvs_<core>_seq}{<tl>}`
- `\__bnvs_seq_clear:c {l__bnvs_<core>_seq}`
- `\seq_put_right:cv {l__bnvs_<seq core>_seq} {l__bnvs_<tl core>_tl}`
- `\seq_set_split:cnn{l__bnvs_<seq core>_seq}{l__bnvs_<tl core>_tl}{<tl>}`


```

548 \BNVS_new_c:nc { seq } { count }
549 \BNVS_new_c:nc { seq } { clear }
550 \BNVS_new_cn:nc { seq } { use }
551 \BNVS_new_cn:nc { seq } { item }
552 \BNVS_new_cn:nc { seq } { remove_all }
553 \BNVS_new_cn:nc { seq } { map_inline }
554 \BNVS_new_cc:nc { seq } { set_eq }
555 \BNVS_new_cc:nc { seq } { gset_eq }
556 \BNVS_new_cv:ncn { seq } { put_left } { t1 }
557 \BNVS_new_cn:ncn { seq } { put_right } { t1 }
558 \BNVS_new_cv:ncn { seq } { put_right } { t1 }
559 \BNVS_new_cnn:nc { seq } { set_split }
560 \BNVS_new_cnv:nc { seq } { set_split }
561 \BNVS_new_cnx:nc { seq } { set_split }
562 \BNVS_new_cc:ncn { seq } { pop_left } { t1 }
563 \BNVS_new_cc:ncn { seq } { pop_right } { t1 }

```

```

\underline{\_bnvs_seq_if_empty:cTF} \_bnvs_seq_if_empty:cTF {<seq core>} {<yes:>} {<no:>}
\underline{\_bnvs_seq_get_right:ccTF} \_bnvs_seq_get_right:ccTF {<seq core>} {<t1 core>} {<yes:>} {<no:>}
\underline{\_bnvs_seq_pop_left:ccTF}
\underline{\_bnvs_seq_pop_right:ccTF}

```

```

564 \cs_new:Npn \BNVS_new_conditional_cc:ncnn #1 #2 #3 #4 {
565   \BNVS_new_conditional:cpnn { #1_#2:cc } ##1 ##2 { #4 } {
566     \BNVS_use:ncncn {
567       \BNVS_use_raw:c { #1_#2:NNTF }
568     } { ##1 } { #1 } { ##2 } { #3 } {
569       \prg_return_true:
570     } {
571       \prg_return_false:
572     }
573   }
574 }
575 \BNVS_new_conditional_c:ncn { seq } { if_empty } { T, F, TF }
576 \BNVS_new_conditional_cc:ncnn
577 { seq } { get_right } { t1 } { T, F, TF }
578 \BNVS_new_conditional_cc:ncnn
579 { seq } { pop_left } { t1 } { T, F, TF }
580 \BNVS_new_conditional_cc:ncnn
581 { seq } { pop_right } { t1 } { T, F, TF }

```

6.3.9 Integers

```

\__bnvs_int_new:c \__bnvs_int_new:c {\core}
\__bnvs_int_use:c \__bnvs_int_use:c {\core}
\__bnvs_int_zero:c \__bnvs_int_incr:c {\core}
\__bnvs_int_inc:c \__bnvs_int_decr:c {\core}
\__bnvs_int_decr:c \__bnvs_int_set:cn {\core} {\value}
\__bnvs_int_set:cn
\__bnvs_int_set:cv

```

These are shortcuts to

- `\int_new:c` `{l__bnvs_<core>_int}`
- `\int_use:c` `{l__bnvs_<core>_int}`
- `\int_incr:c` `{l__bnvs_<core>_int}`
- `\int_decr:c` `{l__bnvs_<core>_int}`
- `\int_set:cn` `{l__bnvs_<core>_int} <value>`

```

582 \BNVS_new_c:nc { int } { new }
583 \BNVS_new_c:nc { int } { use }
584 \BNVS_new_c:nc { int } { zero }
585 \BNVS_new_c:nc { int } { incr }
586 \BNVS_new_c:nc { int } { decr }
587 \BNVS_new_cn:nc { int } { set }
588 \BNVS_new_cv:ncn { int } { set } { int }

```

6.4 Debug facilities

Typesetting file `beanoves.dtx` creates both `beanoves` and `beanoves-debug` style files. The former is intended for everyday use whereas the latter contains supplemental debugging and testing facilities which are intentionally left undocumented. For example, we have aliases for `\group_begin:` and `\group_end:` to allow the display of supplemental informations while debugging. We also have some techniques for coverage as well as inline testing.

6.5 Local variables

We make heavy use of local variables and function scopes. Many functions are executed within a `TEX` group, which ensures no name collision with the caller stack. The number of variables used has not been optimized, nor the `TEX` groups used. Optimization often goes against readability. Next local variables are used by different functions. These are one word letter variables.

`\l__bnvs_I_tl`: A frame identifier

`\l__bnvs_J_tl`: The last frame identifier

`\l__bnvs_K_tl`: ! as regex caught group, if non empty.

`\l__bnvs_S_tl`: A short name, a regex caught group.

`\l__bnvs_P_tl`: A path, a regex caught group.

`\l__bnvs_N_tl`: The representation of an unsigned decimal integer.

```
589 \tl_new:N \l__bnvs_J_tl
590 \tl_new:N \l__bnvs_I_tl
591 \tl_new:N \l__bnvs_K_tl
592 \tl_new:N \l__bnvs_S_tl
593 \tl_new:N \l__bnvs_P_tl
594 \tl_new:N \l__bnvs_D_tl
595 \tl_new:N \l__bnvs_B_tl
596 \tl_new:N \l__bnvs_N_tl
597 \tl_new:N \l__bnvs_T_tl
598 \tl_new:N \l__bnvs_U_tl
599 \tl_new:N \l__bnvs_V_tl
600 \tl_new:N \l__bnvs_A_tl
601 \tl_new:N \l__bnvs_C_tl
602 \tl_new:N \l__bnvs_F_tl
603 \tl_new:N \l__bnvs_L_tl
604 \tl_new:N \l__bnvs_Z_tl
605 \tl_new:N \l__bnvs_Q_tl
606 \tl_new:N \l__bnvs_a_tl
607 \tl_new:N \l__bnvs_z_tl
608 \tl_new:N \l__bnvs_l_tl
609 \tl_new:N \l__bnvs_r_tl
610 \tl_new:N \l__bnvs_n_tl
611 \tl_new:N \l__bnvs_ref_tl
612 \tl_new:N \l__bnvs_v_tl
613 \tl_new:N \l__bnvs_b_tl
614 \tl_new:N \l__bnvs_c_tl
615 \tl_new:N \l__bnvs_ans_tl
616 \tl_new:N \l__bnvs_base_tl
617 \tl_new:N \l__bnvsroup_tl
618 \tl_new:N \l__bnvs_scan_tl
619 \tl_new:N \l__bnvs_query_tl
620 \tl_new:N \l__bnvs_n_incr_tl
621 \tl_new:N \l__bnvs_incr_tl
622 \tl_new:N \l__bnvs_plus_tl
623 \tl_new:N \l__bnvs_rhs_tl
624 \tl_new:N \l__bnvs_post_tl
625 \tl_new:N \l__bnvs_suffix_tl
626 \tl_new:N \l__bnvs_index_tl
627 \int_new:N \g__bnvs_call_int
628 \int_new:N \l__bnvs_i_int
629 \seq_new:N \g__bnvs_def_seq
630 \seq_new:N \l__bnvs_a_seq
631 \seq_new:N \l__bnvs_b_seq
632 \seq_new:N \l__bnvs_ans_seq
633 \seq_new:N \l__bnvs_Q_seq
634 \seq_new:N \l__bnvs_R_seq
635 \seq_new:N \l__bnvs_match_seq
636 \seq_new:N \l__bnvs_split_seq
637 \seq_new:N \l__bnvs_P_seq
638 \bool_new:N \l__bnvs_parse_bool
```

```

639 \bool_set_false:N \l__bnvs_parse_bool
640 \bool_new:N \l__bnvs_deep_bool
641 \bool_set_false:N \l__bnvs_deep_bool
642 \bool_new:N \l__bnvs_no_range_bool
643 \bool_set_false:N \l__bnvs_no_range_bool

```

```

644 \BNVS_new:cpn { tl_clear_ans: } {
645   \__bnvs_tl_clear:c { ans }
646 }

```

```

647 \cs_new:Npn \BNVS_error_ans:x {
648   \__bnvs_tl_put_right:cn { ans } 0
649   \BNVS_error:x
650 }

```

In order to implement the provide feature, we add getters and setters

```

651 \BNVS_new:cpn { set_true:c } #1 {
652   \exp_args:Nc \bool_set_true:N { \BNVS_1:cn { #1 } { bool } }
653 }

```

```

654 \BNVS_new:cpn { set_false:c } #1 {
655   \exp_args:Nc \bool_set_false:N { \BNVS_1:cn { #1 } { bool } }
656 }

```

6.6 Infinite loop management

Unending recursivity is managed here.

`\g__bnvs_call_int` Some functions calls, as well as some loop bodies, decrement this counter. When this counter reaches 0, an error is raised or a computation is aborted.

(End of definition for `\g__bnvs_call_int`.)

```

657 \int_const:Nn \c__bnvs_max_call_int { 8192 }

```

`\__bnvs_greset_call:` `\__bnvs_greset_call:`

Reset globally the call stack counter to its maximum value.

```

658 \BNVS_new:cpn { greset_call: } {
659   \int_gset:Nn \g__bnvs_call_int { \c__bnvs_max_call_int }
660 }

```

`\__bnvs_if_call:TF` `\__bnvs_call_do:TF` `{⟨yes:⟩}` `{⟨no:⟩}`

Decrement the `\g__bnvs_call_int` counter globally and execute `⟨yes:⟩` if we have not reached 0, `⟨no:⟩` otherwise.

```

661 \BNVS_new_conditional:cpnn { if_call: } { T, F, TF } {
662   \int_gdecr:N \g__bnvs_call_int
663   \int_compare:nNnTF \g__bnvs_call_int > 0 {
664     \prg_return_true:
665   } {
666     \prg_return_false:
667   }
668 }

```

6.7 Overlay specification

6.7.1 Registration

We keep track of the $\langle id \rangle! \langle path \rangle$ combinations and provide looping mechanisms. Hereafter, $\langle id \rangle$, $\langle path \rangle$ and $\langle key \rangle$ allways represent pure strings.

```

__bnvs_g_IPK:w  __bnvs_g_IPK:w {<id>} {<path>} {<key>}
__bnvs_g_IP:w   __bnvs_g_IP:w  {<id>} {<path>}
__bnvs_g_I:w    __bnvs_g_I:w   {<id>}

```

Create a unique global name from the arguments.

```

669 \BNVS_new:cpn { g_IPK:w } #1 #2 #3 { g__bnvs@#1!#2/#3_tl }
670 \BNVS_new:cpn { g_IP:w } #1 #2 { g__bnvs@#1!#2_keys_seq }
671 \BNVS_new:cpn { g_I:w } #1 { g__bnvs@#1_paths_seq }

```

$\backslash g_bnvs_I_seq$ Global list of globally registered identifiers.

(End of definition for $\backslash g_bnvs_I_seq$.)

```

672 \seq_new:N \g__bnvs_I_seq

```

6.7.2 Data model

The whole data model is somehow similar to a hierarchical file system: the $\langle id \rangle$ corresponds to the drive, the $\langle path \rangle$ points to a directory, each directory may contain individual files. File named “=” contains initial data whereas file named “@” contains the static resolved data. The latter is initially undefined and becomes empty as soon as the initial data is defined or set. In order to allow index override, files respectively named “ $\langle N \rangle$ ” and “ $@ \langle N \rangle$ ” are used similarly, $\langle N \rangle$ denoting a normalized integer (no leading 0, no leading + sign). The dot (.) is used to separate components of $\langle path \rangle$.

6.7.3 Low level IPK model functions

These are **gset** and **gunset** functions. Each **gset** function must be balanced by a **gunset** function to prevent “memory leaks”.

```

__bnvs_gset:nnnn  __bnvs_gset:nnnn {<id>} {<path>} {<key>} {<def>}
__bnvs_gset:(nnnv|nnne|vnvn|nvnn|nvvn|vvnn|vvnv)

```

Manage the storage. Keep track of all the $\langle path \rangle$ ’s for a given $\langle id \rangle$ and all the $\langle key \rangle$ ’s for a given $\langle id \rangle! \langle path \rangle$ combination.

Implementation details, next macros are defined lazily at the global level:

- $\backslash_bnvs_gset@ \langle id \rangle! \langle path \rangle:nn \{ \langle key \rangle \} \{ \langle def \rangle \}$,
- $\backslash_bnvs_gset@ \langle id \rangle:nnn \{ \langle path \rangle \} \{ \langle key \rangle \} \{ \langle def \rangle \}$,
- $\backslash g_bnvs@ \langle id \rangle!_paths_tl$, records all $\langle path \rangle$ for $\langle id \rangle$,
- $\backslash g_bnvs@ \langle id \rangle! \langle path \rangle_keys_tl$, records all $\langle key \rangle$ for $\langle id \rangle! \langle path \rangle$,
- $\backslash g_bnvs@ \langle id \rangle! \langle path \rangle_keys_tl$,

- `\g__bnvs@<id>!<path>/<key>_tl`, record the `<def>` for the `<id>!<path>/<key>` combination.

```

673 \BNVS_new:cpn { gset:Nn } #1 {
674   \tl_gclear_new:N #1
675   \tl_gset:Nn #1
676 }
677 \BNVS_new:cpn { gset_IPKV:NNw } #1 #2 #3 #4 {
678   \seq_gclear_new:N #1
679   \cs_new:Npn #2 ##1 ##2 {
680     \seq_if_in:NnF #1 { ##1 } {
681       \seq_gput_right:Nn #1 { ##1 }
682     }
683     \exp_args:Nc \__bnvs_gset:Nn {
684       \__bnvs_g_IPK:w { #3 } { #4 } { ##1 }
685     } { ##2 }
686   }
687   #2
688 }
689 \BNVS_new:cpn { gset:NNnnnn } #1 #2 #3 #4 {
690   \cs_if_exist_use:NF #1 {
691     \seq_gput_right:Nn \g__bnvs_I_seq { #3 }
692     \seq_gclear_new:N #2
693     \cs_new:Npn #1 ##1 {
694       \seq_if_in:NnF #2 { ##1 } {
695         \seq_gput_right:Nn #2 { ##1 }
696       }
697       \exp_args:Ncc \__bnvs_gset_IPKV:NNw
698       { \__bnvs_g_IP:w { #3 } { ##1 } }
699       { __bnvs_gset@#3!##1:nn } { #3 } { ##1 }
700     }
701     #1
702   }
703   { #4 }
704 }
705 \BNVS_new:cpn { gset:nnnn } #1 #2 #3 #4 {
706   \cs_if_exist_use:cF { __bnvs_gset@#1!#2:nn } {
707     \exp_args:Ncc \__bnvs_gset:NNnnnn
708     { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
709   } { #3 } { #4 }
710 }
711 \BNVS_new:cpn { gset:vnnn } {
712   \BNVS_tl_use:cv { gset:nnnn }
713 }
714 \BNVS_new:cpn { gset:nvnn } #1 {
715   \BNVS_tl_use:nv { \__bnvs_gset:nnnn { #1 } }
716 }
717 \BNVS_new:cpn { gset:vvnn } {
718   \BNVS_tl_use:Nvv \__bnvs_gset:nnnn
719 }
720 \BNVS_new:cpn { gset:nnnv } #1 #2 #3 {
721   \BNVS_tl_use:nv {
722     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
723   }

```

```

724 }
725 \BNVS_new:cpn { gset:vvnv } #1 #2 #3 {
726   \BNVS_tl_use:nv {
727     \__bnvs_gset:vvnn { #1 } { #2 } { #3 }
728   }
729 }
730 \BNVS_new:cpn { gset:nvnv } {
731   \BNVS_tl_use:Nv \__bnvs_gset:nnnv
732 }
733 \cs_generate_variant:Nn \__bnvs_gset:nnnn { nnne }

```

__bnvs_gset:nnv __bnvs_gset:nnv {<id>} {<path>} {<key>}

Syntactic sugar for __bnvs_gset:nnnv {<id>} {<path>} {<key>} {<key>}.

```

734 \BNVS_new:cpn { gset:nnv } #1 #2 #3 {
735   \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { #3 }
736 }

```

__bnvs_gunset:nnn __bnvs_gunset:nnv {<id>} {<path>} {<key>}

__bnvs_gunset:vvnn Removes the specifications for the <id>!
<path>/<key> combination.

```

737 \seq_new:N \l__bnvs_I_seq
738 \BNVS_new:cpn { seq_gremove_once:Nn } #1 #2 {
739   \seq_clear:N \l__bnvs_I_seq
740   \cs_set:Npn \BNVS_seq_gremove_Nn:n ##1 {
741     \tl_if_eq:nnTF { ##1 } { #2 } {
742       \cs_set:Npn \BNVS_seq_gremove_Nn:n #####1 {
743         \seq_put_right:Nn \l__bnvs_I_seq { #####1 }
744       }
745     } {
746       \seq_put_right:Nn \l__bnvs_I_seq { ##1 }
747     }
748   }
749   \seq_map_function:NN #1 \BNVS_seq_gremove_Nn:n
750   \seq_gset_eq:NN #1 \l__bnvs_I_seq
751 }
752 \BNVS_new:cpn { gunset_IP:Nw } #1 #2 #3 {
753   \__bnvs_seq_gremove_once:Nn #1 { #3 }
754   \seq_if_empty:NT #1 {
755     \__bnvs_seq_gremove_once:Nn \g__bnvs_I_seq { #2 }
756     \cs_undefine:c { __bnvs_gset@#2:nnn}
757   }
758 }
759 \BNVS_new:cpn { gunset:NNnnn } #1 #2 #3 #4 #5 {
760   \tl_if_exist:NT #1 {
761     \cs_undefine:N #1
762     \__bnvs_seq_gremove_once:Nn #2 { #5 }
763     \seq_if_empty:NT #2 {
764       \exp_args:Nc \__bnvs_gunset_IP:Nw { \__bnvs_g_I:w { #3 } } { #3 } { #4 }
765       \cs_undefine:c { __bnvs_gset@#3!#4:nn}
766     }
767   }
768 }
769 \BNVS_new:cpn { gunset:nnn } #1 #2 #3 {

```

```

770 \exp_args:Ncc
771 \__bnvs_gunset:NNnnn
772   { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
773   { \__bnvs_g_IP:w { #1 } { #2 } }
774   { #1 } { #2 } { #3 }
775 }
776 \BNVS_new:cpn { gunshot:vvv } {
777   \BNVS_tl_use:Nvv \__bnvs_gunset:nnn
778 }

```

```

\__bnvs_is_gset:nnnTF \__bnvs_is_gset:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}
\__bnvs_is_gset:(nvn|vvv|vxn)TF \__bnvs_is_gset:nnnTF {<id>} {<path>} {<yes:>} {<no:>}
\__bnvs_is_gset:nnTF

```

Test for the existence of some data for that $\langle id \rangle! \langle path \rangle / \langle key \rangle$ combination. The version with no $\langle key \rangle$ corresponds to key ‘=’.

```

779 \BNVS_new_conditional:cpnn { is_gset:nnn } #1 #2 #3 { T, F, TF } {
780   \tl_if_exist:cTF { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
781   \prg_return_true: \prg_return_false:
782 }
783 \BNVS_new_conditional:cpnn { is_gset:vxn } #1 #2 #3 { T, F, TF } {
784   \exp_args:NNnx \BNVS_tl_use:Nv
785   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
786   \prg_return_true: \prg_return_false:
787 }
788 \BNVS_new_conditional:cpnn { is_gset:nvn } #1 #2 #3 { T, F, TF } {
789   \BNVS_tl_use:nv { \__bnvs_is_gset:nnnTF { #1 } } { #2 } { #3 }
790   \prg_return_true: \prg_return_false:
791 }
792 \BNVS_new_conditional:cpnn { is_gset:vvv } #1 #2 #3 { T, F, TF } {
793   \BNVS_tl_use:Nvv \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
794   \prg_return_true: \prg_return_false:
795 }
796 \BNVS_new_conditional:cpnn { is_gset:nn } #1 #2 { T, F, TF } {
797   \__bnvs_is_gset:nnnTF { #1 } { #2 } = \prg_return_true: \prg_return_false:
798 }

```

6.7.4 Loops

```

\__bnvs_foreach_I:N \__bnvs_foreach_I:N {function:n}
\__bnvs_foreach_I:n \__bnvs_foreach_I:n {<n code>}
\__bnvs_foreach_I_break: \__bnvs_foreach_I_break:
\__bnvs_foreach_I_break:n \__bnvs_foreach_I_break:n {<code>}

```

Execute the $\langle function:n \rangle$ or the $\langle n \text{ code} \rangle$ for each declared identifier. The break commands work like `\seq_map_break:` and

```

799 \BNVS_new:cpn { foreach_I:N } {
800   \seq_map_function:NN \g__bnvs_I_seq
801 }
802 \BNVS_new:cpn { foreach_I:n } {
803   \seq_map_inline:Nn \g__bnvs_I_seq
804 }

```



```

805 \cs_set_eq:NN \__bnvs_foreach_I_break: \seq_map_break:
806 \cs_set_eq:NN \__bnvs_foreach_I_break:n \seq_map_break:n

```

```

\__bnvs_foreach_P:nNTF \__bnvs_foreach_P:nNTF {<id>} <function:nn> {<yes:>} {<no:>}
\__bnvs_foreach_P:nnTF \__bnvs_foreach_P:nnTF {<id>} {<:nn code>} {<yes:>} {<no:>}

```

If $\langle id \rangle$ is a declared identifier, execute $\langle function:nn \rangle$ or $\langle code \rangle$ for each combination of $\langle id \rangle$ and its associate $\langle path \rangle$ s. Both are the arguments passed to $\langle function:nn \rangle$ or $\langle :nn code \rangle$ (through ##1 and ##2).

```

807 \BNVS_new_conditional:cpnn { foreach_P:nN } #1 #2 { T, F, TF } {
808   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
809     \seq_map_inline:cn {
810       \__bnvs_g_I:w { #1 }
811     } {
812       #2 { #1 } { ##1 }
813     }
814     \prg_return_true:
815   } {
816     \prg_return_false:
817   }
818 }
819 \BNVS_new_conditional:cpnn { foreach_P:nn } #1 #2 { T, F, TF } {
820   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
821     \cs_set:Npn \BNVS_foreach_P_nn:nn ##1 ##2 { #2 }
822     \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
823       \BNVS_foreach_P_nn:nn { #1 } { ##1 }
824     }
825     \prg_return_true:
826   } {
827     \prg_return_false:
828   }
829 }

```

```

\__bnvs_foreach_K:nnN \__bnvs_foreach_K:nnN {<id>} {<path>} <function:nnn>
\__bnvs_foreach_K:nnn \__bnvs_foreach_K:nnn {<id>} {<path>} {<:nnn code>}

```

Execute the $\langle function:nnn \rangle$ or the $\langle :nnn code \rangle$ for each of $\langle id \rangle!$ $\langle path \rangle$ / $\langle key \rangle$ combination. These are the arguments to the function or code argument.

```

830 \BNVS_new:cpn { foreach_K:NnnN } #1 #2 #3 #4 {
831   \seq_if_exist:NT #1 {
832     \seq_map_inline:Nn #1 {
833       #4 { #2 } { #3 } { ##1 }
834     }
835   }
836 }
837 \BNVS_new:cpn { foreach_K:nnN } #1 #2 #3 {
838   \exp_args:Nc \__bnvs_foreach_K:NnnN
839   { \__bnvs_g_IP:w { #1 } { #2 } }
840   { #1 } { #2 } #3
841 }
842 \BNVS_new:cpn { foreach_K:nnn } #1 #2 #3 {
843   \cs_set:Npn \BNVS_foreach_K_nnn:w ##1 ##2 ##3 { #3 }
844   \__bnvs_foreach_K:nnN { #1 } { #2 } \BNVS_foreach_K_nnn:w
845 }

```

<code>_bnvs_foreach_IP:N</code> <code>_bnvs_foreach_IP:n</code>	<code>_bnvs_foreach_IP:N <function:nn></code> <code>_bnvs_foreach_IP:n {<:nn code>}</code>
--	---

Execute the `<function:nn>` or the `<:nn code>` for each `<id>!``<path>` combination.

```

846 \BNVS_new:cpn { foreach_IP:N } #1 {
847   \\_bnvs_foreach_I:n {
848     \\_bnvs_foreach_P:nNT { ##1 } #1 { }
849   }
850 }

851 \BNVS_new:cpn { foreach_IP:NT } #1 #2 {
852   \\_bnvs_foreach_I:n {
853     \\_bnvs_foreach_P:nNT { ##1 } #1 { #2 }
854   }
855 }

856 \BNVS_new:cpn { foreach_IP:n } #1 {
857   \cs_set:Npn \BNVS_foreach_IP_n:nn ##1 ##2 { #1 }
858   \\_bnvs_foreach_I:n {
859     \\_bnvs_foreach_P:nNT { ##1 } \BNVS_foreach_IP_n:nn { }
860   }
861 }

```

6.7.5 Low level IP model functions

6.7.6 Low level I model functions

6.7.7 Getting

<code>_bnvs_if_get:nnncTF</code> <code>_bnvs_if_spec:nnncTF</code> <code>_bnvs_if_get:nncTF</code> <code>_bnvs_if_get:vvcTF</code>	<code>_bnvs_if_get:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}</code> <code>_bnvs_if_spec:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}</code> <code>_bnvs_if_get:nncTF {<id>} {<path>} {<ans>} {<yes:>} {<no:>}</code>
---	---

The `\\_bnvs_if_get:nnnc...` variant puts what was stored for `<id>!``<path>/``<key>` into the `<ans>` variable, if any, then executes `<yes:>`. Otherwise executes `<no:>` without changing the contents of the `<ans>` tl variable.

`\\_bnvs_if_get:nncTF` is a convenient shortcut for `\\_bnvs_if_get:nnncTF` when `<key>` and `<ans>` happen to be the same.

The `_spec:` variant is similar except that it uses `<id>!``<path>/``<key>` combinations when available or `!``<path>/``<key>` otherwise.

```

862 \BNVS_new_conditional:cpnn { if_get:nnnc } #1 #2 #3 #4 { T, F, TF } {
863   \\_bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
864     \BNVS_tl_use:Nc \cs_set_eq:Nc
865       { #4 } { \\_bnvs_g_IPK:w { #1 } { #2 } { #3 } }
866     \prg_return_true:
867   } {
868     \prg_return_false:
869   }
870 }

871 \BNVS_new_conditional:cpnn { if_spec:nnnc } #1 #2 #3 #4 { T, F, TF } {
872   \\_bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {

```

```

873     \tl_set_eq:cc { \BNVS_1:cn { #4 } { tl } } {
874         \__bnvs_g_IPK:w { #1 } { #2 } { #3 }
875     }
876     \prg_return_true:
877 } {
878     \tl_if_empty:nTF { #1 } {
879         \prg_return_false:
880     } {
881         \__bnvs_if_spec:nnncTF {} { #2 } { #3 } { #4 }
882         \prg_return_true: \prg_return_false:
883     }
884 }
885 }

886 \BNVS_new_conditional:cpnn { if_get:nnc } #1 #2 #3 { T, F, TF } {
887     \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #3 }
888     \prg_return_true: \prg_return_false:
889 }

890 \BNVS_new_conditional:cpnn { if_get:vvc } #1 #2 #3 { T, F, TF } {
891     \BNVS_tl_use:Nvv \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 }
892     \prg_return_true: \prg_return_false:
893 }

```

__bnvs_if_resolved:nnncTF __bnvs_if_resolved:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}

Execute <yes:> if resolution succeeded for either <id>!<<path>/<key> or the fallback !<path>/<key>, otherwise execute <no:>.

```

894 \BNVS_new_conditional:cpnn { if_resolved:nnnc } #1 #2 #3 #4 { T, F, TF } {
895     \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #4 } {
896         \__bnvs_is_will_gset:cTF { #3 }
897         \prg_return_true: \prg_return_false:
898     } {
899         \tl_if_empty:nTF { #1 } {
900             \prg_return_false:
901         } {
902             \__bnvs_if_resolved:nnncTF { #1 } { #2 } { #3 } { #4 }
903             \prg_return_true: \prg_return_false:
904         }
905     }
906 }

```

__bnvs_require:nnnc __bnvs_require:nnnc {<id>} {<path>} {<key>} {<ans>}
__bnvs_require:nnc __bnvs_require:nnc {<id>} {<path>} {<ans>}
__bnvs_require:vvc

Calls __bnvs_if_get:nnncTF, does nothing on success and, on failure, does nothing or raise when in debug mode. __bnvs_require:nnc is a shortcut for the more qualified function __bnvs_require:nnnc when <key> and <ans> happen to be identical.

```

907 \BNVS_new:cpn { require:nnnc } #1 #2 #3 #4 {
908     \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { #4 } {
909         \BNVS_fatal_unreachable:
910     }
911 }

```

`\_bnvs_gunset:nn` `\_bnvs_gunset:nn {<id>} {<path>}`
`\_bnvs_gunset:nv` Removes the specifications for the $\langle id \rangle! \langle path \rangle$ and all possible keys combination.

```

912 \BNVS_new:cpn { gunset:NNNnn } #1 #2 #3 #4 #5 {
913   \seq_map_inline:Nn #1 {
914     \_bnvs_gunset:nnn { #4 } { #5 } { ##1 }
915   }
916   \cs_undefine:N #1
917   \_bnvs_seq_gremove_once:Nn #2 { #5 }
918   \seq_if_empty:NT #2 {
919     \cs_undefine:N #2
920     \_bnvs_seq_gremove_once:Nn #3 { #4 }
921     \cs_undefine:c { \_bnvs_gset@ #4 :nnn }
922   }
923 }
924 \BNVS_new:cpn { gunset:Nnn } #1 #2 #3 {
925   \seq_map_inline:Nn #1 {
926     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
927       \exp_args:Nc \_bnvs_gunset:NNNnn
928         { \_bnvs_g_IP:w { #2 } { ##1 } }
929       #1 \g_bnvs_I_seq { #2 } { ##1 }
930       \cs_undefine:c { \_bnvs_gset@{ #2 } { ##1 }:nn }
931     }
932   }
933 }
934 \BNVS_new:cpn { gunset:nn } #1 #2 {
935   \exp_args:Nc
936   \_bnvs_gunset:Nnn { \_bnvs_g_I:w { #1 } } { #1 } { #2 }
937 }

938 \BNVS_new:cpn { gunset:nv } #1 {
939   \BNVS_tl_use:nv { \_bnvs_gunset:nn { #1 } }
940 }

941 \BNVS_new:cpn { gunset:vn } {
942   \BNVS_tl_use:cv { gunset:nn }
943 }

944 \BNVS_new:cpn { gunset:vv } {
945   \BNVS_tl_use:Nvv \_bnvs_gunset:nn
946 }

```

`\_bnvs_gunset_deep:nn` `\_bnvs_gunset_deep:nn {<id>} {<path>}`
Removes the specifications for each existing $\langle id \rangle! \langle path \rangle. \langle subpath \rangle$ combination.

```

947 \BNVS_new:cpn { gunset_deep:Nnn } #1 #2 #3 {
948   \seq_if_exist:NT #1 {
949     \seq_map_inline:Nn #1 {

```

#3 is exactly **##1** or start with **##1.**, which is equivalent to **#3.** starts with **##1.**:

```

950     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
951       \__bnvs_gunset:nn { #2 } { ##1 }
952     }
953   }
954 }
955 }

956 \BNVS_new:cpn { gunset:nn } #1 #2 {
957   \exp_args:Nc \__bnvs_gunset_deep:Nnn
958   { \__bnvs_g_I:w { #1 } }
959   { #1 } { #2 }
960 }

961 \BNVS_new:cpn { gunset:NNNn } #1 #2 #3 #4 {
962   \cs_if_exist:NT #1 {
963     \cs_undefine:N #1
964     \seq_map_inline:Nn #2 {
965       \__bnvs_gunset:nn { #4 } { ##1 }
966     }
967     \cs_undefine:N #2
968     \__bnvs_seq_gremove_once:Nn #3 { #4 }
969   }
970 }

971 \BNVS_new:cpn { gunset:n } #1 {
972   \exp_args:Ncc \__bnvs_gunset:NNNn
973   { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } }
974   \g__bnvs_I_seq { #1 }
975 }

976 \BNVS_new:cpn { gunset: } {
977   \seq_map_inline:Nn \g__bnvs_I_seq {
978     \__bnvs_gunset:n { ##1 }
979   }
980 }

```

__bnvs_VS_gunset: __bnvs_V_gunset:

Removes any value for the “V” and “S” keys and any $\langle id \rangle!$ $\langle path \rangle$ combination.

```

981 \BNVS_new:cpn { VS_gunset: } {
982   \__bnvs_foreach_IP:n {
983     \__bnvs_gunset:nnn { ##1 } { ##2 } V
984     \__bnvs_gunset:nnn { ##1 } { ##2 } S
985   }
986 }

```

6.7.8 Circular dependencies

```

__bnvs_will_gset:nnn  __bnvs_will_gset:nnn {<id>} {<path>} {<key>}

```

To manage circular dependencies. After this command, `__bnvs_is_will_gset:nnnTF` is true for the same arguments.

```

987 \BNVS_new:cpn { will_gset:nnn } #1 #2 #3 {
988   __bnvs_gset:nnnn { #1 } { #2 } { #3 } { \q_no_value }
989 }

```

```

__bnvs_is_will_gset:cTF  __bnvs_is_will_gset:cTF {<in>} {<yes:>} {<no:>}

```

To manage circular dependencies. See `__bnvs_will_gset:nnn` above.

```

1  __bnvs_will_gset:nnn {<id>} {<path>} {<key>}
2  ...
3  __bnvs_if_get:nnncT {<id>} {<path>} {<key>} {<in>} {
4    ...
5    __bnvs_is_will_gset:cTF
6    {<in>}
7    {<yes:>}
8    {<no:>}
9    ...
10 }

```

```

990 \BNVS_new_conditional:cpnn { is_will_gset:c } #1 { T, F, TF } {
991   \BNVS_tl_use:Nv \quark_if_no_value:nTF { #1 } {
992     \prg_return_true:
993   } {
994     \prg_return_false:
995   }
996 }

```

6.7.9 Spring cleaning

```

__bnvs_gclear:nn  __bnvs_gclear:nn {<id>} {<path>}
__bnvs_gclear:nv  __bnvs_gclear:n {<id>}
__bnvs_gclear:    __bnvs_gclear:
__bnvs_greset:nn  __bnvs_greset:nn {<id>} {<path>}
__bnvs_greset:nv  __bnvs_greset:n {<id>}
__bnvs_greset:    __bnvs_greset:

```

The first ones clear out every data stored for `<id>!``<path>`, including deeper and registration data. When `<path>` is omitted, any known path is considered. When `<id>` is omitted, any known id is considered. The second one only clears out the data created on the fly after the initialization step. The initial data (for key =) is left untouched.

```

997 \BNVS_new:cpn { greset:nn } #1 #2 {

```

```

998   \_bnvs_foreach_P:nnT { #1 } {
999     \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1000       \_bnvs_foreach_K:nnn { #1 } { ##2 } {
1001         \tl_if_in:nnF { \q_bnvs ####3 } { \q_bnvs = } {
1002           \_bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1003         }
1004       }
1005     }
1006   } { }
1007 }

```

Clears out every data associated to $\langle id \rangle!$ $\langle path \rangle$, and any $\langle id \rangle!$ $\langle path \rangle.$ $\langle subpath \rangle$.

```

1008 \BNVS_new:cpn { gclear:nn } #1 #2 {
1009   \_bnvs_foreach_P:nnT { #1 } {
1010     \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1011       \_bnvs_foreach_K:nnn { #1 } { ##2 } {
1012         \tl_if_in:nnTF { \q_bnvs ####3 } { \q_bnvs = } {
1013           \_bnvs_gset:nnnn { #1 } { ##2 } { ####3 } { 0 }
1014         } {
1015           \_bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1016         }
1017       }
1018     }
1019   } { }
1020 }

1021 \BNVS_new:cpn { gclear:n } #1 {
1022   \_bnvs_foreach_P:nNT { #1 } \_bnvs_gclear:nn {}
1023 }

1024 \BNVS_new:cpn { gclear: } {
1025   \_bnvs_foreach_IP:N \_bnvs_gclear:nn
1026 }

1027 \BNVS_new:cpn { greset:n } #1 {
1028   \_bnvs_foreach_P:nNT { #1 } \_bnvs_greset:nn {}
1029 }

1030 \BNVS_new:cpn { greset: } {
1031   \_bnvs_foreach_IP:N \_bnvs_greset:nn
1032 }

```

```

\_bnvs_gclear_resolved:nn \_bnvs_gclear_resolved:nn {<id>} {<path>}

```

```

\_bnvs_gclear_resolved:nv

```

Clear the resolved data, aka any key that does not start with “=”.

```

1033 \BNVS_new:cpn { gclear_resolved:nn } #1 #2 {
1034   \_bnvs_foreach_IP:nnn { #1 } { #2 } {
1035     \tl_if_in:nnF { @ ##1 } { @ = } {
1036       \_bnvs_gunset:nnn { #1 } { #2 } { ##1 }
1037     }
1038   }
1039 }

```

```

1040 \BNVS_new:cpn { gclear_resolved:nv } #1 {
1041   \BNVS_tl_use:nv {
1042     \__bnvs_gclear_resolved:nn { #1 }
1043   }
1044 }

```

6.7.10 The provide mode

\l__bnvs_provide_bool Manage the provide mode.

```

1045 \bool_new:N \l__bnvs_provide_bool

(End of definition for \l__bnvs_provide_bool.)

```

```

1046 \BNVS_new:cpn { provide_ON: } {
1047   \__bnvs_set_true:c { provide }
1048 }

1049 \BNVS_new:cpn { provide_OFF: } {
1050   \__bnvs_set_false:c { provide }
1051 }
1052 \__bnvs_provide_OFF:

```

```

\__bnvs_if_should_provide:nnnTF \__bnvs_if_should_provide:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}
\__bnvs_if_should_provide:nnTF \__bnvs_if_should_provide:nnTF {<id>} {<path>} {<yes:>} {<no:>}

```

Execute $\langle yes: \rangle$ when in provide mode and $\langle id \rangle! \langle path \rangle / \langle key \rangle$ is known, $\langle no: \rangle$ otherwise.
When not specified, $\langle key \rangle$ is “=”.

```

1053 \BNVS_new_conditional:cpnn { if_should_provide:nnn } #1 #2 #3 { T, F, TF } {
1054   \__bnvs_if:cTF { provide } {
1055     \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
1056       \prg_return_false:
1057     } {
1058       \prg_return_true:
1059     }
1060   } {
1061     \prg_return_true:
1062   }
1063 }

1064 \BNVS_new_conditional:cpnn { if_should_provide:nn } #1 #2 { T, F, TF } {
1065   \__bnvs_if_should_provide:nnnTF { #1 } { #2 } =
1066   \prg_return_true: \prg_return_false:
1067 }

```

```

\__bnvs_is_gset_and_provide:nnnTF \__bnvs_is_gset_and_provide:nnnTF {<id>} {<path>} {<key>}
\__bnvs_is_gset_and_provide:(nvn|vvn)TF {<yes:>} {<no:>}
\__bnvs_is_gset_and_provide:nnTF \__bnvs_is_gset_and_provide:nnTF {<id>} {<path>} {<yes:>}
\__bnvs_is_gset_and_provide:nnTF {<no:>}

```

Execute $\langle yes: \rangle$ when in provide mode and $\langle id \rangle! \langle path \rangle / \langle key \rangle$ is known, $\langle no: \rangle$ otherwise.
When not specified, $\langle key \rangle$ is “=”.


```

1068 \BNVS_new_conditional:cpnn { is_gset_and_provide:nnn }
1069     #1 #2 #3 { T, F, TF } {
1070     \__bnvs_if:cTF { provide } {
1071     \if_cs_exist:w \__bnvs_g_IPK:w { #1 } { #2 } { #3 } \cs_end:
1072     \prg_return_true:
1073     \else:
1074     \prg_return_false:
1075     \fi:
1076     } {
1077     \prg_return_false:
1078     }
1079 }

1080 \BNVS_new_conditional:cpnn { is_gset_and_provide:nvn }
1081     #1 #2 #3 { T, F, TF } {
1082     \BNVS_tl_use:nv {
1083     \__bnvs_is_gset_and_provide:nnnTF { #1 }
1084     } { #2 } { #3 }
1085     \prg_return_true: \prg_return_false:
1086     }

1087 \BNVS_new_conditional:cpnn { is_gset_and_provide:vvv }
1088     #1 #2 #3 { T, F, TF } {
1089     \BNVS_tl_use:Nvv \__bnvs_is_gset_and_provide:nnnTF { #1 } { #2 } { #3 }
1090     \prg_return_true: \prg_return_false:
1091     }

```

```

\__bnvs_gprovide:TnnnnF \__bnvs_gprovide:TnnnnF {<pre code>} {<id>} {<path>} {<key>} {<value>} {<no:>}

```

```

\__bnvs_gprovide:TvnnnF Execute <no:> exclusively when not in provide mode. Does nothing when something was
\__bnvs_gprovide:TvvnnF set for the <id>!\<path>/\<key> combination. Execute <pre code> before setting.

```

```

1092 \BNVS_new:cpn { gprovide:TnnnnF } #1 #2 #3 #4 #5 {
1093     \__bnvs_if:cTF { provide } {
1094     \__bnvs_is_gset:nnnF { #2 } { #3 } { #4 } {
1095     #1
1096     \__bnvs_gset:nnnn { #2 } { #3 } { #4 } { #5 }
1097     }
1098     }
1099 }

1100 \BNVS_new:cpn { gprovide:TvnnnF } #1 {
1101     \BNVS_tl_use:nv { \__bnvs_gprovide:TnnnnF { #1 } }
1102     }
1103 \BNVS_new:cpn { gprovide:TvvnnF } #1 {
1104     \BNVS_tl_use:nvv { \__bnvs_gprovide:TnnnnF { #1 } }
1105     }

```

6.7.11 Initialize

<code>_bnvs_ginit:nnn</code> <code>_bnvs_ginit:(nnv vvn)</code>	<code>_bnvs_ginit:nnn {<id>} {<path>} {<definition>}</code>
--	---

This function supports provide mode. Somehow a shortcut to `\\_bnvs_gset:nnnn` with key “=”. If `<path>` has more than one component, each intermediate path is modified or defined appropriately.

```

1106 \BNVS_new:cpn { ginit:nnn } #1 #2 #3 {
1107   \\_bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
1108     \\_bnvs_greset:nn { #1 } { #2 }
1109     \\_bnvs_gset:nnnn { #1 } { #2 } = { #3 }
1110   }
1111 }

1112 \BNVS_new:cpn { ginit:nnv } #1 #2 {
1113   \BNVS_tl_use:nv { \\_bnvs_ginit:nnn { #1 } { #2 } }
1114 }

1115 \BNVS_new:cpn { ginit:vvn } {
1116   \BNVS_tl_use:Nvv \\_bnvs_ginit:nnn
1117 }
```

<code>_bnvs_ginit_IPND:w</code>	<code>_bnvs_ginit_IPND:w {<id>} {<path>} {<index>} {<definition>}</code>
-----------------------------------	--

These functions support provide mode. Somehow a shortcut to `\\_bnvs_gset:nnnn` with key `<index>`. When `<index>` is provided, it is already normalized. If `<id>!<path>/= is not set, it is initialized from <definition> shifted appropriately relative to <index>.`

```

1118 \BNVS_new:cpn { ginit_IPND:w } #1 #2 #3 #4 {
1119   \\_bnvs_is_gset_and_provide:nnnF { #1 } { #2 } { #3 } {
1120     \\_bnvs_gunset:nnn { #1 } { #2 } { #3 }
1121     \\_bnvs_gset:nnnn { #1 } { #2 } { =#3 } { #4 }
1122   }
1123   \\_bnvs_is_gset:nnnF { #1 } { #2 } = {
1124     \\_bnvs_gset:nnne { #1 } { #2 } = {
1125       \exp_not:n { #4 } - \int_eval:n { #3 - 1 }
1126     }
1127   }
1128 }
```

<code>_bnvs_gdef_by_key:nnnn</code>	<code>_bnvs_gdef_by_key:nnnn {<id>} {<path>} {<key>} {<definition>}</code>
---------------------------------------	--

This function calls `\\_bnvs_gdef_by_key:nnn` with `<path>.<key>` as second argument.

```

1129 \BNVS_new:cpn { gdef_by_key:nnnn } #1 #2 #3 #4 {
1130   \\_bnvs_gdef_by_key:nnn { #1 } { #2.#3 } { #4 }
1131 }
```

6.7.12 Provide mode

__bnvs_is_provided:nnnTF __bnvs_is_provided:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}

If <id>!
<path>/<key> has been successfully resolved and not unset since then, execute <yes:>. Otherwise execute <no:>.

```

1132 \tl_new:N \l__bnvs_is_provided_tl
1133 \BNVS_new_conditional:cpnn { is_provided:nnn } #1 #2 #3 { T, F, TF } {
1134   \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { is_provided } {
1135     \__bnvs_is_will_gset:cTF { is_provided } {
1136       \prg_return_false:
1137     } {
1138       \prg_return_true:
1139     }
1140   } {
1141     \prg_return_false:
1142   }
1143 }
```

6.8 Regular expressions

\c__bnvs_S_regex This regular expression is used for both short names and dot path components. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit. It must contain one uppercase letter.

```

1144 \regex_const:Nn \c__bnvs_S_regex {
1145   (?:[a-z_][a-z0-9_]*)?[A-Z][[:alnum:]]*
1146 }
```

(End of definition for \c__bnvs_S_regex.)

\c__bnvs_P_regex A possibly empty list of .<short name> items representing a path.

```

1147 \regex_const:Nn \c__bnvs_P_regex {
1148   (?:\. \ur{c__bnvs_S_regex})*
1149 }
```

(End of definition for \c__bnvs_P_regex.)

\c__bnvs_U_regex An <attribute>, no uppercase variable.

```

1150 \regex_const:Nn \c__bnvs_U_regex {
1151   [a-z_][a-z0-9_]*
1152 }
```

(End of definition for \c__bnvs_U_regex.)

\c__bnvs_O_regex A possibly empty list of .<attributes> items representing a path.

```

1153 \regex_const:Nn \c__bnvs_O_regex {
1154   (?:\. \ur{c__bnvs_U_regex})*
1155 }
```

(End of definition for \c__bnvs_O_regex.)

`\c__bnvs_A_N_Z_regex` Signed integer with two capture groups.

(End of definition for \c__bnvs_A_N_Z_regex.)

```
1156 \regex_const:Nn \c__bnvs_A_N_Z_regex {
1157   \A \. \s* [-+\s]* \d+ \Z
1158 }
```

`\c__bnvs_A_index_Z_regex` Signed integer with no capture groups.

(End of definition for \c__bnvs_A_index_Z_regex.)

```
1159 \regex_const:Nn \c__bnvs_A_index_Z_regex { \A [-+\s]* \d + \Z }
```

`\c__bnvs_A_reserved_Z_regex` Reserved words.

(End of definition for \c__bnvs_A_reserved_Z_regex.)

```
1160 \regex_const:Nn \c__bnvs_A_reserved_Z_regex {
1161   \A_*[a-z][_a-z0-9]*\Z
1162 }
```

`\c__bnvs_A_IKPN_Z_regex` Matches the whole string, split into $\langle id \rangle$ and $\langle path \rangle$.

(End of definition for \c__bnvs_A_IKPN_Z_regex.)

1: The full match,

```
1163   \regex_const:Nn \c__bnvs_A_IKPN_Z_regex {
```

2,3: the optional frame $\langle id \rangle$ and !

```
1164   \A (?: ( \ur{c__bnvs_S_regex} )? (!) )?
```

4: The tag (short name and dotted path)

```
1165   ( \ur{c__bnvs_S_regex} \ur{c__bnvs_P_regex} ? )
```

5: a signed number that will be normalized.

```
1166   (?: \. \s* ([-+]? \s* \d+ ) )? \Z
1167 }
```

`\c__bnvs_A_SP_Z_regex` Matches the whole string.

(End of definition for \c__bnvs_A_SP_Z_regex.)

```
1168 \regex_const:Nn \c__bnvs_A_SP_Z_regex {
```

1: The full match,

2: the frame $\langle id \rangle$

```
1169   \A ( \ur{c__bnvs_S_regex} | [-+]? \d+ )
```

3: The dotted path.

```
1170      ( (? : \. \ur{c__bnvs_S_regex} | \. [-+]? \d+ )* ) \Z
1171    }
```

`\c__bnvs_A_P_Z_regex` Matches the whole string.

(End of definition for \c__bnvs_A_P_Z_regex.)

```
1172 \regex_const:Nn \c__bnvs_A_P_Z_regex {
1173   \A \ur{c__bnvs_S_regex} \ur{c__bnvs_P_regex} \Z
1174 }
```

`\c__bnvs_colons_regex` For ranges defined by a colon syntax. One catching group for more than one colon.

```
1175 \regex_const:Nn \c__bnvs_colons_regex { :(:+)? }
```

(End of definition for \c__bnvs_colons_regex.)

`\c__bnvs_A_VAZLLZZL_Z_regex` Used to parse named overlay specifications. V, A:Z, A::L on one side, :Z, :Z::L and ::L:Z on the other sides. Next are the capture groups. The first one is for the whole match.

(End of definition for \c__bnvs_A_VAZLLZZL_Z_regex.)

```
1176 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
1177   \A \s* (?:
```

- 2 $\rightarrow$ V

```
1178     ( [^: ]+? )
```

- 3, 4, 5 $\rightarrow$ A : Z? or A :: L?

```
1179     | (?: ( [^: ]+? ) \s* : (?: \s* ( [^: ]*? ) | : \s* ( [^: ]*? ) ) ) )
```

- 6, 7 $\rightarrow$::(L:Z)?

```
1180     | (?: :: \s* (?: ( [^: ]+? ) \s* : \s* ( [^: ]+? ) )? )
```

- 8, 9 $\rightarrow$:(Z::L)?

```
1181     | (?: : \s* (?: ( [^: ]+? ) \s* :: \s* ( [^: ]*? ) )? )
```

```
1182   )
```

```
1183   \s* \Z
```

```
1184 }
```

6.9 End group setters

```

1185 \cs_new:Npn \BNVS_after_end_tl_put_right:cv #1 {
1186   \BNVS_after_end:n {
1187     \__bnvs_tl_put_right:cn { #1 }
1188   }
1189   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1190 }

1191 \cs_new:Npn \BNVS_end_tl_put_right:cv #1 #2 {
1192   \BNVS_after_end_tl_put_right:cv { #1 } { #2 }
1193   \BNVS_end:
1194 }

1195 \cs_new:Npn \BNVS_after_end_tl_gset:nnnv #1 #2 #3 {
1196   \BNVS_after_end:n {
1197     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
1198   }
1199   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1200 }

1201 \cs_new:Npn \BNVS_end_tl_gset:nnnv #1 #2 #3 #4 {
1202   \BNVS_after_end_tl_gset:nnnv { #1 } { #2 } { #3 } { #4 }
1203   \BNVS_end:
1204 }

1205 \cs_new:Npn \BNVS_after_end_tl_set:cv #1 {
1206   \BNVS_after_end:n {
1207     \__bnvs_tl_set:cn { #1 }
1208   }
1209   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1210 }

1211 \cs_new:Npn \BNVS_end_tl_set:cv #1 #2 {
1212   \BNVS_after_end_tl_set:cv { #1 } { #2 }
1213   \BNVS_end:
1214 }

1215 \cs_new:Npn \BNVS_tl_set:ncv #1 #2 {
1216   \BNVS_tl_use:nv {
1217     #1
1218     \__bnvs_tl_set:cn { #2 }
1219   }
1220 }

1221 \cs_new:Npn \BNVS_tl_set:ncvcv #1 #2 #3 #4 {
1222   \BNVS_tl_set:ncv {
1223     \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1224   } { #4 }
1225 }

```

```

1226 \cs_new:Npn \BNVS_tl_set:ncvcvcv #1 #2 #3 #4 #5 #6 {
1227   \BNVS_tl_set:ncv {
1228     \BNVS_tl_set:ncv {
1229       \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1230     } { #4 } { #5 }
1231   } { #6 }
1232 }

1233 \cs_new:Npn \BNVS_after_end_int_set:cv #1 {
1234   \BNVS_after_end:n {
1235     \__bnvs_int_set:cn { #1 }
1236   }
1237   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1238 }

1239 \cs_new:Npn \BNVS_end_int_set:cv #1 #2 {
1240   \BNVS_after_end_int_set:cv { #1 } { #2 }
1241   \BNVS_end:
1242 }

```

6.10 beamer.cls interface

Work in progress.

```

1243 \RequirePackage{keyval}

1244 \define@key{beamerframe}{beanoves~id}[] {
1245   \tl_set:Nx \l__bnvs_J_tl { #1 }
1246 }

1247 \bool_new:N \l__bnvs_in_frame_bool
1248 \bool_set_false:N \l__bnvs_in_frame_bool
1249 \AddToHook{env/beamer@frameslide/before}{
1250   \__bnvs_greset_call:
1251   \__bnvs_VS_gunset:
1252   \__bnvs_set_true:c { in_frame }
1253 }

1254 \AddToHook{env/beamer@frameslide/after}{
1255   \__bnvs_set_false:c { in_frame }
1256 }

```

6.11 Utilities

6.11.1 Split utilities

```

\__bnvs_if_regex_split:cncTF \__bnvs_if_regex_split:cncTF {<regex core>} {<expression>} <seq core>
\__bnvs_if_regex_split:cnTF {<yes:>} {<no:>}
\__bnvs_if_regex_split:cnTF {<regex core>} {<expression>} {<yes:>} {<no:>}

```

These are shortcuts to `\regex_split:NnNTF` with the `split` sequence as last `N` argument.

```

1257 \BNVS_new_conditional:cpnn { if_regex_split:cnc } #1 #2 #3 { T, F, TF } {
1258   \BNVS_seq_use:nc {
1259     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1260   } { #3 } \prg_return_true: \prg_return_false:
1261 }

```

```

1262 \BNVS_new_conditional:cpnn { if_regex_split:cn } #1 #2 { T, F, TF } {
1263   \BNVS_seq_use:nc {
1264     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1265   } { split } \prg_return_true: \prg_return_false:
1266 }

```

__bnvs_split_pop:	__bnvs_split_pop:	
__bnvs_split_pop:c	__bnvs_split_pop:c	{\var_1}
__bnvs_split_pop:cc	__bnvs_split_pop:cc	{\var_1} {\var_2}
__bnvs_split_pop:ccc	__bnvs_split_pop:ccc	{\var_1} {\var_2} {\var_3}
__bnvs_split_pop:cccc	__bnvs_split_pop:cccc	{\var_1} {\var_2} {\var_3} {\var_4}
__bnvs_split_pop:ccccc	__bnvs_split_pop:ccccc	{\var_1} {\var_2} {\var_3} {\var_4} {\var_5}

```

1267 \tl_new:N \l__bnvs_split_pop_tl
1268 \BNVS_new:cpn { split_pop: } {
1269   \seq_pop_left:NN \l__bnvs_split_seq \l__bnvs_split_pop_tl
1270 }

```

```

1271 \BNVS_new:cpn { split_pop:c } #1 {
1272   \BNVS_tl_use:nc {
1273     \seq_pop_left:NN \l__bnvs_split_seq
1274   } { #1 }
1275 }

```

```

1276 \BNVS_new:cpn { split_pop:cc } #1 {
1277   \__bnvs_split_pop:c { #1 }
1278   \__bnvs_split_pop:c
1279 }

```

```

1280 \BNVS_new:cpn { split_pop:ccc } #1 {
1281   \__bnvs_split_pop:c { #1 }
1282   \__bnvs_split_pop:cc
1283 }

```

```

1284 \BNVS_new:cpn { split_pop:cccc } #1 {
1285   \__bnvs_split_pop:c { #1 }
1286   \__bnvs_split_pop:ccc
1287 }

```

```

1288 \BNVS_new:cpn { split_pop:ccccc } #1 {
1289   \__bnvs_split_pop:c { #1 }
1290   \__bnvs_split_pop:cccc
1291 }

```

```

1292 \BNVS_new_conditional:cpnn { split_if_pop_left:c } #1 { T, F, TF } {
1293   \__bnvs_seq_pop_left:ccTF { split } { #1 } {
1294     \prg_return_true:
1295   } {
1296     \prg_return_false:
1297   }
1298 }

```


6.11.2 Match utilities

```

__bnvs_match_if_once:NnTF \__bnvs_match_if_once:NnTF <regex variable> {(expression)}
__bnvs_match_if_once:NvTF {\yes:} {\no:}
__bnvs_match_if_once:nnTF \__bnvs_match_if_once:nnTF {<regex>} {(expression)}
{\yes:} {\no:}

```

These are shortcuts to

- \regex_match_if_once:NnNTF with the match sequence as N argument
- \regex_match_if_once:nnNTF with the match sequence as N argument
- \regex_split:NnNTF with the split sequence as last N argument

```

1299 \BNVS_new_conditional:cpnn { if_extract_once:Ncn } #1 #2 #3 { T, F, TF } {
1300   \BNVS_use:ncn {
1301     \regex_extract_once:NnNTF #1 { #3 }
1302   } { #2 } { seq } \prg_return_true: \prg_return_false:
1303 }

```

\l__bnvs_match_seq Local storage for the match result.

(End of definition for \l__bnvs_match_seq.)

```

1304 \BNVS_new_conditional:cpnn { match_if_once:Nn } #1 #2 { T, F, TF } {
1305   \BNVS_use:ncn {
1306     \regex_extract_once:NnNTF #1 { #2 }
1307   } { match } { seq } \prg_return_true: \prg_return_false:
1308 }

1309 \BNVS_new_conditional:cpnn { if_extract_once:Ncv } #1 #2 #3 { T, F, TF } {
1310   \BNVS_seq_use:nc {
1311     \BNVS_tl_use:nv {
1312       \regex_extract_once:NnNTF #1
1313     } { #3 }
1314   } { #2 } \prg_return_true: \prg_return_false:
1315 }

1316 \BNVS_new_conditional:cpnn { match_if_once:Nv } #1 #2 { T, F, TF } {
1317   \BNVS_seq_use:nc {
1318     \BNVS_tl_use:nv {
1319       \regex_extract_once:NnNTF #1
1320     } { #2 }
1321   } { match } \prg_return_true: \prg_return_false:
1322 }

1323 \BNVS_new_conditional:cpnn { match_if_once:nn } #1 #2 { T, F, TF } {
1324   \BNVS_seq_use:nc {
1325     \regex_extract_once:nnNTF { #1 } { #2 }
1326   } { match } \prg_return_true: \prg_return_false:
1327 }

```

```

1328 \BNVS_new_conditional:cpnn { match_if_pop_left:c } #1 { T, F, TF } {
1329     \BNVS_tl_use:nc {
1330         \BNVS_seq_use:Nc \seq_pop_left:NNTF { match }
1331     } { #1 } {
1332         \prg_return_true:
1333     } {
1334         \prg_return_false:
1335     }
1336 }

```

__bnvs_match_pop:	__bnvs_match_pop:
__bnvs_match_pop:c	__bnvs_match_pop:c {<var_1>}
__bnvs_match_pop:cc	__bnvs_match_pop:cc {<var_1>} {<var_2>}
__bnvs_match_pop:ccc	__bnvs_match_pop:ccc {<var_1>} {<var_2>} {<var_3>}
__bnvs_match_pop:cccc	__bnvs_match_pop:cccc {<var_1>} {<var_2>} {<var_3>} {<var_4>}
__bnvs_match_pop:ccccc	__bnvs_match_pop:ccccc {<var_1>} {<var_2>} {<var_3>} {<var_4>} {<var_5>}

```

1337 \tl_new:N \l__bnvs_match_pop_tl
1338 \BNVS_new:cpn { match_pop: } {
1339     \seq_pop_left:NN \l__bnvs_match_seq \l__bnvs_match_pop_tl
1340 }

1341 \BNVS_new:cpn { match_pop:c } #1 {
1342     \BNVS_tl_use:nc {
1343         \seq_pop_left:NN \l__bnvs_match_seq
1344     } { #1 }
1345 }

1346 \BNVS_new:cpn { match_pop:cc } #1 {
1347     \__bnvs_match_pop:c { #1 }
1348     \__bnvs_match_pop:c
1349 }

1350 \BNVS_new:cpn { match_pop:ccc } #1 {
1351     \__bnvs_match_pop:c { #1 }
1352     \__bnvs_match_pop:cc
1353 }

1354 \BNVS_new:cpn { match_pop:cccc } #1 {
1355     \__bnvs_match_pop:c { #1 }
1356     \__bnvs_match_pop:ccc
1357 }

1358 \BNVS_new:cpn { match_pop:ccccc } #1 {
1359     \__bnvs_match_pop:c { #1 }
1360     \__bnvs_match_pop:cccc
1361 }

```

\c__bnvs_A_IKSP_Z_regex A qualified dotted name is the qualified name of an overlay set possibly followed by a dotted path. Matches the whole string. Four capture groups.

(End of definition for \c__bnvs_A_IKSP_Z_regex.)

```

1362 \regex_const:Nn \c__bnvs_A_IKSP_Z_regex {

```

1: the frame $\langle id \rangle$
 2: the exclamation mark

```
1363 \A (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?
```

3: The short name.

```
1364 ( \ur{c__bnvs_S_regex} )
```

4: the path, if any.

```
1365 ( \ur{c__bnvs_P_regex} ) \Z
1366 }
```

```
\__bnvs_if_ISP:nTF \__bnvs_if_ISP:nTF {<name>} {<yes:>} {<no:>}
\__bnvs_if_ITN:nTF {<name>} {<yes:>} {<no:>}
\__bnvs_if_ISP:nnTF {<root>} {<relative>} {<yes:>} {<no:>}
```

If $\langle name \rangle$ is a reference, put the frame id it defines, or the current frame id, into I the short name into S, the dotted path into P, or the tag into T and the normalized number index into N, if any, then execute $\langle yes: \rangle$. Otherwise execute $\langle no: \rangle$.

The second version calls the first one with $\langle name \rangle$ equals $\langle relative \rangle$ prepended with $\langle root \rangle$.

```
1367 \BNVS_new_conditional:cpnn { if_ISP:n } #1 { T, F, TF } {
1368   \BNVS_begin:
1369   \__bnvs_match_if_once:NnTF \c__bnvs_A_IKSP_Z_regex { #1 } {
1370     \__bnvs_match_pop:
1371     \__bnvs_match_pop:cccc IKSP
1372     \cs_set:Npn \BNVS_if_ISP:nnn ##1 ##2 ##3 {
1373       \BNVS_end:
1374       \__bnvs_tl_set:cn I { ##1 }
1375       \__bnvs_tl_set:cn S { ##2 }
1376       \__bnvs_tl_set:cn P { ##3 }
1377     }
1378     \__bnvs_tl_if_empty:cTF K {
1379       \BNVS_tl_use:Nvvv \BNVS_if_ISP:nnn J
1380     } {
1381       \BNVS_tl_use:Nvvv \BNVS_if_ISP:nnn I
1382     } S P
1383     \__bnvs_tl_set:cv T P
1384     \__bnvs_tl_put_left:cv T S
1385     \__bnvs_tl_set:cv J I
1386     \prg_return_true:
1387   } {
1388     \BNVS_end:
1389     \prg_return_false:
1390   }
1391 }

1392 \BNVS_new_conditional:cpnn { if_ITN:n } #1 { T, F, TF } {
1393   \BNVS_begin:
```

```

1394 \__bnvs_match_if_once:NnTF \c__bnvs_A_IKPN_Z_regex { #1 } {
1395   \__bnvs_match_pop:
1396   \__bnvs_match_pop:cccc IKPN
1397   \cs_set:Npn \BNVS_if_ITN:w ##1 ##2 ##3 {
1398     \BNVS_end:
1399     \__bnvs_tl_set:cn I { ##1 }
1400     \__bnvs_tl_set:cn T { ##2 }
1401     \__bnvs_tl_set:cn N { ##3 }
1402     \__bnvs_normalize_N:
1403   }
1404   \__bnvs_tl_if_empty:cTF K {
1405     \BNVS_tl_use:Nvvv \BNVS_if_ITN:w J
1406   } {
1407     \BNVS_tl_use:Nvvv \BNVS_if_ITN:w I
1408   } T N
1409   \__bnvs_tl_set:cv J I
1410   \prg_return_true:
1411 } {
1412   \BNVS_end:
1413   \prg_return_false:
1414 }
1415 }

```

6.11.3 Path utilities

`\c__bnvs_A_PN_Z_regex` Matches the whole string.

(End of definition for `\c__bnvs_A_PN_Z_regex`.)

```

1416 \regex_const:Nn \c__bnvs_A_PN_Z_regex {

```

1: The full match,

2: the path

```

1417 \A ( \ur{c__bnvs_S_regex} \ur{c__bnvs_P_regex} )

```

3: The sign and the value of the trailing digits component.

```

1418 (?: \. \s * ( [-+]? \s* \d+ ) )? \Z

```

```

1419 }

```

```

\__bnvs_if_PN:nTTF \__bnvs_if_PN:nTTF {<path>} {<yes::n>} {<yes::nn>} {<no::>}

```

If `<path>` is a correct path with no trailing number, execute `<yes::n>` with the whole `<path>` as argument. If `<path>` is a correct path with a trailing number, executes `<yes::nn>` with the whole `<path>` but its last component as first argument and the normalized number as second arguments. If `<path>` is not a correct path, execute `<no::>`.

```

1420 \BNVS_new:cpn { if_PN:nTTF } #1 #2 #3 #4 {

```

```

1421   \BNVS_begin:

```

```

1422 \__bnvs_match_if_once:NnTF \c__bnvs_A_PN_Z_regex { #1 } {
1423   \__bnvs_match_pop:
1424   \__bnvs_match_pop:cc PN
1425   \__bnvs_tl_if_empty:cTF { N } {
1426     \cs_set:Npn \BNVS_if_path_nTTF:w ##1 { \BNVS_end: #2 }
1427     \BNVS_tl_use:Nv \BNVS_if_path_nTTF:w P
1428   } {
1429     \__bnvs_normalize_N:
1430     \cs_set:Npn \BNVS_if_path_nTTF:w ##1 ##2 { \BNVS_end: #3 }
1431     \BNVS_tl_use:Nvv \BNVS_if_path_nTTF:w P N
1432   }
1433 } {
1434   \BNVS_end:
1435   #4
1436 }
1437 }

```

6.11.4 Other utilities

```

1438 \BNVS_new:cpn { warn_until_s_stop:w } #1 \s_stop {
1439   \tl_if_empty:nF { #1 } {
1440     \BNVS_warning:n { Ignored:~#1 }
1441   }
1442 }
1443 \BNVS_new:cpn { scan_until_s_stop:w } {
1444   \peek_meaning:NnTF \s_stop {
1445     \use_none:n
1446   } {
1447     \__bnvs_warn_until_s_stop:w
1448   }
1449 }

```

```

\__bnvs_peek_branch_until_s_stop:nnnnw \__bnvs_peek_branch_until_s_stop:nnnnw
{<square:n>} {<curly:n>} {<non empty:n>} {<empty:>}
{...} \s_stop

```

Branch according to the following token. Each argument, except `<empty:>`, is the body of a function that takes one argument. It catches errors like `foo=[bar]baz`, where `baz` is not expected: no other argument should lay before `\s_stop`.

```

1450 \cs_new:Npn \BNVS_square:n { \exp_not:N \BNVS_square:n }
1451 \cs_new:Npn \BNVS_curly:n { \exp_not:N \BNVS_curly:n }
1452 \BNVS_new:cpn { peek_branch_until_s_stop:nnnnw } #1 #2 #3 #4 {
1453   \peek_meaning:NnTF \BNVS_square:n {
1454     \cs_set:Npn \BNVS_peek_branch:n ##1 { #1 }
1455     \cs_set:Npn \BNVS_peek_branch:nn ##1 ##2 {
1456       \BNVS_peek_branch:n { ##2 }
1457       \__bnvs_warn_until_s_stop:w
1458     }
1459     \BNVS_peek_branch:nn
1460   } {
1461     \peek_meaning:NnTF \BNVS_curly:n {

```

```

1462     \cs_set:Npn \BNVS_peek_branch:n ##1 { #2 }
1463     \cs_set:Npn \BNVS_peek_branch:nn ##1 ##2 {
1464         \BNVS_peek_branch:n { ##2 }
1465         \__bnvs_warn_until_s_stop:w
1466     }
1467     \BNVS_peek_branch:nn
1468 } {
1469     \peek_meaning:NTF \s_stop {
1470         #4
1471         \use_none:n
1472     } {
1473         \cs_set:Npn \BNVS_peek_branch:w ##1 \s_stop {
1474             #3
1475         }
1476         \BNVS_peek_branch:w
1477     }
1478 }
1479 }
1480 }

```

__bnvs_keyval:ncc __bnvs_keyval:ncc {<definitions>} {<V>} {<a>}

Parse definitions into the named tl variables (which should be different). Outer braces in definitions are kept. It is meant to be called within a group. On return <V> contains the leading standalone value, if any, <a> is a list of `\BNVS:nn{<key>}{<value>}` and `\BNVS:n{<key or value>}`.

```

1481 \BNVS_new:cpn { keyval:ncc } #1 #2 #3 {
1482     \cs_set:Npn \BNVS: {
1483         \cs_set:Npn \BNVS:n #####1 {
1484             \__bnvs_tl_put_right:cn { #3 } { \BNVS:n { #####1 } }
1485         }
1486         \cs_set:Npn \BNVS:nn #####1 #####2 {
1487             \__bnvs_tl_put_right:cn { #3 } { \BNVS:nn { #####1 } { #####2 } }
1488         }
1489     }
1490     \cs_set:Npn \BNVS:n ##1 {
1491         \BNVS:
1492         \__bnvs_tl_set:cn { #2 } { ##1 }
1493     }
1494     \cs_set:Npn \BNVS:nn {
1495         \BNVS:
1496         \BNVS:nn
1497     }
1498     \__bnvs_tl_set:cn { #2 } { #1 }
1499     \BNVS_tl_use:nc {
1500         \regex_replace_all:nnN { \cB . } { \c{BNVS:} \0 }
1501     } { #2 }
1502     \__bnvs_tl_clear:c { #3 }
1503     \BNVS_tl_use:nv {
1504         \__bnvs_tl_clear:c { #2 }
1505         \keyval_parse:NNn \BNVS:n \BNVS:nn
1506     } { #2 }
1507     \BNVS_tl_use:nc {

```

```

1508   \regex_replace_all:nnN { \c{BNVS:} } { }
1509 } { #2 }
1510 \BNVS_tl_use:nc {
1511   \regex_replace_all:nnN { \c{BNVS:} } { }
1512 } { #3 }
1513 }

1514 \BNVS_new:cpn { normalize_N: } {
1515   \tl_if_empty:NF \l__bnvs_N_tl {
1516     \exp_args:NNe \tl_set:Nn \l__bnvs_N_tl {
1517       \exp_last_unbraced:NV \int_value:w \l__bnvs_N_tl \exp_stop_f:
1518     }
1519   }
1520 }

```

6.11.5 Next index

```

1521 \BNVS_int_new:c { next_index }
1522 \BNVS_new:cpn { next_index:nn } #1 #2 {
1523   \__bnvs_int_incr:c { next_index }
1524   \exp_args:Nnne \__bnvs_is_gset:nnnT { #1 } { #2 } {
1525     = \int_use:N \l__bnvs_next_index_int
1526   } {
1527     \__bnvs_next_index:nn { #1 } { #2 }
1528   }
1529 }

```

`\__bnvs_next_index:nnn` `\__bnvs_next_index:nnn {<id>} {<path>} {<code:n>}`

`<code:n>` takes one argument: the next available tag.

```

1530 \BNVS_new:cpn { next_index:nnn } #1 #2 #3 {
1531   \BNVS_begin:
1532   \__bnvs_int_zero:c { next_index }
1533   \__bnvs_next_index:nn { #1 } { #2 }
1534   \cs_set:Npn \BNVS:n ##1 {
1535     \BNVS_end:
1536     \__bnvs_int_set:cn { next_index } { ##1 }
1537   } #3
1538 }
1539 \BNVS_int_use:Nv \BNVS:n { next_index }
1540 }

```

6.12 Definition

Lower level function names generally contain `_gdef`. We parse definitions and collect material between curly or square brackets. There is a pretreatment before the storage to anticipate the resolution.

`\__bnvs_gdef_by_key:nnn` `\__bnvs_gdef_by_key:nnn {<id>} {<path>} {<definitions>}`
`\__bnvs_gdef_by_key:(vvn|nvn)`

Here is the [documentation](#). Example: `<id>{<path>={<definitions>}}`

```

1541 \BNVS_new:cpn { gdef_by_key:nnn } #1 #2 #3 {
1542   \tl_if_empty:nTF { #3 } {
1543     \__bnvs_gclear:nn { #1 } { #2 }
1544   } {
1545     \BNVS_begin:
1546     \keyval_parse:nnn {
1547       \__bnvs_ginit:nnn { #1 } { #2 }
1548     } {
1549       \__bnvs_gdef_by_key:nnnn { #1 } { #2 }
1550     } { #3 }
1551     \BNVS_end:
1552   }
1553 }

1554 \BNVS_new:cpn { gdef_by_key:nvn } #1 {
1555   \BNVS_tl_use:nv { \__bnvs_gdef_by_key:nnn { #1 } }
1556 }

1557 \BNVS_new:cpn { gdef_by_key:vvv } {
1558   \BNVS_tl_use:Nvv \__bnvs_gdef_by_key:nnn
1559 }

```

For $\langle id \rangle! \langle path \rangle = \langle def \rangle$ where $\langle def \rangle$ is the most general one. Workflow: `\__bnvs_gclear:nn` or

```

▶ \__bnvs_keyval:ncc
▶ \__bnvs_gclear:nn or \__bnvs_gdef_square_keyval:nnn
▶ \__bnvs_gclear:nn or \__bnvs_gdef_by_key:nnn
▶ \__bnvs_ginit:nnn.

```

`\__bnvs_keyval_IPND:w` `\__bnvs_keyval_IPND:w {<id> {<path> {<index> {<definitions>}}`

Parses the definitions as a standalone value. Other values are errors. It does not mean that standalone values are free from errors by themselves.

```

1560 \BNVS_new:cpn { keyval_IPND:w } #1 #2 #3 #4 {
1561   \tl_if_empty:nTF { #4 } {
1562     \__bnvs_gunset:nnn { #1 } { #2 } { =#3 }
1563     \__bnvs_gunset:nnn { #1 } { #2 } { #3 }
1564   } {
1565     \BNVS_begin:
1566     \__bnvs_keyval:ncc { #4 } V a
1567     \BNVS_tl_last_unbraced:nv {
1568       \__bnvs_peek_branch_until_s_stop:nnnnw {
1569         \BNVS_error:n { No~`[...]~allowed~with~index. }
1570       } {

```

For `ID!TAG.1={FOO}`, go recursive.

```

1571   \__bnvs_keyval_IPND:w { #1 } { #2 } { #3 } { ##1 }
1572   } {
1573     \__bnvs_ginit_IPND:w { #1 } { #2 } { #3 } { \BNVS:n { #4 } }

```


Support \BNVS_value:n

```

1574 \__bnvs_ginit:nnn { #1 } { #2.#3 } { #4 }
1575 \__bnvs_is_gset:nnnF { #1 } { #2 } 0 {
1576 \__bnvs_gset:nnne { #1 } { #2 } 0 {
1577 \exp_not:n { \BNVS_value:n { #4 } - } \int_eval:n { #3 - 1 }
1578 }
1579 }
1580 } {
1581 }
1582 } V \s_stop
1583 \__bnvs_tl_if_empty:cF a {
1584 \cs_set:Npn \BNVS:n ##1 { \exp_not:n { ##1, } }
1585 \cs_set:Npn \BNVS:nn ##1 ##2 { \exp_not:n { ##1 = ##2, } }
1586 \BNVS_error:x { Ignored:~\__bnvs_tl_use:c a }
1587 }
1588 \BNVS_end:
1589 }
1590 }

```

6.12.1 Material between square brackets

Here is the [documentation](#). The name of the function contains “square”. Workflow:

```

▶ \__bnvs_gdef_square:nnnn
▶ \__bnvs_gdef_square_keyval:nnn
▶ \__bnvs_gdef_square:nnn or \__bnvs_gdef_square:nnnn
▶ \__bnvs_gdef_square_keyval:nnn or \__bnvs_ginit:nnn.
▶ \__bnvs_if_set:ncccTF \__bnvs_gunset_deep:nn.

```

Examples: Id!Tag=[1=A,B,3=C].

```

\__bnvs_gdef_square:nnn \__bnvs_gdef_square:nnn {<id>} {<path>} {<definition>}

```

To parse what is inside square brackets. \BNVS_V:w{<definition>} is stored.

For:

- <id>!<path>=[<definitions>],
- <id>!<path>=[<definition>] and
- <id>!<path>=[<index>=<definitions>].

```

1591 \BNVS_new:cpn { gdef_square:nnn } #1 #2 #3 {

```

Find the first available index.

```

1592 \__bnvs_next_index:nnn { #1 } { #2 } {
1593 \__bnvs_ginit_IPND:w { #1 } { #2 } { ##1 } { #3 }
1594 }
1595 }

```

```

\__bnvs_if_index:nTF \__bnvs_if_index:nTF {<what>} {<yes::n>} {<no::>}

```

If <what> is an index, execute <yes::n> otherwise execute <no::>. The <yes::n> argument is a function definition with one argument: the normalized index built from <what>.

```

1596 \BNVS_new:cpn { if_index:nTF } #1 #2 #3 {
1597   \BNVS_begin:
1598   \__bnvs_match_if_once:NnTF \c__bnvs_A_index_Z_regex { #1 } {
1599     \__bnvs_match_pop:c N
1600     \__bnvs_normalize_N:
1601     \cs_set:Npn \BNVS:n ##1 {
1602       \BNVS_end:
1603       #2
1604     }
1605     \BNVS_tl_use:Nv \BNVS:n N
1606   } {
1607     \BNVS_end:
1608     #3
1609   }
1610 }

```

__bnvs_gdef_square:nnnn __bnvs_gdef_square:nnnn {<id>} {<path>} {<index>} {<definition>}

To parse what is inside square brackets. \BNVS_V:w{<definition>} is stored.

```

1611 \BNVS_new:cpn { gdef_square:nnnn } #1 #2 #3 #4 {
1612   \__bnvs_if_index:nTF { #3 } {
1613     \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } { = ##1 } {
1614       \__bnvs_ginit_IPND:w { #1 } { #2 } { ##1 } { #4 }
1615     }
1616   } {
1617     \__bnvs_next_index:nnn { #1 } { #2 } {
1618       \__bnvs_ginit_IPND:w { #1 } { #2 } { ##1 } { #4 }
1619     }
1620   }
1621 }

```

__bnvs_gdef_square_keyval:nnn __bnvs_gdef_square_keyval:nnn {<id>} {<path>} {<definitions>}

To parse what is inside square brackets. \BNVS_V:w{<definition>} is stored.
Example: X=[<definitions>].

```

1622 \BNVS_new:cpn { gdef_square_keyval:nnn } #1 #2 #3 {
1623   \tl_if_empty:nTF { #2 } {
1624     \BNVS_error:n { Unexpected~list~at~top~level. }
1625   } {
1626     \tl_if_empty:nF { #3 } {
1627       \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
1628         \__bnvs_gunset_deep:nn { #1 } { #2 }
1629         \BNVS_begin:
1630         \keyval_parse:nnn
1631           { \__bnvs_gdef_square:nnn { #1 } { #2 } }
1632           { \__bnvs_gdef_square:nnnn { #1 } { #2 } }
1633           { #3 }
1634         \BNVS_end:
1635       }
1636     }
1637   }

```

```

1638 }

1639 \BNVS_new:cpn { gdef_square_keyval:nvn } #1 {
1640   \BNVS_tl_use:nv { \_bnvs_gdef_square_keyval:nnn { #1 } }
1641 }

1642 \BNVS_new:cpn { gdef_square_keyval:vvv } {
1643   \BNVS_tl_use:Nvv \_bnvs_gdef_square_keyval:nnn
1644 }

1645 \BNVS_new:cpn { gdef_square_keyval_I:nn } {
1646   \BNVS_tl_use:cv { gdef_square_keyval:nnn } I
1647 }

1648 \BNVS_new:cpn { gdef_square_keyval_I:vn } {
1649   \BNVS_tl_use:cv { gdef_square_keyval_I:nn }
1650 }

```

6.12.2 Range or value lists

```

\_bnvs_root_parsed:n \_bnvs_root_parsed:n {<key>}
\_bnvs_root_parsed:nn \_bnvs_root_parsed:nn {<key>} {<value>}

```

```

1651 \BNVS_new:cpn { root_parsed:n } #1 {
1652   \BNVS_if_ITN:nTF { \token_to_str:c { #1 } } {
1653     \_bnvs_tl_if_blank:vTF N {
1654       \STOP
1655       \_bnvs_ginit:vvv IP 1
1656     } {
1657       \STOP
1658       \_bnvs_gundef_N:vvv IPN
1659     }
1660   } {
1661     \BNVS_error:n { Unsupported-#1 }
1662   }
1663 }

1664 \BNVS_new:cpn { root_parsed:nn } #1 #2 {
1665 }

```

```

\_bnvs_root_keyval:nc \_bnvs_root_keyval:nc {<definitions>} {<aux>}

```

`<aux>` is an auxiliary variable name unused on return. Is called by `\BeanovesReset` and `\_bnvs_root_keyval:NNn`.

1. `x`-expands `<definitions>` into the `<aux>` variable.
2. prepend any `TEX` group with `\BNVS_curly:n` such that outer braces are not removed from values.
3. replace any `[<...>]` with `\BNVS_bracket:n{<...>}`, testing for unbalanced delimiter,
4. Parse with keyval based on `\keyval_parsed:nn`.

```

1666 \BNVS_new:cpn { root_keyval:nc } #1 #2 {
1667   \__bnvs_tl_set:cx { #2 } { #1 }
1668   \BNVS_tl_use:nc {
1669     \regex_replace_all:nnN { \cB . } { \c { BNVS_curly:n } \0 }
1670   } { #2 }
1671   \cs_set:Npn \BNVS:N ##1 {
1672     \cs_set:Npn \BNVS_loop: {
1673       \regex_replace_all:nnNT { \[ (.*?) \] } {
1674         \c { BNVS_by_index:n } \cB \{ \1 \cE \}
1675       } ##1 {
1676         \BNVS_loop:
1677       }
1678     }
1679     \BNVS_loop:
1680     \regex_match:nVT { \[|\] } ##1 {
1681       \BNVS_error:n { Unbalanced~[|] }
1682     }
1683   }
1684   \BNVS_tl_use:Nc \BNVS:N { #2 }
1685   \BNVS_tl_use:nv {
1686     \keyval_parse:NNn \__bnvs_root_parsed:n \__bnvs_root_parsed:nn
1687   } { #2 }
1688 }

```

`\__bnvs_root_keyval:NNn` `\__bnvs_root_keyval:NNn` *<is at top>* *<on provide mode>* *{<definitions>}*

Calls `\__bnvs_root_keyval:nc` after some setup. Called by `\Beanoves`. The a tl auxiliary variable is used.

```

1689 \BNVS_new:cpn { root_keyval:NNn } #1 #2 #3 {
1690   \bool_if:NT #1 {

```

At the document level, clear everything.

```

1691     \__bnvs_gclear:
1692   }
1693   \BNVS_begin:
1694   \bool_if:NTF #2 {
1695     \__bnvs_provide_ON:
1696   } {
1697     \__bnvs_provide_OFF:
1698   }
1699   \__bnvs_int_zero:c { i }
1700   \__bnvs_tl_set:cn a { #3 }
1701   \bool_if:NT #1 {

```

At the document level, use the global definitions.

```

1702     \seq_if_empty:NF \g__bnvs_def_seq {
1703       \__bnvs_tl_put_left:cx a {
1704         \seq_use:Nn \g__bnvs_def_seq , ,
1705       }
1706     }
1707   }
1708   \BNVS_tl_use:Nv \__bnvs_root_keyval:nc a a

```

This is a list, possibly at the top

```
1709 \BNVS_end_tl_set:cv J J
1710 }
```

\Beanoves `\Beanoves {<key-value list>}`

The keys are the slide overlay references. When no value is provided, it defaults to 1. On the contrary, <key-value> items are parsed by `\__bnvs_parse:nn`.

```
1711 \NewDocumentCommand \Beanoves { sm } {
1712   \__bnvs_set_false:c { reset }
1713   \__bnvs_set_false:c { reset_all }
1714   \__bnvs_set_false:c { only }
1715   \tl_if_empty:NTF \@currenvir {
```

We are most certainly in the preamble, record the definitions globally for later use.

```
1716   \seq_gput_right:Nn \g__bnvs_def_seq { #2 }
1717 } {
1718   \tl_if_eq:NnTF \@currenvir { document } {
1719     \IfBooleanTF {#1} {
1720       \__bnvs_root_keyval:NNn \c_true_bool \c_true_bool
1721     } {
1722       \__bnvs_root_keyval:NNn \c_true_bool \c_false_bool
1723     }
1724   } {
1725     \IfBooleanTF {#1} {
1726       \__bnvs_root_keyval:NNn \c_false_bool \c_true_bool
1727     } {
1728       \__bnvs_root_keyval:NNn \c_false_bool \c_false_bool
1729     }
1730   } { #2 }
1731   \ignorespaces
1732 }
1733 }
```

If we use the frame `beanoves` option, we can provide default values to the various name ranges.

```
1734 \define@key{beamerframe}{beanoves}{\Beanoves*{#1}}
```

6.13 Scanning named overlay specifications

Patch some beamer commands to support `?(...)` instructions in overlay specifications.

<code>__bnvs_@frame</code>	<code>__bnvs_@frame {<overlay specification>}</code>
<code>__bnvs_@masterdecode</code>	<code>__bnvs_@masterdecode {<overlay specification>}</code>

Preprocess <overlay specification> before beamer reads it.

`\l__bnvs_ans_tl` Storage for the translated overlay specification, <...> instructions are replaced by their static counterparts.

(End of definition for `\l__bnvs_ans_tl`.)

Save the original macros `\beamer@frame` and `\beamer@masterdecode` then override them to properly preprocess the argument. We start by defining the overloads.

```

1735 \makeatletter
1736 \cs_set:Npn \__bnvs_@frame < #1 > {
1737   \BNVS_begin:
1738   \__bnvs_tl_clear:c { ans }
1739   \__bnvs_resolve:nc { #1 } { ans }
1740   \BNVS_set:cpn { :n } ##1 { \BNVS_end: \BNVS_saved@frame < ##1 > }
1741   \BNVS_tl_use:cv { :n } { ans }
1742 }

1743 \cs_set:Npn \__bnvs_@masterdecode #1 {
1744   \BNVS_begin:
1745   \__bnvs_tl_clear:c { ans }
1746   \__bnvs_resolve:nc { #1 } { ans }
1747   \BNVS_tl_use:nv {
1748     \BNVS_end:
1749     \BNVS_saved@masterdecode
1750   } { ans }
1751 }

1752 \cs_new:Npn \BeanovesOff {
1753   \cs_set_eq:NN \beamer@frame \BNVS_saved@frame
1754   \cs_set_eq:NN \beamer@masterdecode \BNVS_saved@masterdecode
1755 }

1756 \cs_new:Npn \BeanovesOn {
1757   \cs_set_eq:NN \beamer@frame \__bnvs_@frame
1758   \cs_set_eq:NN \beamer@masterdecode \__bnvs_@masterdecode
1759 }

1760 \AddToHook{begindocument/before}{
1761   \cs_if_exist:NTF \beamer@frame {
1762     \cs_set_eq:NN \BNVS_saved@frame \beamer@frame
1763     \cs_set_eq:NN \BNVS_saved@masterdecode \beamer@masterdecode
1764   } {
1765     \cs_set:Npn \BNVS_saved@frame < #1 > {
1766       \BNVS_error:n {Missing~package~beamer}
1767     }
1768     \cs_set:Npn \BNVS_saved@masterdecode < #1 > {
1769       \BNVS_error:n {Missing~package~beamer}
1770     }
1771   }
1772   \BeanovesOn
1773 }
1774 \makeatother

```

6.14 Resolution

Resolution means to transform $?(\langle \text{queries} \rangle)$ and alike into static beamer range expressions. This is a quite complex task because we allow at the same time algebraic expressions and beamer range expressions which seem like differences but are not algebraic expressions by themselves. Function names generally contain “resolve”.

Overall sketch: `\_bnvs_resolve:nc` is the top level resolution function. It is called by `\_bnvs_@frame`, `\_bnvs_@masterdecode` or `\BeanovesResolve` and for the resolution of embedded subqueries $?(\langle \text{queries} \rangle)$ or $!(\langle \text{queries} \rangle)$ as well, it calls `\_bnvs_resolve_query:nc`, which in turns calls `\_bnvs_resolve_queries:nc`. The main functions used for resolution are

❗ `\_bnvs_resolve_query:nc`, called first, the input is a raw query, which is not a clist object, on return the answer is **(IN PROGRESS)**

- `\BNVS_V:w{\langle v \rangle}`, for a single integer literal value $\langle v \rangle$,
- `\BNVS_AZ:w{\langle a \rangle}{\langle z \rangle}`, for a single range of integers from literal integer $\langle a \rangle$ to literal integer $\langle z \rangle$, or from literal integer $\langle a \rangle$ if $\langle z \rangle$ is empty,
- a comma separated list of the above, ordered, with no overlap and no contiguous ranges

The result may be used as a single integer value as well as a beamer list of ranges. Depending on the context, the functions will be defined appropriately.

❗ `\_bnvs_resolve_prepare:nc`, called first by `\_bnvs_resolve_query:nc`, on return the prepared material consists of

- `\BNVS_raw_to_stop:w{raw text}\s_stop` for everything not special, including commas,
- `\BNVS_paren:n{\dots}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{\langle b \rangle}{\langle I \rangle}{\langle K \rangle}{\langle D \rangle}{\langle P \rangle}{\langle T \rangle}{\langle N \rangle}{\langle U \rangle}{\langle a \rangle}`, for specifications
- `\BNVS_Ua:w{\langle U \rangle}{\langle a \rangle}` for static attributes after a “.” character and following ++,
- `\BNVS_n:n{\dots}` for an integer expression between square brackets,
- `\BNVS_s:n{\dots}` for a general subquery inside $?(\langle \dots \rangle)$,
- `\BNVS_assign:n{\langle /+/? \rangle}`, for =, += or ?=,
- `\::BNVS:` and `\:BNVS:` are range delimiters

❗ `\_bnvs_resolve_subqueries:c`, after resolution of subqueries the material consists of

- `\BNVS_raw:n{\langle raw text \rangle}` for everything not special,
- `\BNVS_bIPTNUa:w{\langle b \rangle}{\langle I \rangle}{\langle P \rangle}{\langle T \rangle}{\langle N \rangle}{\langle U \rangle}{\langle a \rangle}`, for specifications
- `\BNVS_Ua:w{\langle attributes \rangle}{\langle number of ++ \rangle}`,
- `\BNVS_N:w{\dots}` for a single static resolved integer,
- `\BNVS_S:w{\dots}` containing output similar to that of `\_bnvs_resolve_query:nc`,
- `\BNVS_assign:n{\langle /+/? \rangle}`, for =, += or ?=,

- `\::BNVS:` and `\:BNVS:`, range delimiters.

¶ `\_bnvs_resolve_check:c` after resolution the material consists of

- `\BNVS_raw:n{⟨raw text⟩}` for everything not special,
- `\BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
- `\BNVS_N:w{⟨...⟩}` for a single static resolved integer,
- `\BNVS_S:w{⟨...⟩}` for a comma separated list of static resolved $\langle A \rangle$, $\langle A \rangle:$ and $\langle A \rangle:\langle Z \rangle$ ranges, $\langle A \rangle$ and $\langle Z \rangle$ being unsigned integer decimal denotations,
- `\BNVS_assign:n{⟨/+/?⟩}`, for $=$, $+=$ or $?=$,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

¶ `\_bnvs_resolve_rhs:c` after resolution of assignments the material consists of
(IN PROGRESS)

- `\BNVS_V:w{⟨v⟩}`, for a single integer literal value $\langle v \rangle$,
- `\BNVS_AZ:w{⟨a⟩}{⟨z⟩}`, for a single range of integers from literal integer $\langle a \rangle$ to literal integer $\langle z \rangle$, or from literal integer $\langle a \rangle$ if $\langle z \rangle$ is empty,
- a comma separated list of the above, ordered, with no overlap and no contiguous ranges

¶ `\_bnvs_resolve_assign:nc` after resolution of assignments the material consists of

- `\BNVS_raw:n{⟨raw text⟩}` for everything not special,
- `\BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
- `\BNVS_N:w{⟨...⟩}` for a single static resolved integer,
- `\BNVS_S:w{⟨...⟩}` for a comma separated list of static resolved $\langle A \rangle$, $\langle A \rangle:$ and $\langle A \rangle:\langle Z \rangle$ ranges, $\langle A \rangle$ being positive and $\langle Z \rangle$ being nonnegative,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

¶ `\_bnvs_assign:Nwnnc`

The top level queries are a comma separated list of individual queries as augmented range specifications. Each individual query is temporarily stored in its own variable, then commands are inserted as markup. This variable is then executed to feed the top level resolution variable.

The augmentations in individual queries can take different forms. At the top level we have only $?(\langle queries \rangle)$ or $!(\langle queries \rangle)$ and inside we may have

- embedded $?(\langle queries \rangle)$ or $!(\langle queries \rangle)$,
- colon delimited ranges,
- various assignments (standard, with increment and as provider),
- references with possible square brackets specification,
- pre and post increment.

The right hand side expression in assignments may contain commas. In that case there can be a conflict with top level commas. To avoid such ambiguity, one can put rounded brackets around the critical part.

`\c__bnvs_bIKDPN_regex` Used by `\__bnvs_resolve_prepare:nc`.

```

1775 \regex_const:Nn \c__bnvs_bIKDPN_regex {
1776   ( (? : \+ \+ ) * )
1777   (? : ( \ur{c__bnvs_S_regex} ) ? ( ! ) | ( \. ) ) ?
1778   ( \ur{c__bnvs_S_regex} ( ? : \. \ur{c__bnvs_S_regex} ) * )
1779   ( ? : \. \s* ( [-+\s]* \s* \d+ ) ) ?
1780   \s*
1781 }
```

(End of definition for \c__bnvs_bIKDPN_regex.)

`\c__bnvs_bIKDPTNUa_regex` Used by `\__bnvs_resolve_prepare:nc`.

- ⟨b⟩: an optional collection of ++ prefixes,
- ⟨I⟩: an optional ⟨id⟩
- ⟨K⟩: followed by a “!” or
- ⟨D⟩: an optional “.”,
- ⟨P⟩: a dotted path,
- ⟨T⟩: optional dotted ⟨attributes⟩,
- ⟨N⟩: a litteral optionally signed integer,
- ⟨U⟩: other optional dotted ⟨attributes⟩,
- ⟨a⟩: an optional collection of ++ suffixes.

```

1782 \regex_const:Nn \c__bnvs_bIKDPTNUa_regex {
1783   ( (? : \+ \+ ) * )
1784   (? : ( \ur{c__bnvs_S_regex} ) ? ( ! ) | ( \. ) ) ?
1785   ( \ur{c__bnvs_S_regex} ( ? : \. \ur{c__bnvs_S_regex} ) * )
1786   ( (? : \. \ur{c__bnvs_U_regex} ) * )
1787   ( ? : \. \s* ( [-+\s]* \s* \d+ ) ) ?
1788   ( (? : \. \ur{c__bnvs_U_regex} ) * )
1789   ( (? : \+ \+ ) * )
1790   \s*
1791 }
```

(End of definition for \c__bnvs_bIKDPTNUa_regex.)

`\c__bnvs_UaU_regex` Used by `\__bnvs_resolve_prepare:nc`.

- ⟨U⟩: optional dotted ⟨attributes⟩ and
- ⟨a⟩: post increment,
- ⟨U⟩: or standalone optional dotted ⟨attributes⟩.

```

1792 \regex_const:Nn \c__bnvs_UaU_regex {
1793   (? : ( (? : \. \ur{c__bnvs_U_regex} ) * ) ( (? : \+)+ )
1794     | ( (? : \. \ur{c__bnvs_U_regex} ) + ) )
1795   \s*
1796 }

```

(End of definition for `\c__bnvs_UaU_regex`.)

```

\__bnvs_resolve_prepare:nc \__bnvs_resolve_prepare:nc {<queries>} {<var>}

```

Called by `\__bnvs_resolve_query:nc` and by `\__bnvs_resolve_queries:nc`. Prepare the `<queries>` into `<var>`. Inside `<queries>`, ranges are denoted with a colon syntax.

On return `<var>` consists of

- `\BNVS_raw_to_stop:w<raw text>\s_stop` for everything not special, including commas,
- `\BNVS_paren:n{<...>}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{<b>}{<I>}{<K>}{<D>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_Ua:w{<U>}{<a>}` for static attributes after a “.” character and following ++,
- `\BNVS_n:n{<...>}` for an integer expression between square brackets,
- `\BNVS_s:n{<...>}` for a general subquery inside `?(<...>)`,
- `\BNVS_assign:n{</+/?>}`, for `=`, `+=` or `?=`,
- `\::BNVS:` and `\:BNVS:` are range delimiters

```

1797 \BNVS_new:cpn { resolve_prepare:nc } #1 #2 {

```

❗ One shot function, subsequent calls within the same group are just restricted to the raw assignment of `<queries>` to `<var>`, because the `<queries>` argument here is some part of a more general `<queries>` that has already been prepared.

```

1798   \BNVS_set:cpn { resolve_prepare:nc } ##1 ##2 {
1799     \__bnvs_tl_set:cn { ##2 } { ##1 }
1800   }

```

❗ Turn the queries into a string by expanding macros, #2 will contain only characters.

```

1801   \__bnvs_tl_set:ce { #2 } { \exp_args:Nc \cs_to_str:N { #1 } }

```

❗ Enclose the material between commas, there is no `TEX` group yet so this is not a clist object. The leading and trailing marks will be inserted at the end of this function body.

```

1802   \BNVS_tl_use:Nc \tl_replace_all:Nnn { #2 }
1803     { , } { \s_stop , \BNVS_raw_to_stop:w}

```

❗ Replace “`<...>`” with

```

\s_stop
\BNVS_paren:n{\BNVS_raw_to_stop:w<...>\s_stop}
\BNVS_raw_to_stop:w.

```

Standard delimiters are expected to be properly balanced, no error recovery policy for that matter. Category codes now come back into play.

```

1804 \cs_set:Npn \BNVS_loop: {
1805   \BNVS_tl_use:nc { \regex_replace_all:nnNT { \ ( ( [^\(%)
1806     ]*? ) \) } {
1807     \c { s_stop }
1808     \c { BNVS_paren:n } \cB \{
1809       \c { BNVS_raw_to_stop:w } \1 \c { s_stop }
1810     \cE \}
1811     \c { BNVS_raw_to_stop:w }
1812   } } { #2 } \BNVS_loop:
1813 }
1814 \BNVS_loop:

```

❗ Replace “[⟨...⟩]” with

```

\s_stop
\BNVS_v:n{\BNVS_raw_to_stop:w⟨...⟩\s_stop}
\BNVS_raw_to_stop:w.

```

The content will be resolved as a static signed integer and will be used as an index.

```

1815 \cs_set:Npn \BNVS_loop: {
1816   \BNVS_tl_use:nc { \regex_replace_all:nnNT { \.* \ ( [^\)]%[
1817     ]* ) \] } {
1818     \c { s_stop } \c { BNVS_v:n } \cB \{
1819       \c { BNVS_raw_to_stop:w } \1 \c { s_stop }
1820     \cE \} \c { BNVS_raw_to_stop:w }
1821   } } { #2 } \BNVS_loop:
1822 }
1823 \BNVS_loop:

```

❗ Also replace references, increments, assignments and colons.

```

1824 \BNVS_tl_use:nc { \regex_replace_case_all:nN {

```

Basically, “?⟨...⟩” is replaced by “\BNVS_s:n{⟨...⟩}” that will be evaluated as a subquery for a list of values and ranges.

```

1825   { (?: \?|!: ) \c { s_stop } \c { BNVS_paren:n } } {
1826     \c { s_stop } \c { BNVS_s:n      }
1827   }
1828   { \c__bnvs_bIKDPTNUa_regex } {
1829     \c { s_stop }
1830     \c { BNVS_bIKDPTNUa:w }
1831     { \1 } { \2 } { \3 } { \4 } { \5 } { \6 } { \7 } { \8 } { \9 }
1832     \c { BNVS_raw_to_stop:w }
1833   }

```

Post increment operators and attributes are collected

```

1834   { \c__bnvs_UaU_regex } {
1835     \c { s_stop }
1836     \c { BNVS_Ua:w } \cB \{ \1\3 \cE \} \cB \{ \2 \cE \}
1837     \c { BNVS_raw_to_stop:w }
1838   }

```

Assignment operators together with their optional modifier are collected

```

1839 { \s* ([+?])?= \s* } {
1840   \c { s_stop }
1841   \c { BNVS_assign:n } \cB \{ \1 \cE \}
1842   \c { BNVS_raw_to_stop:w }
1843 }

```

Colon range delimiters:

```

1844 { \s* ::+ \s* } {
1845   \c { s_stop }
1846   \c { ::BNVS: }
1847   \c { BNVS_raw_to_stop:w }
1848 }
1849 { \s* : \s* } {
1850   \c { s_stop }
1851   \c { :BNVS: }
1852   \c { BNVS_raw_to_stop:w }
1853 }
1854 } } { #2 }
1855 \__bnvs_tl_put_left:cn { #2 } { \BNVS_raw_to_stop:w }
1856 \__bnvs_tl_put_right:cn { #2 } { \s_stop }
1857 }

```

```

__bnvs_resolve_subqueries:c  __bnvs_resolve_subqueries:c {<query var>}
__bnvs_resolve_subqueries:nc __bnvs_resolve_subqueries:nc {<query>} {<var>}

```

Called after `\__bnvs_resolve_prepare:nc`.

On input, `<query>` or `<var>` **consists of**

- `\BNVS_raw_to_stop:w<raw text>\s_stop` for everything not special, including commas,
- `\BNVS_paren:n{<...>}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{<b>}{<I>}{<K>}{<D>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_Ua:w{<U>}{<a>}` for static attributes after a “.” character and following ++,
- `\BNVS_n:n{<...>}` for an integer expression between square brackets,
- `\BNVS_s:n{<...>}` for a general subquery inside `?(<...>)`,
- `\BNVS_assign:n{<|+|?>}`, for =, += or ?=,
- `\::BNVS:` and `\:BNVS:` are range delimiters

and on output, `<var>` **consists of**

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_Ua:w{<.attributes>}{<number of ++>}`,
- `\BNVS_N:w{<...>}` for a single static resolved integer,

- `\BNVS_S:w{⟨...⟩}` containing output similar to that of `\__bnvs_resolve_query:nc`,
- `\BNVS_assign:n{⟨/+/?⟩}`, for =, += or ?=,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

The `⟨query var⟩` is executed after the commands it contains are defined on the run. In particular, `\BNVS_n:n` and `\BNVS_s:n` use `\__bnvs_resolve_query:nc`. Run inside a group.

```
1858 \BNVS_new:cpn { resolve_subqueries:nc } #1 #2 {
1859   \BNVS_begin:
```

¶ Raw text.

```
1860   \cs_set:Npn \BNVS_raw_to_stop:w ##1 \s_stop {
1861     \tl_if_blank:nF { ##1 } {
1862       \__bnvs_tl_put_right:cn Q { \BNVS_raw:n { ##1 } }
1863     }
1864   }
```

¶ Parenthesis:

```
1865   \cs_set:Npn \BNVS_paren:n ##1 {
1866     \__bnvs_tl_put_right:cn Q { \BNVS_raw:n }
1867     \__bnvs_resolve_subqueries:nc { ##1 } A
1868     \__bnvs_resolve_check:c A
1869     \__bnvs_resolve_query:c A
1870     \BNVS_head_N:c A
1871     \__bnvs_tl_put_right:ce Q { { ( \l__bnvs_A_tl ) } }
1872   }
```

¶ Integer value.

```
1873   \cs_set:Npn \BNVS_n:n ##1 {
1874     \__bnvs_tl_put_right:cn Q { \BNVS_N:w }
1875     \__bnvs_resolve_subqueries:nc { ##1 } A
1876     \__bnvs_resolve_check:c A
1877     \__bnvs_resolve_query:c A
1878     \BNVS_head_N:c A
1879     \__bnvs_tl_put_right:ce Q { { \l__bnvs_A_tl } }
1880   }
```

¶ Ranges.

```
1881   \cs_set:Npn \BNVS_s:n ##1 {
1882     \__bnvs_tl_put_right:cn Q { \BNVS_S:w }
1883     \__bnvs_resolve_subqueries:nc { ##1 } A
1884     \__bnvs_resolve_check:c A
1885     \__bnvs_resolve_query:c A
1886     \__bnvs_tl_put_right:ce Q { { \l__bnvs_A_tl } }
1887   }
```

¶ ++.

```
1888   \cs_set:Npn \BNVS_Ua:w ##1 ##2 {
1889     \__bnvs_tl_put_right:cn Q { \BNVS_Ua:w { ##1 } }
1890     \__bnvs_tl_put_right:ce Q { { \int_eval:n { \tl_count:n { ##2 } / 2 } } }
1891   }
```

¶ References.

```

1892 \cs_set:Npn \BNVS_bIKDPTNUa:w ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 {
1893   \__bnvs_tl_put_right:cn Q { \BNVS_bIPTNUa:w }

```

We manage relative path, missing id and normalize the N part. The I and P variables are persistent. The first argument is the number of ‘++’ pairs in the b argument.

```

1894   \tl_if_empty:nTF { ##1 } {
1895     \__bnvs_tl_put_right:cn Q { { } }
1896   } {
1897     \__bnvs_tl_put_right:ce Q {
1898       { \int_eval:n { \tl_count:n { ##1 } / 2 } }
1899     }
1900   }

```

The I and P arguments.

```

1901   \tl_if_empty:nTF { ##4 } {

```

Not a relative path (the D argument is empty):

```

1902     \__bnvs_tl_set:cn P { ##5 }
1903     \tl_if_empty:nF { ##3 } {
1904       \__bnvs_tl_set:cn I { ##2 }
1905     }
1906     \__bnvs_tl_put_right:ce Q {
1907       { \l__bnvs_I_tl }
1908       { \l__bnvs_P_tl }
1909     }
1910   } {
1911     \__bnvs_tl_put_right:ce Q {
1912       { \l__bnvs_I_tl }
1913       { \l__bnvs_P_tl.##5 }
1914     }
1915   }

```

Copy the T argument as is

```

1916   \__bnvs_tl_put_right:cn Q { { ##6 } }

```

Manage the N argument

```

1917   \tl_if_blank:nTF { ##7 } {
1918     \__bnvs_tl_put_right:cn Q { { } }
1919   } {
1920     \__bnvs_tl_set:ce A { \int_eval:n { ##7 - 1 } }
1921     \__bnvs_tl_if_eq:cnTF A { 0 } {
1922       \__bnvs_tl_put_right:cn Q { { } }
1923     } {
1924       \__bnvs_tl_put_right:ce Q { { \l__bnvs_A_tl } }
1925     }
1926   }

```

Copy the U argument as is

```

1927   \__bnvs_tl_put_right:cn Q { { ##8 } }

```

The a argument.

```

1928     \tl_if_empty:nTF { ##9 } {
1929         \__bnvs_tl_put_right:cn Q { { } }
1930     } {
1931         \__bnvs_tl_put_right:ce Q {
1932             { \int_eval:n { \tl_count:n { ##9 } / 2 } }
1933         }
1934     }
1935 }

! Assignments:

1936 \cs_set:Npn \BNVS_assign:n ##1 {
1937     \__bnvs_tl_put_right:cn Q { \BNVS_assign:n { ##1 } }
1938 }

! Colons:

1939 \cs_set:Npn \::BNVS: {
1940     \__bnvs_tl_put_right:cn Q { \::BNVS: }
1941 }
1942 \cs_set:Npn \:BNVS: {
1943     \__bnvs_tl_put_right:cn Q { \:BNVS: }
1944 }
1945 \__bnvs_tl_clear:c Q
1946 #1
1947 \BNVS_tl_set:ncvcvcv { \BNVS_end: } { #2 } Q I I P P
1948 }

1949 \BNVS_new:cpn { resolve_subqueries:c } #1 {
1950     \BNVS_tl_use:Nv \__bnvs_resolve_subqueries:nc { #1 } { #1 }
1951 }

```

`\__bnvs_resolve_check:c` `\__bnvs_resolve_check:c {<query var>}`

Check for syntax in place. Called just after `\__bnvs_resolve_subqueries:c`.

On input `<query var>` consists of

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_Ua:w{.<attributes>}{<number of ++>}`,
- `\BNVS_N:w{<...>}` for a single static resolved integer,
- `\BNVS_S:w{<...>}` containing output similar to that of `\__bnvs_resolve_query:nc`,
- `\BNVS_assign:n{</+/?>}`, for =, += or ?=,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

and on output, it consists of

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_N:w{<...>}` for a single static resolved integer,

- `\BNVS_S:w{⟨...⟩}` for a comma separated list of static resolved $\langle A \rangle$, $\langle A \rangle:$ and $\langle A \rangle:\langle Z \rangle$ ranges, $\langle A \rangle$ and $\langle Z \rangle$ being unsigned integer decimal denotations,
- `\BNVS_assign:n{⟨/+/?⟩}`, for =, += or ?=,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

```

1952 \BNVS_new:cpn { resolve_check:c } #1 {
1953   \BNVS_begin:
❧ Rule: only peeked \BNVS_Ua:w are allowed,
1954   \cs_set:Npn \BNVS_Ua:w ##1 ##2 {
1955     \BNVS_error:n {
1956       Unexpected~`.attribute'~or~`+',~you~should~press~`x'~now.
1957     }
1958   }

❧ Rule: only peeked \BNVS_assign:n are allowed,
1959   \cs_set:Npn \BNVS_assign:n ##1 {
1960     \BNVS_error:n {
1961       Unexpected~`##1=',~you~should~press~`x'~now.
1962     }
1963   }

❧ Rule: only peeked \BNVS_N:w are allowed,
1964   \cs_set:Npn \BNVS_N:w ##1 {
1965     \BNVS_error:n {
1966       Unexpected~`[...]',~you~should~press~`x'~now.
1967     }
1968   }

❧ Rule: \BNVS_bIPTNUa:w peeks following \BNVS_N:w and \BNVS_Ua:w in that order.
❧ Rule: \BNVS_assign:n must only follow \BNVS_bITN:w and what it has eventually
    peeked, counted from the start,
1969   \cs_set:Npn \BNVS_bIPTNUa:w ##1 ##2 ##3 ##4 ##5 ##6 ##7 {
1970     \__bnvs_tl_put_right:cn Q {
1971       \BNVS_bIPTNUa:w { ##1 } { ##2 } { ##3 } { ##4 }
1972     }
1973     \__bnvs_tl_set:cn N { ##5 }
1974     \__bnvs_tl_set:cn U { ##6 }
1975     \__bnvs_tl_set:cn a { ##7 }
1976   }

❧ Rule: only peeked \BNVS_N:w are allowed,
1976   \BNVS_peek_after_bIPTNUa:ccw N U a
1977 }
1978 \cs_set:Npn \BNVS_peek_after_bIPTNUa:ccw ##1 ##2 ##3 {
1979   \peek_meaning_remove:NTF \BNVS_N:w {
1980     \cs_set:Npn \BNVS:n #####1 {
1981       \__bnvs_tl_put_right:cn ##1 { + ( 0#####1 ) - 1 }
1982       \BNVS_peek_after_bIPTNUa:ccw ##1 ##2 ##3
1983     }
1984     \BNVS:n
1985   } {
1986     \peek_meaning_remove:NTF \BNVS_Ua:w {

```



```

1987     \cs_set:Npn \BNVS:nn #####1 #####2 {
1988         \__bnvs_tl_put_right:cn ##2 { #####1 }
1989         \__bnvs_tl_put_right:cn ##3 { + ( 0#####2 ) }
1990         \BNVS_peek_after_bIPTNUa:cccw ##1 ##2 ##3
1991     }
1992     \BNVS:nn
1993 } {
1994     \BNVS_tl_round:c N
1995     \__bnvs_tl_put_right:ce Q { { \l__bnvs_N_tl } }
1996     \__bnvs_tl_put_right:ce Q { { \l__bnvs_U_tl } }
1997     \BNVS_tl_round:c a
1998     \__bnvs_tl_put_right:ce Q { { \l__bnvs_a_tl } }
1999     \peek_meaning_remove:NT \BNVS_assign:n {
2000         \__bnvs_tl_put_right:cn Q { \BNVS_assign:n }
2001         \cs_set:Npn \BNVS:n #####1 {
2002             \__bnvs_tl_put_right:cn Q { { #####1 } }
2003         }
2004         \BNVS:n
2005     }
2006 }
2007 }
2008 }

```

🔗 Rule: raw copy with check.

```

2009     \cs_set:Npn \BNVS_S:w ##1 {
2010         \__bnvs_tl_put_right:cn Q { \BNVS_S:w { ##1 } }
2011         \peek_meaning:NTF \BNVS_N:w {
2012             \BNVS_error:n {
2013                 Syntax-error,~you~should~press~`x'~now.
2014             }
2015         } {
2016             \peek_meaning:NTF \BNVS_S:w {
2017                 \BNVS_error:n {
2018                     Syntax-error,~you~should~press~`x'~now.
2019                 }
2020             } {
2021                 \peek_meaning:NT \BNVS_bIPTNUa:w {
2022                     \BNVS_error:n {
2023                         Syntax-error,~you~should~press~`x'~now.
2024                     }
2025                 }
2026             }
2027         }
2028     }

```

🔗 Rule: raw copy of raw text.

```

2029     \cs_set:Npn \BNVS_raw:n ##1 {
2030         \__bnvs_tl_put_right:cn Q { \BNVS_raw:n { ##1 } }
2031     }

```

🔗 Rule: only one `\::BNVS:` or `\:BNVS:` allowed between assignments, checked later. Raw copy for the moment.

```

2032     \cs_set:Npn \::BNVS: {
2033         \__bnvs_tl_put_right:cn Q { \::BNVS: }
2034     }

```

```

2035 \cs_set:Npn \:BNVS: {
2036   \__bnvs_tl_put_right:cn Q { \:BNVS: }
2037 }

2038 \BNVS_tl_use:nv { \__bnvs_tl_clear:c Q \use:n } { #1 }
2039 \BNVS_end_tl_set:cv { #1 } Q
2040 }

```

__bnvs_resolve:nnnc __bnvs_resolve:nnnc {<id>} {<path>} {<key>} {<var>}

Resolve the <id>!*<path>/<key>* reference and set <var> to the result. Use the default <id> mechanism. We start with two utilities used to ensure that definitions are not circular because __bnvs_resolve:nnnc calls __bnvs_resolve_query:nc which may in turn call __bnvs_resolve:nnnc. See __bnvs_will_gset:....

```

2041 \BNVS_new:cpn { no_circular:nnnn } #1 #2 #3 #4 {
2042   \__bnvs_gset:nnnn { #1 } { #2 } { ?#3 } {}
2043   #4
2044   \__bnvs_gunset:nnn { #1 } { #2 } { ?#3 }
2045 }

2046 \BNVS_new:cpn { is_circular:nnnTF } #1 #2 #3 #4 {
2047   \__bnvs_is_gset:nnnTF { #1 } { #2 } { ?#3 } {
2048     \__bnvs_is_gset:nnnF { #1 } { #2 } { !#3 } {
2049       \__bnvs_gset:nnnn { #1 } { #2 } { !#3 } { }
2050       \BNVS_warning:n { Circular~definition:~#1!#2/#3 (Once-per-frame) }
2051     }
2052     #4
2053   }
2054 }

2055 \BNVS_tl_new:c { resolve_nnnc }
2056 \BNVS_new:cpn { resolve:nnnc } #1 #2 #3 #4 {
2057   \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { resolve_nnnc } {
2058     \__bnvs_is_circular:nnnTF { #1 } { #2 } { #3 } {
2059       \__bnvs_tl_set:cn { resolve_nnnc } { 1 }
2060       \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { resolve_nnnc }
2061     } {
2062       \__bnvs_if_get:nnncTF { #1 } { #2 } { =#3 } { resolve_nnnc } {

```

❗ The <id>!*<path>/<key>* combination has been initialized.

```

2063   \__bnvs_no_circular:nnnn { #1 } { #2 } { #3 } {
2064     \__bnvs_resolve_query:c { resolve_nnnc }
2065   }
2066 } {

```

❗ The <id>!*<path>/<key>* combination has never been used. Use the default <id> mechanism.

```

2067   \__bnvs_tl_set:cn { resolve_nnnc } { 1 }
2068   \tl_if_empty:nF { #1 } {
2069     \__bnvs_if_get:nnncF { } { #2 } { #3 } { resolve_nnnc } {
2070       \__bnvs_if_get:nnncT { } { #2 } { =#3 } { resolve_nnnc } {

```

```

2071         \_bnvs_no_circular:nnnn { #1 } { #2 } { #3 } {
2072         \_bnvs_no_circular:nnnn {   } { #2 } { #3 } {
2073         \_bnvs_resolve_query:c { resolve_nnncc }
2074         }
2075     }
2076     \_bnvs_gset:nnnv { } { #2 } { #3 } { resolve_nnncc }
2077 }
2078 }
2079 }
2080 }
2081 }
2082 \_bnvs_gset:nnnv { #1 } { #2 } { #3 } { resolve_nnncc }
2083 }
2084 \_bnvs_tl_set:cv { #4 } { resolve_nnncc }
2085 }

```

```

\_bnvs_if_resolve:nnnccTF \_bnvs_if_resolve:nnnccTF {<id>} {<path>} {<key>} {<V var>} {<S var>} {<yes:>}
{<no:>}

```

Resolve the $\langle id \rangle!$ $\langle path \rangle/\langle key \rangle$ reference and put the counter value into $\langle V \text{ var} \rangle$ and the set part into $\langle S \text{ var} \rangle$.

```

2086 \BNVS_tl_new:c { resolve_nnncc }
2087 \BNVS_new_conditional:cpnn { if_resolve:nnncc } #1 #2 #3 #4 #5 { TF } {
2088   \_bnvs_if_get:nnnccTF { #1 } { #2 } { #3 } { resolve_nnncc } {

```

Resolution has already occurred successfully once, most expected situation.

```

2089   \cs_set:Npn \BNVS_VS:w ##1 ##2 {
2090     \_bnvs_tl_set:cn { #4 } { ##1 }
2091     \_bnvs_tl_set:cn { #5 } { ##2 }
2092   }
2093   \l_bnvs_resolve_nnncc_tl
2094   \prg_return_true:
2095 } {
2096   \_bnvs_if_get:nnnccTF { #1 } { #2 } { =#3 } { resolve_nnncc } {

```

Reentrancy management, catch circular references only once.

```

2097   \_bnvs_gset:nnnn { #1 } { #2 } { #3 } {
2098     \_bnvs_gset:nnnn { #1 } { #2 } { #3 } { \BNVS_VS:w 0 0 }
2099     \BNVS_warning:n { Circular~reference:~#1!#2/#3,~defaults~to~'0'. }
2100     \BNVS_VS:w 0 0
2101   }
2102   \_bnvs_if_resolve:vccTF { resolve_nnncc } { #4 } { #5 } {

```

There is a definiton with no error.

```

2103   \_bnvs_gset:nnne { #1 } { #2 } { #3 } {
2104     \exp_not:N \BNVS_VS:w
2105     { \BNVS_tl_use:c { #4 } }
2106     { \BNVS_tl_use:c { #5 } }
2107   }
2108   \prg_return_true:

```

```

2109     } {
2110         \BNVS_warning:n { Wrong-definition:~#1!#2/#3,~defaults~to~`0'. }
2111         \__bnvs_tl_set:cn { #4 } { 0 }
2112         \__bnvs_tl_set:cn { #5 } { 0 }
2113         \__bnvs_gset:nnnn { #1 } { #2 } { #3 } { \BNVS_VS:w 0 0 }
2114         \prg_return_false:
2115     }
2116 } {
2117     \tl_if_empty:nTF { #1 } {
2118         \BNVS_warning:n { Unknown~reference:~#1!#2/#3,~defaults~to~`0'. }
2119         \__bnvs_tl_set:cn { #4 } { 0 }
2120         \__bnvs_tl_set:cn { #5 } { 0 }
2121         \__bnvs_gset:nnnn { #1 } { #2 } { #3 } { \BNVS_VS:w 0 0 }
2122         \prg_return_false:
2123     } {
2124         \__bnvs_if_get:nnncTF { } { #2 } { #3 } { resolve_nnncc } {

```

Resolution has already occurred successfully once for the empty $\langle id \rangle$.

```

2125         \cs_set:Npn \BNVS_VS:w ##1 ##2 {
2126             \__bnvs_tl_set:cn { #4 } { ##1 }
2127             \__bnvs_tl_set:cn { #5 } { ##2 }
2128         }
2129         \l__bnvs_resolve_nnncc_tl
2130         \prg_return_true:
2131     } {
2132         \__bnvs_if_get:nnncTF { } { #2 } { =#3 } { resolve_nnncc } {

```

Reentrancy management for both $\langle id \rangle$'s.

```

2133         \__bnvs_gset:nnnn { #1 } { #2 } { #3 } {
2134             \__bnvs_gset:nnnn { #1 } { #2 } { #3 } { \BNVS_VS:w 0 0 }
2135             \BNVS_warning:n { Circular~reference:~#1!#2/#3,~defaults~to~`0'. }
2136             \BNVS_VS:w 0 0
2137         }
2138         \__bnvs_gset:nnnn { } { #2 } { #3 } {
2139             \__bnvs_gset:nnnn { } { #2 } { #3 } { \BNVS_VS:w 0 0 }
2140             \BNVS_warning:n { Circular~reference:~!#2/#3,~defaults~to~`0'. }
2141             \BNVS_VS:w 0 0
2142         }
2143         \__bnvs_if_resolve:vccTF { resolve_nnncc } { #4 } { #5 } {
2144             \__bnvs_gset:nnne { } { #2 } { #3 } {
2145                 \exp_not:N \BNVS_VS:w
2146                 { \BNVS_tl_use:c { #4 } }
2147                 { \BNVS_tl_use:c { #5 } }
2148             }

```

We do not set a value for the $\langle id \rangle! \langle path \rangle / \langle key \rangle$ combination. The default $! \langle path \rangle / \langle key \rangle$ combination will be used on each request.

```

2149         \__bnvs_gunset:nnn { #1 } { #2 } { #3 }
2150         \prg_return_true:
2151     } {
2152         \BNVS_warning:n { Wrong-definition:~!#2/#3,~defaults~to~`0'. }
2153         \__bnvs_tl_set:cn { #4 } { 0 }
2154         \__bnvs_tl_set:cn { #5 } { 0 }
2155         \__bnvs_gset:nnnn { } { #2 } { #3 } { \BNVS_VS:w 0 0 }
2156         \__bnvs_gunset:nnn { #1 } { #2 } { #3 }

```

```

2157         \prg_return_false:
2158     }
2159 } {
2160     \BNVS_warning:n { Unknown~reference:~!#2/#3,~defaults~to~`0'. }
2161     \__bnvs_tl_set:cn { #4 } { 0 }
2162     \__bnvs_tl_set:cn { #5 } { 0 }
2163     \__bnvs_gset:nnnn { } { #2 } { #3 } { \BNVS_VS:w 0 0 }
2164     \prg_return_false:
2165 }
2166 }
2167 }
2168 }
2169 }
2170 }
2171 \BNVS_undefine:c { if_resolve:nncc }

```

__bnvs_map_AZ:nw __bnvs_map_AZ:nw {<:nnn>} <i> : <j> : <k> \s_stop

Used to extract ranges in colon denotation. <:nnn> is called with integer arguments <i>, <j> and <k>. <i> is a standalone value, <j> ‘-’ <k> is a range, <k> only may be empty, <j> <k> when it makes sense.

```

2172 \BNVS_new:cpn { map_AZ:nw } #1 #2 : #3 : #4 \s_stop {
2173     \cs_set:Npn \BNVS_map_AZ_nw:w ##1 ##2 ##3 { #1 }
2174     \tl_if_blank:nTF { #4 } {
2175         \BNVS_map_AZ_nw:w { #2 } { } { }
2176     } {
2177         \tl_if_empty:nTF { #3 } {
2178             \BNVS_map_AZ_nw:w { } { #2 } { }
2179         } {
2180             \int_compare:nNnTF { 0#2 } > { #3 } {
2181                 \BNVS_map_AZ_nw:w { } { } { }
2182             } {
2183                 \int_compare:nNnTF { 0#2 } = { #3 } {
2184                     \BNVS_map_AZ_nw:w { #2 } { } { }
2185                 } {
2186                     \BNVS_map_AZ_nw:w { } { #2 } { #3 }
2187                 }
2188             }
2189         }
2190     }
2191 }

```

__bnvs_map_AZ:nww __bnvs_map_AZ:nww {<:nnnn>} <A_1> : <Z_1> \s_stop <A_2> : <Z_2> \s_stop

Used for example to compare ranges in colon denotation. <:nnnn>, as a body function, is called with arguments <A_1>, <Z_1>, <A_2> and <Z_2>.

```

2192 \BNVS_new:cpn { map_AZ:nww } #1 #2 : #3 \s_stop #4 : #5 \s_stop {
2193     \cs_set:Npn \BNVS_map_AZ_nww:w ##1 ##2 ##3 ##4 { #1 }
2194     \BNVS_map_AZ_nww:w { #2 } { #3 } { #4 } { #5 }
2195 }

```

`\__bnvs_sort:c` `\__bnvs_sort:c {<query var>}`

Only called by `\__bnvs_if_resolve:nccTF`. Sort in place comma separated ranges in colon denotation with respect to the first index. This index is expected to be nonnegative and in decimal denotation.

```
2196 \BNVS_new:cpn { sort:c } #1 {
2197   \BNVS_tl_use:Nc \clist_sort:Nn { #1 } {
2198     \__bnvs_map_AZ:nw {
2199       \int_compare:nNnTF { #####1 } > { #####3 }
2200       \sort_return_swapped:
2201       \sort_return_same:
2202     } ##1 : \s_stop ##2 : \s_stop
2203   }
2204 }
```

`\__bnvs_merge:c` `\__bnvs_merge:c {<query var>}`

Only called by `\__bnvs_if_resolve:nccTF`. Merge consecutive ranges in place.

```
2205 \BNVS_new:cpn { merge:c } #1 {
2206   \tl_map_inline:nn { VAZ } {
2207     \__bnvs_tl_clear:c ##1
2208   }
2209   \cs_set:Npn \BNVS_merge_c:n ##1 {
2210     \__bnvs_map_AZ:nw {
2211       \__bnvs_tl_set:cn V { #####1 }
2212       \__bnvs_tl_set:cn A { #####2 }
2213       \__bnvs_tl_set:cn Z { #####3 }
2214       \tl_if_empty:nF { #####2 } {
2215         \tl_if_empty:nTF { #####3 } {
2216           \clist_map_break:
2217         } {
2218           \int_compare:nNnTF { #####3 } < { #####2 } {
2219             \use_none_delimit_by_q_stop:w
2220           } {
2221             \int_compare:nNnTF { #####3 } = { #####2 } {
2222               \__bnvs_tl_set:cn V { #####2 }
2223               \__bnvs_tl_clear:c A
2224               \__bnvs_tl_clear:c Z
2225             }
2226           }
2227         }
2228       } ##1 : : \s_stop
2229       \cs_set:Npn \BNVS_merge_c:n #####1 {
2230         \__bnvs_map_AZ:nw {
2231           \tl_if_blank:nTF { #####1 } {
2232             \tl_if_blank:nF { #####2 } {
2233               \tl_if_blank:nTF { #####3 } {
2234                 \__bnvs_tl_if_blank:vTF V {
2235                   \BNVS_tl_use:Nv \int_compare:nNnTF Z < { #####2 - 1 } {
2236                     \__bnvs_tl_put_right:ce { #1 } {
2237                       , \BNVS_tl_use:c A - \BNVS_tl_use:c Z
2238                       , #####2 -
2239                     }
2240                   }
2241                 }
2242               }
2243             }
2244           }
2245         }
2246       }
2247     }
2248   }
2249 }
```

```

2240     }
2241   } {
2242     \__bnvs_tl_put_right:ce { #1 } {
2243       , \BNVS_tl_use:c A -
2244     }
2245   }
2246   \__bnvs_tl_clear:c A
2247   \__bnvs_tl_clear:c Z
2248   \clist_map_break:
2249 } {
2250   \BNVS_tl_use:Nv \int_compare:nNnTF V < { #####2 - 1 } {
2251     \__bnvs_tl_put_right:ce { #1 } {
2252       , \BNVS_tl_use:c V
2253       , #####2 -
2254     }
2255   } {
2256     \__bnvs_tl_put_right:ce { #1 } {
2257       , \BNVS_tl_use:c V -
2258     }
2259   }
2260   \__bnvs_tl_clear:c V
2261   \clist_map_break:
2262 }
2263 } {
2264   \__bnvs_tl_if_blank:vTF V {
2265     \BNVS_tl_use:Nv \int_compare:nNnTF Z < { #####2 - 1 } {
2266       \__bnvs_tl_put_right:ce { #1 } {
2267         , \BNVS_tl_use:c A - \BNVS_tl_use:c Z
2268       }
2269       \__bnvs_tl_set:cn A { #####2 }
2270       \__bnvs_tl_set:cn Z { #####3 }
2271     } {
2272       \BNVS_tl_use:Nv \int_compare:nNnT Z < { #####3 } {
2273         \__bnvs_tl_set:cn Z { #####3 }
2274       }
2275     }
2276   } {
2277     \BNVS_tl_use:Nv \int_compare:nNnTF V < { #####2 - 1 } {
2278       \__bnvs_tl_put_right:ce { #1 } { , \BNVS_tl_use:c V }
2279       \__bnvs_tl_clear:c V
2280       \__bnvs_tl_set:cn A { #####2 }
2281       \__bnvs_tl_set:cn Z { #####3 }
2282       \tl_if_empty:nF { #####2 } {
2283         \tl_if_empty:nT { #####3 } {
2284           \clist_map_break:
2285         }
2286       }
2287     } {
2288       \__bnvs_tl_set:cv A V
2289       \__bnvs_tl_set:cn Z { #####3 }
2290       \__bnvs_tl_clear:c V
2291     }
2292   }
2293 }

```

```

2294     }
2295 } {
2296     \__bnvs_tl_if_blank:vTF V {
2297         \BNVS_tl_use:Nv \int_compare:nNnTF Z < { #####1 - 1 } {
2298             \__bnvs_tl_put_right:ce { #1 } {
2299                 , \BNVS_tl_use:c A - \BNVS_tl_use:c Z
2300             }
2301             \__bnvs_tl_set:cn V { #####1 }
2302             \__bnvs_tl_clear:c A
2303             \__bnvs_tl_clear:c Z
2304         } {
2305             \BNVS_tl_use:Nv \int_compare:nNnT Z = { #####1 - 1 } {
2306                 \__bnvs_tl_set:cn Z { #####1 }
2307             }
2308         }
2309     } {
2310         \BNVS_tl_use:Nv \int_compare:nNnTF V = { #####1 - 1 } {
2311             \__bnvs_tl_set:cv A V
2312             \__bnvs_tl_set:cn Z { #####1 }
2313             \__bnvs_tl_clear:c V
2314         } {
2315             \__bnvs_tl_put_right:ce { #1 } {
2316                 , \BNVS_tl_use:c V
2317             }
2318             \__bnvs_tl_set:cn V { #####1 }
2319         }
2320     }
2321 }
2322 } #####1 : : \s_stop
2323 }
2324 \use_none_delimit_by_q_stop:w
2325 \q_stop
2326 }
2327 \BNVS_tl_use:nv {
2328     \__bnvs_tl_clear:c { #1 }
2329     \clist_map_function:nN
2330 } { #1 } \BNVS_merge_c:n
2331 \__bnvs_tl_if_empty:cTF V {
2332     \__bnvs_tl_if_empty:cF A {
2333         \__bnvs_tl_if_empty:cTF Z {
2334             \__bnvs_tl_put_right:ce { #1 } {
2335                 , \BNVS_tl_use:c A -
2336             }
2337         } {
2338             \BNVS_tl_use:nv { \BNVS_tl_use:Nv \int_compare:nNnTF A < } Z {
2339                 \__bnvs_tl_put_right:ce { #1 } {
2340                     , \BNVS_tl_use:c A - \BNVS_tl_use:c Z
2341                 }
2342             } {
2343                 \BNVS_tl_use:nv { \BNVS_tl_use:Nv \int_compare:nNnTF A = } Z {
2344                     \__bnvs_tl_put_right:ce { #1 } { , \BNVS_tl_use:c A }
2345                 }
2346             }
2347         }

```



```

2348     }
2349   } {
2350     \__bnvs_tl_put_right:ce { #1 } { , \BNVS_tl_use:c V }
2351   }
2352   \BNVS_tl_use:Nc \tl_replace_once:Nnn { #1 } { , } { }
2353 }

```

Utility.

```

2354 \BNVS_new:cpn { get_start:cw } #1 #2 - #3 \s_stop {
2355   \__bnvs_tl_set:cn { #1 } { #2 }
2356 }

```

__bnvs_if_resolve:nccTF __bnvs_if_resolve:nccTF {<query>} {<V var>} {<S var>} {<yes:>} {<no:>}

__bnvs_if_resolve:vccTF Called by __bnvs_if_resolve:nnnccTF. Put the result into the <S var>. Also put the value into the <V var>. Relies on __bnvs_resolve:nc.

```

2357 \BNVS_tl_new:c { if_resolve:ncc }
2358 \BNVS_new_conditional:cpnn { if_resolve:ncc } #1 #2 #3 { T, F, TF } {
2359   \__bnvs_resolve:nc { #1 } { #3 }
2360   \__bnvs_sort:c { #3 }
2361   \__bnvs_merge:c { #3 }
2362   \BNVS_tl_use:Nv \clist_map_inline:nn { #2 } {
2363     \__bnvs_get_start:cw { #2 } ##1 - \s_stop
2364     \clist_map_break:
2365   }
2366   \prg_return_true:
2367 }
2368 \BNVS_undefine:c { if_resolve:ncc }

2369 \BNVS_new_conditional:cpnn { if_resolve:vcc } #1 #2 #3 { T, F, TF } {
2370   \BNVS_tl_use:Nv \__bnvs_if_resolve:nccTF { #1 } { #2 } { #3 }
2371   \prg_return_true:
2372   \prg_return_false:
2373 }
2374 \BNVS_undefine:c { if_resolve:vcc }

```

__bnvs_AZ:nnnn __bnvs_AZ:nnn {<A:w>} {<A::w>} {<AZ:w>} {<range>}

Split the <range> in colon denotation into pieces and call the proper code accordingly. <A:w> for a single value <A>, <A::w> for an infinite range <A>:, <AZ:w> for a finite range <A>:<Z>.

```

2375 \cs_set:Npn \BNVS_AZ:nnnw #1 #2 #3 #4 : #5 : #6 \s_stop {
2376   \tl_if_empty:nTF { #6 } {
2377     \cs_set:Npn \BNVS_AZ:n ##1 {
2378       #1
2379     }
2380     \BNVS_AZ:n { #4 }
2381   } {
2382     \tl_if_empty:nTF { #5 } {
2383       \cs_set:Npn \BNVS_AZ:n ##1 {

```

```

2384         #2
2385     }
2386     \BNVS_AZ:n { #4 }
2387 } {
2388     \cs_set:Npn \BNVS_AZ:nn ##1 ##2 {
2389         #3
2390     }
2391     \BNVS_AZ:nn { #4 } { #5 }
2392 }
2393 }
2394 }
2395 \BNVS_new:cpn { AZ:nnnn } #1 #2 #3 #4 {
2396     \BNVS_AZ:nnnw { #2 } { #3 } { #4 } #1 :: \s_stop
2397 }

```

_bnvs_shift:cn _bnvs_shift:cn {<set var> } {<shift> }

Shift in place any component of the content of <set var> by <shift> amount. Called by
_bnvs_resolve_bIPTNUa:w.

```

2398 \BNVS_tl_new:c { shift_cn }
2399 \BNVS_tl_new:c { shift_cn_A }
2400 \BNVS_new:cpn { shift_cn } #1 #2 {
2401     \_bnvs_tl_clear:c { shift_cn }
2402     \BNVS_tl_use:Nv \clist_map_inline:nn { #1 } {
2403         \_bnvs_AZ:nnnn { ##1 } {
2404             \_bnvs_tl_set:cn { shift_cn_A } { #####1 + #2 }
2405             \BNVS_tl_round:c { shift_cn_A }
2406             \_bnvs_tl_put_right:cn { shift_cn } { , }
2407             \_bnvs_tl_put_right:cv { shift_cn } { shift_cn_A }
2408         } {
2409             \_bnvs_tl_set:cn { shift_cn_A } { #####1 + #2 }
2410             \BNVS_tl_round:c { shift_cn_A }
2411             \_bnvs_tl_put_right:cn { shift_cn } { , }
2412             \_bnvs_tl_put_right:cv { shift_cn } { shift_cn_A }
2413             \_bnvs_tl_put_right:cn { shift_cn } { : }
2414         } {
2415             \_bnvs_tl_set:cn { shift_cn_A } { #####1 + #2 }
2416             \BNVS_tl_round:c { shift_cn_A }
2417             \_bnvs_tl_put_right:cn { shift_cn } { , }
2418             \_bnvs_tl_put_right:cv { shift_cn } { shift_cn_A }
2419             \_bnvs_tl_put_right:cn { shift_cn } { : }
2420             \_bnvs_tl_set:cn { shift_cn_A } { #####2 + #2 }
2421             \BNVS_tl_round:c { shift_cn_A }
2422             \_bnvs_tl_put_right:cv { shift_cn } { shift_cn_A }
2423         }
2424     }
2425     \tl_replace_once:Nnn \l__bnvs_shift_cn_tl { , } {}
2426     \_bnvs_tl_set:cv { #1 } { shift_cn }
2427 }

```

_bnvs_first:nnnn _bnvs_first:nnnn {<set> } {<:n> } {<:n:> } {<:nn> }

Take the first range of <set> and apply one of the codes.

```

2428 \tl_new:N \l__bnvs_first_nnnn_tl
2429 \BNVS_new:cpn { first:nnnn } #1 {
2430   \__bnvs_tl_clear:c { first_nnnn }
2431   \clist_map_inline:nn { #1 } {
2432     \tl_if_empty:nF { ##1 } {
2433       \clist_map_break:n {
2434         \__bnvs_tl_set:cn { first_nnnn } { ##1 }
2435       }
2436     }
2437   }
2438   \BNVS_tl_use:Nv \__bnvs_AZ:nnnn { first_nnnn }
2439 }

```

__bnvs_first:c __bnvs_first:c {<set var>}

Replace the content of the <set var> by its first value.

```

2440 \BNVS_new:cpn { first:c } #1 {
2441   \BNVS_tl_use:Nv \__bnvs_first:nnnn { #1 } {
2442     \__bnvs_tl_set:cn { #1 } { ##1 }
2443   } {
2444     \__bnvs_tl_set:cn { #1 } { ##1 }
2445   } {
2446     \__bnvs_tl_set:cn { #1 } { ##1 }
2447   }
2448 }

```

__bnvs_last:nnnn __bnvs_last:nnnn {<set>} {<:n>} {<:n:>} {<:nn>}

Take the last range of <set> and apply one of the codes.

```

2449 \tl_new:N \l__bnvs_last_nnnn_tl
2450 \BNVS_new:cpn { last:nnnn } #1 {
2451   \__bnvs_tl_clear:c { last_nnnn }
2452   \clist_map_inline:nn { #1 } {
2453     \tl_if_empty:nF { ##1 } {
2454       \__bnvs_tl_set:cn { last_nnnn } { ##1 }
2455     }
2456   }
2457   \BNVS_tl_use:Nv \__bnvs_AZ:nnnn { last_nnnn }
2458 }

```

__bnvs_resolve_first:nnc __bnvs_resolve_first:nnc {<I>} {<P>} {<var>}

Only called by __bnvs_resolve_bIPTNUa:w.

```

2459 \BNVS_new_conditional:cpnn { if_resolve_first:nnc } #1 #2 #3 { T, F, TF } {
2460   \__bnvs_if_resolve:nncTF { #1 } { #2.first } { #3 } {
2461     \prg_return_true:
2462   } {
2463     \__bnvs_if_resolved:nnncTF { #1 } { #2 } A { #3 } {
2464       \prg_return_true:
2465     } {
2466       \__bnvs_if_resolve_R:nncTF { #1 } { #2 } { #3 } {
2467         \__bnvs_require:nnnc { #1 } { #2 } A { #3 }

```

```

2468     \prg_return_true:
2469   } {
2470     \prg_return_false:
2471   }
2472 }
2473 }
2474 }

```

`\__bnvs_resolve_bIPTNUa:w` `\__bnvs_resolve_bIPTNUa:w {<b>}< /b> {<I>}< /I> {<P>}< /P> {<T>}< /T> {<N>}< /N> {<U>}< /U> {<a>}< /a>}`

Only called by `\__bnvs_resolve_rhs:nc`. Some rules:

•

```

2475 \tl_new:N \l__bnvs_resolve_bIPTNUa_tl
2476 \BNVS_new:cpn { resolve_bIPTNUa:w } #1 #2 #3 #4 #5 #6 #7 {
2477
2478   \__bnvs_if_resolve:nnccTF { #2 } { #3 } {} V S {
2479     \BNVS_begin:
2480
2481
2482
2483     \bool_if:nTF {
2484       \tl_if_empty_p:n { #1 } &&
2485       \tl_if_empty_p:n { #5 } &&
2486       \tl_if_empty_p:n { #7 }
2487     } {
2488       \__bnvs_tl_set:ce Q {
2489         \exp_not:N \BNVS_S:w { \l__bnvs_S_tl }
2490       }
2491     } {
2492       \__bnvs_tl_clear:c Q
2493       \__bnvs_tl_set:cn { resolve_bIPTNUa } { #4 }
2494       \tl_if_empty:nT { #5 } {
2495         \__bnvs_tl_put_right:cn { resolve_bIPTNUa } { #6 }
2496       }
2497       \BNVS_tl_use:Nc \tl_replace_once:Nnn { resolve_bIPTNUa } { . } { }
2498       \BNVS_tl_use:Nc \tl_replace_all:Nnn { resolve_bIPTNUa } { . } { , }
2499       \clist_map_inline:Nn \l__bnvs_resolve_bIPTNUa_tl {
2500         \str_case:nn { ##1 } {
2501           { reset } {
2502             \__bnvs_gunset:nnn { #2 } { #3 } {}
2503             \__bnvs_if_resolve:nnccF { #2 } { #3 } {} V S {
2504               \__bnvs_tl_set:cn V 0
2505               \__bnvs_tl_set:cn S 0
2506             }
2507           }
2508           { first } {
2509             \__bnvs_if_resolve:nnccF { #2 } { #3 } F V S {
2510               \__bnvs_tl_set:cv S V
2511             }
2512           }
2513           { previous } {
2514             \__bnvs_tl_put_right:cn V { - 1 }

```

```

2515         \BNVS_round:c V
2516         \__bnvs_tl_set:cv S V
2517     }
2518     { last } {
2519         \BNVS_tl_use:Nv
2520         \__bnvs_last:nnn S {
2521             \__bnvs_tl_set:cn V { ####1 }
2522             \__bnvs_tl_set:cn S { ####1 }
2523         } {
2524             \__bnvs_tl_set:cn V { 0 }
2525             \__bnvs_tl_set:cn S { 0: }
2526         }
2527     }
2528     { next } {
2529         \BNVS_tl_use:Nv
2530         \__bnvs_last:nnn S {
2531             \__bnvs_tl_set:cn V { ####1 }
2532             \__bnvs_tl_put_right:cn V { + 1 }
2533             \BNVS_round:c V
2534             \__bnvs_tl_set:cn S { ####1 }
2535         } {
2536             \__bnvs_tl_set:cn V { 0 }
2537             \__bnvs_tl_set:cn S { 0: }
2538         }
2539     }
2540 }
2541 }
2542 \tl_if_empty:nF { #1#5 } {
2543     \__bnvs_shift:cn V { #1+#5 }
2544     \__bnvs_shift:cn S { #1+#5 }
2545     \__bnvs_tl_set:cn { resolve_bIPTNUa } { #6 }
2546 }
2547 }
2548 }
2549 \BNVS_end_tl_put_right:cv Q Q
2550 } {
2551     \__bnvs_tl_put_right:cn Q { \BNVS_raw:n { 0 } }
2552 }
2553 }

```

```

\__bnvs_resolve_rhs:c \__bnvs_resolve_rhs:c {<query var>}
\__bnvs_resolve_rhs:nc \__bnvs_resolve_rhs:nc {<query>} {<var>}

```

Only called by `\__bnvs_resolve_assign:nc`. Resolve in place the `<query var>`. On input, the `<query>` or the contents of the `<query var>` **consists of**

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_N:w{<...>}` for a single static resolved integer,
- `\BNVS_S:w{<...>}` for a comma separated list of static resolved `<A>`, `<A>`: and `<A>:<Z>` ranges, `<A>` being positive and `<Z>` being nonnegative,
- `\:BNVS:` and `\:BNVS:`, range delimiters.

and on return, **it contains only one of next object between each range delimiters**

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_N:w{<...>}` for a single static resolved integer,
- `\BNVS_S:w{<...>}` for a comma separated list of static resolved `<A>`, `<A>`: and `<A>:<Z>` ranges, `<A>` being positive and `<Z>` being nonnegative,
- `\:BNVS:` and `\:BNVS:`, range delimiters.

Similarly, the second function resolves `<query>` into `<var>`. Each one is run in its own group.

```

2554 \BNVS_new:cpn { resolve_rhs:c } #1 {
2555   \BNVS_begin:
2556   \cs_set:Npn \BNVS_raw:n ##1 {
2557     \__bnvs_tl_put_right:cn Q { ##1 }
2558   }

2559   \cs_set:Npn \BNVS_bIPTNUa:w { \__bnvs_bIPTNUa:w }
2560   % \BNVS_tl_use:nv { \__bnvs_tl_clear:c Q } #1
2561   \__bnvs_tl_set:cn Q { 123456 }
2562   \typeout{*****}
2563   \BNVS_end_tl_set:cv { #1 } Q
2564 }

2565 \BNVS_new:cpn { resolve_rhs:nc } #1 #2 {
2566   \__bnvs_tl_set:cn { #2 } { #1 }
2567   \__bnvs_resolve_rhs:c { #2 }
2568 }

```

```

\BNVS_head_V:c \BNVS_head_V:c {<resolved var>}
\BNVS_head_V:nc \BNVS_head_V:nc {<resolved>} {<var>}
\BNVS_head_N:c \BNVS_head_N:c {<resolved var>}
\BNVS_head_N:nc \BNVS_head_N:nc {<resolved>} {<var>}

```

Restrict in place *<resolved var>* to its leading value or put the leading value of *<resolved>* into *<var>*. In the ‘N’ variant, negative values are mapped to ‘0’. In the “:n.” variant, the variable is first fed with *<resolved>*.

```

2569 \cs_new:Npn \BNVS_head_V:c #1 {
2570   \BNVS_tl_use:nc {
2571     \regex_replace_once:nnNF { \A ( -? \d+ ) .* \Z } { \1 }
2572   } { #1 } {
2573     \__bnvs_tl_set:cn { #1 } 0
2574   }
2575 }

2576 \cs_new:Npn \BNVS_head_V:nc #1 #2 {
2577   \__bnvs_tl_set:cn { #2 } { #1 }
2578   \BNVS_head_V:c { #2 }
2579 }

2580 \cs_new:Npn \BNVS_head_N:c #1 {
2581   \BNVS_tl_use:nc {
2582     \regex_replace_once:nnNF { \A ( \d+ ) .* \Z } { \1 }
2583   } { #1 } {
2584     \__bnvs_tl_set:cn { #1 } 0
2585   }
2586 }

2587 \cs_new:Npn \BNVS_head_N:nc #1 #2 {
2588   \__bnvs_tl_set:cn { #2 } { #1 }
2589   \BNVS_head_N:c { #2 }
2590 }

```

```

\BNVS_round:N \BNVS_round:N <tl variable>
\BNVS_tl_round:c \BNVS_tl_round:c {<tl core name>}

```

```

\BNVS_round_nonnegative:N <tl variable>
\BNVS_tl_round_nonnegative:c {<tl core name>}

```

Replaces the variable content with its rounded floating point evaluation. The nonnegative variants also restrict to the nonnegative part.

```

2591 \cs_new:Npn \BNVS_round:N #1 {
2592   \tl_if_empty:NTF #1 {
2593     \tl_set:Nn #1 { 0 }
2594   } {
2595     \tl_set:Ne #1 { \fp_eval:n { round(#1) } }
2596   }
2597 }

2598 \cs_new:Npn \BNVS_round_nonnegative:N #1 {
2599   \tl_if_empty:NTF #1 {
2600     \tl_set:Nn #1 { 0 }
2601   } {
2602     \tl_set:Ne #1 { \fp_eval:n { round(max(#1,0)) } }
2603   }

```

```

2604 }

2605 \cs_new:Npn \BNVS_tl_round:c {
2606   \BNVS_tl_use:Nc \BNVS_round:N
2607 }

2608 \cs_new:Npn \BNVS_tl_round_nonnegative:c {
2609   \BNVS_tl_use:Nc \BNVS_round_nonnegative:N
2610 }

```

```

\__bnvs_resolve_query:c \__bnvs_resolve_query:nc {<query>} {<var>}
\__bnvs_resolve_query:nc \__bnvs_resolve_query:c {<query var>}

```

Calls

- `\__bnvs_resolve_prepare:nc`,
- `\__bnvs_resolve_subqueries:c`,
- `\__bnvs_resolve_check:c` and
- `\__bnvs_resolve_assign:c`.

May call itself. On input, `<query>` is a single query. On return `<var>` consists of

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_N:w{<...>}` for a single static resolved integer,
- `\BNVS_S:w{<...>}` for a comma separated list of static resolved `<A>`, `<A>`: and `<A>:<Z>` ranges, `<A>` being positive and `<Z>` being nonnegative,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

Both functions do the same except that `\__bnvs_resolve_query:c` acts on `<query var>` in place.

```

2611 \BNVS_new:cpn { resolve_query:nc } #1 #2 {
2612   \BNVS_begin:
2613   \__bnvs_resolve_prepare:nc { #1 } { #2 }
2614   \__bnvs_resolve_subqueries:c { #2 }
2615   \__bnvs_resolve_check:c { #2 }
2616   \__bnvs_resolve_assign:c { #2 }
2617   \BNVS_end_tl_set:cv { #2 } { #2 }
2618 }

2619 \BNVS_new:cpn { resolve_query:c } #1 {
2620   \BNVS_tl_use:Nv \__bnvs_resolve_query:nc { #1 } { #1 }
2621 }

```

```

\__bnvs_resolve_assign:nc \__bnvs_resolve_assign:c {<query var>}
\__bnvs_resolve_assign:c \__bnvs_resolve_assign:nc {<query>} {<var>}

```

Only called by `\__bnvs_resolve_query:nc`. We split the `<query>` at each function `\BNVS_assign:n`, then resolve the last item of the sequence by calling the function `\__bnvs_resolve_rhs:c` and assign the result to the previous one by calling the auxiliary function `\__bnvs_assign_NbITNannc:w` as necessary. Finally the result is put back to `<var>`. On input `<query>` **consists of**

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_N:w{<...>}` for a single static resolved integer,
- `\BNVS_S:w{<...>}` for a comma separated list of static resolved `<A>`, `<A>:` and `<A>:<Z>` ranges, `<A>` and `<Z>` being unsigned integer decimal denotations,
- `\BNVS_assign:n{<|+/?>}`, for `=`, `+=` or `?=`,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

and on output, **it consists of**

- `\BNVS_raw:n{<raw text>}` for everything not special,
- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_N:w{<...>}` for a single static resolved integer,
- `\BNVS_S:w{<...>}` for a comma separated list of static resolved `<A>`, `<A>:` and `<A>:<Z>` ranges, `<A>` being positive and `<Z>` being nonnegative,
- `\::BNVS:` and `\:BNVS:`, range delimiters.

```

2622 \BNVS_new:cpn { resolve_assign:c } #1 {
2623   \BNVS_tl_use:Nv \__bnvs_resolve_assign:nc { #1 } { #1 }
2624 }

2625 \BNVS_new:cpn { resolve_assign:nc } #1 #2 {
2626   \__bnvs_if_call:TF {
2627     \BNVS_begin:
2628     \regex_split:nnNTF
2629     { \c { BNVS_assign:n } \cB . ( .*? ) \cE . } { #1 } \l__bnvs_split_seq {
2630       \__bnvs_seq_pop_right:cc { split } Q
2631       \__bnvs_tl_if_empty:cF Q {

```

The last item must not be empty. Unreachable?

```

2632     \BNVS_error:x {
2633       Unexpected~"\tl_to_string:N \l__bnvs_Q_tl",
2634       you~should~press~"x"~now.
2635     }
2636   }
2637   \__bnvs_seq_pop_right:cc { split } Q
2638   \__bnvs_resolve_rhs:c Q
2639   \__bnvs_seq_pop_right:cc { split } B
2640   \cs_set:Npn \BNVS_loop: {

```

```

2641     \_bnvs_seq_pop_right:cc { split } C
2642     \BNVS_tl_use:nvv {
2643       \exp_last_unbraced:NV
2644       \_bnvs_assign_NbITNannc:w \l__bnvs_C_tl
2645     } BQ Q
2646     \seq_pop_right:NNT \l__bnvs_split_seq \l__bnvs_B_tl {
2647       \BNVS_loop:
2648     }
2649   }
2650   \BNVS_loop:
2651 } {
2652   \_bnvs_tl_set:cn Q { #1 }
2653   \_bnvs_resolve_rhs:c Q
2654 }
2655 \BNVS_end_tl_set:cv { #2 } Q
2656 } {
2657   \BNVS_error:n { TOO~MANY~NESTED~CALLS/Resolution }
2658   \_bnvs_tl_set:cn { #2 } { 1 }
2659 }
2660 }

```

```

\_bnvs_assign_NbITNannc:w:n \_bnvs_assign_NbITNannc:w:n
    \BNVS_bITNa:w {<b>} {<I>} {<T>} {<N>} {<a>}
    {<type>} {<rhs>} {<var>}

```

Find the specification for $\langle I \rangle$, $\langle T \rangle$, $\langle N \rangle$ applies eventually $\langle b \rangle$ and $\langle a \rangle$, makes the assignment according to $\langle type \rangle$. On input, $\langle rhs \rangle$ consists of

- $\backslash\text{BNVS_raw:n}\{\langle raw\ text \rangle\}$ for everything not special,
- $\backslash\text{BNVS_bIPTNUa:w}\{\langle b \rangle\}\{\langle I \rangle\}\{\langle P \rangle\}\{\langle T \rangle\}\{\langle N \rangle\}\{\langle U \rangle\}\{\langle a \rangle\}$, for specifications
- $\backslash\text{BNVS_N:w}\{\langle \dots \rangle\}$ for a single static resolved integer,
- $\backslash\text{BNVS_S:w}\{\langle \dots \rangle\}$ for a comma separated list of static resolved $\langle A \rangle$, $\langle A \rangle:$ and $\langle A \rangle:\langle Z \rangle$ ranges, $\langle A \rangle$ being positive and $\langle Z \rangle$ being nonnegative,
- $\backslash\text{:BNVS:}$ and $\backslash\text{:BNVS:}$, range delimiters.

On output, $\langle var \rangle$ contains the static result.

```

2661 \BNVS_new:cpn { assign_NbITNannc:w } \BNVS_bITNa:w #1 #2 #3 #4 #5 #6 #7 #8 {
2662   \str_case:nnF { #6 } {
2663     { ? } {
2664
2665     }
2666     { + } {
2667
2668     }
2669   } {
2670     \_bnvs_is_gset:nnnT { #2 } { #3 } { #4 } {
2671       \tl_if_empty:nF { #1 } {
2672         \_bnvs_tl_put_right:cn { #8 } { + #1 }
2673         \regex_match:nnTF { [-,] } { #8 } {
2674           \STOP
2675         } {

```

```

2676     \__bnvs_tl_set:cn { #8 } { #7 + #1 }
2677     \BNVS_round:c { #8 }
2678   }
2679   \__bnvs_gset:nnnv { #2 } { #3 } { #4 } { #8 }
2680   \STOP
2681 }
2682 }
2683 }
2684 }

```

\BNVS_to_AZ:n \BNVS_to_AZ:n {<range>}

Split <range> into A and Z variables. Range components are delimited by \:BNVS: or \::BNVS:. The A variable contains a positive integer, which is the start of the range. The Z variable contains contains \q_nil for a one length range, a nonnegative integer, or nothing for an infinite range. Auxiliary functions are \BNVS_to_AZL:n, \BNVS_to_AZL:nnn and \BNVS_to_ZL:nnnn.

```

2685 \cs_new:Npn \BNVS_to_ZL:nnnn #1 #2 #3 #4 {
2686   \tl_if_empty:nF { #3 } {
2687     \BNVS_error:x {
2688       Bad~colon~range~syntax~"\tl_to_str:n { #4 ( #3 ) }".
2689     }
2690   }
2691   \__bnvs_tl_set:cn Z { #1 }
2692   \__bnvs_tl_set:cn L { #2 }
2693 }

2694 \cs_new:Npn \BNVS_to_AZL:nnn #1 #2 #3 {
2695   \__bnvs_tl_set:cn A { #1 }
2696   \__bnvs_tl_set:cn Z { #2 }
2697   \tl_if_empty:nF { #2 } {
2698     \regex_replace_case_all:nN {
2699       { \c { :BNVS: } } { : }
2700       { \c { ::BNVS: } } { :: }
2701     } \l__bnvs_Z_tl
2702     \regex_replace_case_once:nN {
2703       { ( [^:] )+ (?: ::+ ( [^:]*) ( .* ) ) * }
2704       { \cB \{ \1 \cE \} \cB \{ \2 \cE \} \cB \{ \3 \cE \} }
2705       { :+ ( [^:] ) * (?: : ( [^:]*) ( .* ) ) * }
2706       { \cB \{ \2 \cE \} \cB \{ \1 \cE \} \cB \{ \3 \cE \} }
2707     } \l__bnvs_Z_tl
2708     \exp_last_unbraced:NV \BNVS_to_ZL:nnnn \l__bnvs_Z_tl { #3 }
2709   }
2710 }

2711 \cs_new:Npn \BNVS_to_AZL:n #1 {
2712   \__bnvs_tl_clear:c L
2713   \tl_if_empty:nTF { #1 } {
2714     \__bnvs_tl_set:cn A 1
2715     \__bnvs_tl_set:cn Z { \q_nil }
2716   } {
2717     \tl_set:Nn \l__bnvs_A_tl { #1 }
2718     \regex_replace_once:nnNTF { ( [^:] ) * : ( .* ) } {
2719       \cB \{ \1 \cE \} \cB \{ \2 \cE \}

```

```

2720 } \l__bnvs_A_tl {
2721   \exp_last_unbraced:NV \BNVS_to_AZL:nnn \l__bnvs_A_tl { #1 }
2722 } {
2723   \__bnvs_tl_set:ce A { \int_max:nn 0 { #1 } }
2724   \__bnvs_tl_set:cn Z { \q_nil }
2725 }
2726 }
2727 }

2728 \cs_new:Npn \BNVS_to_AZ:n #1 {
2729   \BNVS_to_AZL:n { #1 }
2730   \quark_if_nil:VTF \l__bnvs_Z_tl {
2731     \tl_if_empty:NT \l__bnvs_A_tl {
2732       \__bnvs_tl_set:cn A 1
2733     }
2734   } {
2735     \__bnvs_tl_if_empty:cTF A {
2736       \__bnvs_tl_if_empty:cTF Z {
2737         \__bnvs_tl_set:cn A 1
2738         \__bnvs_tl_if_empty:cF L {
2739           \__bnvs_tl_set:ce Z { \int_max:nn 0 { \l__bnvs_L_tl } }
2740         }
2741       } {
2742         \__bnvs_tl_if_empty:cTF L {
2743           \__bnvs_tl_set:cn A 1
2744         } {
2745           \__bnvs_tl_set:ce A { \int_max:nn 1 {
2746             \l__bnvs_Z_tl - \l__bnvs_L_tl + 1
2747           } }
2748         }
2749         \__bnvs_tl_set:ce Z { \int_max:nn 0 { \l__bnvs_Z_tl } }
2750       }
2751     } {
2752       \__bnvs_tl_if_empty:cTF Z {
2753         \__bnvs_tl_if_empty:cF L {
2754           \__bnvs_tl_set:ce Z { \int_max:nn 0 {
2755             \l__bnvs_A_tl + \l__bnvs_L_tl - 1
2756           } }
2757         }
2758       } {
2759         \__bnvs_tl_set:ce Z { \int_max:nn 0 { \l__bnvs_Z_tl } }
2760       }
2761     }
2762   }
2763 }

```

```

\__bnvs_resolve_colons:c \__bnvs_resolve_colons:c {<var>}
\__bnvs_resolve_colons:nc {<query>} {<var>}

```

Resolve in place the ranges. On return, we only have static numbers and one colon eventually. If there are colons, we use a $\langle first \rangle : \langle last \rangle$ notation where $\langle first \rangle$ and $\langle last \rangle$ are nonnegative integers.

```

2764 \BNVS_new:cpn { resolve_colons:c } #1 {
2765   \BNVS_tl_use:nv { \BNVS_to_AZ:n } { #1 }

```

```

2766 \BNVS_tl_round:c A
2767 \__bnvs_tl_set:cv { #1 } A
2768 \quark_if_nil:NF \l__bnvs_Z_tl {
2769   \__bnvs_tl_put_right:cn { #1 } :
2770   \__bnvs_tl_if_empty:cF Z {
2771     \BNVS_tl_round:c Z
2772     \__bnvs_tl_put_right:cv { #1 } Z
2773   }
2774 }
2775 }

2776 \BNVS_new:cpn { resolve_colons:nc } #1 #2 {
2777   \__bnvs_tl_set:cn { #2 } { #1 }
2778   \__bnvs_resolve_colons:c { #2 }
2779 }

```

__bnvs_resolve_queries:nc **__bnvs_resolve_queries:nc** {<queries>} {<ans>}

One of the top level resolution entry point. May be called recursively. For each individual <query> of the <queries> clist, it replaces in the <queries> all the ?(<...>) query instructions as well as [<...>] index references with their static counterpart then it evaluates the instructions to obtain a single value or a set of references. Ranges are in colon (:) syntax. The function **__bnvs_resolve_query:nc** is called for each individual query in the <queries> clist. The Q seq variable stores the specifications whereas the Q tl variable stores the result. The implementation is not highly efficient in the sense that queries are scanned multiple times, but it does not cost that much as long as queries are reasonably short.

```

2780 \BNVS_new:cpn { resolve_queries:nc } #1 #2 {
2781   \BNVS_begin:
2782   \__bnvs_tl_clear:c Q
2783   \__bnvs_seq_clear:c Q
2784
2785   \clist_map_inline:nn {
2786     \__bnvs_resolve_query:nc { ##1 } { #2 }
2787
2788   }
2789 }
2790
2791 \__bnvs_resolve_prepare:nc { #1 } { #2 }
2792
2793 \cs_set:Npn \BNVS_raw:n ##1 {
2794   \tl_if_blank:nF { ##1 } {
2795     \__bnvs_seq_put_right:cn Q { ##1 }
2796   }
2797 }
2798 \cs_set:Npn \BNVS_set_R:n ##1 {
2799   \__bnvs_tl_if_empty:cT Q {
2800     \__bnvs_tl_set:cn Q { \BNVS_S:w }
2801     \cs_set:Npn \BNVS_S:w ####1 {
2802       \__bnvs_seq_put_right:cn Q { ####1 }
2803     }
2804     \cs_set_eq:NN \BNVS_V:w \BNVS_S:w
2805   }

```

```

2806   \__bnvs_seq_put_right:cn Q { ##1 }
2807 }
2808 \cs_set_eq:NN \BNVS_S:w \BNVS_set_R:n
2809 \cs_set:Npn \BNVS_V:w ##1 {
2810   \__bnvs_seq_put_right:cn Q { ##1 }
2811   \cs_set_eq:NN \BNVS_V:w \BNVS_set_R:n
2812 }
2813 \clist_map_inline:nn { #1 } {
2814   \__bnvs_resolve_query:nc { ##1 } A
2815   \BNVS_tl_use:Nv \use:n A
2816 }
2817 \__bnvs_tl_if_empty:cT Q {
2818   \__bnvs_tl_set:cn Q { \BNVS_V:w }
2819 }
2820 \exp_args:Nne \__bnvs_tl_put_right:cn Q {
2821   { \__bnvs_seq_use:cn Q { , } }
2822 }
2823 \BNVS_end_tl_set:cv { #2 } Q
2824 }

```

__bnvs_resolve:nc **__bnvs_resolve:nc {<queries>} {<ans>}**

Top level resolution function. Calls **__bnvs_resolve_queries:nc** and replace in **<ans>** any colon (:) by a dash (-).

```

2825 \BNVS_new:cpn { resolve:nc } #1 #2 {
2826   \__bnvs_resolve_queries:nc { #1 } { #2 }
2827   \BNVS_tl_use:Nc \tl_replace_all:Nnn { #2 } { : } { - }
2828 }

```

A group is created to use local variables:

\l__bnvs_ans_tl The token list that will be appended to **<tl variable>** on return.

(End of definition for \l__bnvs_ans_tl.)

\l__bnvs_query_tl Storage for the overlay query expression to be evaluated.

(End of definition for \l__bnvs_query_tl.)

Given a name, a frame id and a dotted path, we resolve any intermediate standalone reference. For example, with A=B and B=C, A is resolved in C. But with A=B+1 and B=C, A is not resolved in C+1. With A=B:D and B=C, A is not resolved in C:D neither.

```

2829 \quark_new:N \q__bnvs_X

```

6.15 Evaluation bricks

6.15.1 Helpers

```

2830 \BNVS_new:cpn { seq_merge:cc } #1 #2 {
2831   \__bnvs_seq_if_empty:cF { #2 } {
2832     \__bnvs_seq_set_split:cnx { #1 } { \q__bnvs_X } {
2833       \__bnvs_seq_use:cn { #1 } { \q__bnvs_X }
2834       \exp_not:n { \q__bnvs_X }
2835       \__bnvs_seq_use:cn { #2 } { \q__bnvs_X }

```

```

2836     }
2837     \__bnvs_seq_remove_all:cn { #1 } { }
2838   }
2839 }

```

6.15.2 Resolve from initial values

Lower level resolution functions.

```

\__bnvs_resolve_V_or_R:nncc \__bnvs_resolve_V_or_R:nncc {<id>} {<path>} {<spec>} {<V>} {<R>}

```

Auxiliary function used by __bnvs_resolve:nn. It does not change the <id>!
<path> data model. It resolves <spec> into either <V> or <R> in the <id>!
<path> context.

```

2840 \regex_const:Nn \c__bnvs_A_AZ_Z_regex { \A ([^-]*)[-]([^-]*) \Z }
2841 \tl_new:N \l__bnvs_vs_tl
2842 \tl_new:N \l__bnvs_VS_tl
2843 \BNVS_new:cpn { resolve_V_or_R:nncc } #1 #2 #3 #4 #5 {
2844   \__bnvs_tl_clear:c { VS }
2845   \cs_set:Npn \BNVS:n ##1 {
2846     \__bnvs_tl_clear:c { vs }
2847     \__bnvs_if_append:nnccF { #1 } { #2 } { ##1 } { vs } {
2848       \__bnvs_if_resolve:ncTF { ##1 } { vs } {
2849         \BNVS_tl_use:Nv \clist_map_inline:nn { vs } {
2850           \tl_if_empty:nF { #####1 } {
2851             \__bnvs_tl_if_empty:cF { VS } {
2852               \__bnvs_tl_put_right:cn { VS } { , }
2853             }
2854             \__bnvs_match_if_once:NnTF \c__bnvs_A_AZ_Z_regex { #####1 } {
2855               \__bnvs_match_pop:
2856               \__bnvs_match_pop:cc az
2857               \__bnvs_tl_if_empty:cT a {
2858                 \__bnvs_tl_set:cn a 1
2859               }
2860               \__bnvs_gset_azl:nn { #1 } { #2 }
2861               \STOP \BNVS_range:nn { \l__bnvs_a_tl } { \l__bnvs_z_tl } ???
2862               \__bnvs_tl_put_right:cx { VS } {
2863                 \l__bnvs_a_tl - \l__bnvs_z_tl
2864               }
2865             } {
2866               \__bnvs_tl_put_right:cn { VS } { #####1 }
2867             }
2868           }
2869         }
2870       } {
2871         \__bnvs_tl_if_empty:cF { VS } {
2872           \__bnvs_tl_put_right:cn { VS } { , }
2873         }
2874         \__bnvs_tl_put_right:cn { VS } { 1 }
2875       }
2876     }
2877   }
2878   \tl_if_head_eq_meaning:nNTF { #3 } \BNVS:n {
2879     #3

```

```

2880 } {
2881   \BNVS:n { #3 }
2882 }
2883 \__bnvs_tl_if_empty:cTF { VS } {
2884   \tl_if_empty:nF { #4 } { \__bnvs_tl_clear:c { #4 } }
2885   \tl_if_empty:nF { #5 } { \__bnvs_tl_clear:c { #5 } }
2886 } {
2887   \__bnvs_gunset:nn { #1 } { #2 }
2888   \BNVS_tl_use:nv { \regex_match:nnTF { [-,] } } { VS } {
2889     \__bnvs_gset:nnv { #1 } { #2 } R { VS }
2890     \tl_if_empty:nF { #4 } { \__bnvs_tl_clear:c { #4 } }
2891     \tl_if_empty:nF { #5 } { \__bnvs_tl_set:cv { #5 } { VS } }
2892   } {
2893     \__bnvs_gset:nnv { #1 } { #2 } V { VS }
2894     \tl_if_empty:nF { #4 } { \__bnvs_tl_set:cv { #4 } { VS } }
2895     \tl_if_empty:nF { #5 } { \__bnvs_tl_clear:c { #5 } }
2896   }
2897 }
2898 }

```

__bnvs_if_resolve_init:nnTF __bnvs_if_resolve_init:nnTF {<id>} {<path>} {<yes:>} {<no:>}

Auxiliary function used by __bnvs_if_resolve_?:. If it can resolve <spec>, setup the <id>!<<path> data model accordingly and execute <yes:>. Otherwise mark unsolvability and execute <no:>.

On entering, this function assumes that there is absolutely no data for <id>!<<path>/V nor <id>!<<path>/S (it has been unset for example).

This function is executed within a T_EX group in order not to alter the outer world, because it must be reentrant.

```

2899 \BNVS_new_conditional:cpnn { if_resolve_init:nn } #1 #2 { T, F, TF } {
2900   \BNVS_begin:
2901   \__bnvs_if_get:nnncTF { #1 } { #2 } = { VS } {
2902     <id>!<<path> is known: prepare for circular references.
2903     \__bnvs_will_gset:nnn { #1 } { #2 } V
2904     \__bnvs_will_gset:nnn { #1 } { #2 } S
2905     \__bnvs_tl_clear:c { VS }
2906     \cs_set:Npn \BNVS:n ##1 {
2907       \__bnvs_tl_clear:c { vs }
2908       \__bnvs_if_append:nnncF { #1 } { #2 } { ##1 } { vs } {
2909         \__bnvs_if_resolve:ncTF { ##1 } { vs } {
2910           \BNVS_tl_use:Nv \clist_map_inline:nn { vs } {
2911             \tl_if_empty:nF { ####1 } {
2912               \__bnvs_tl_if_empty:cF { VS } {
2913                 \__bnvs_tl_put_right:cn { VS } { , }
2914               }
2915               \__bnvs_match_if_once:NnTF \c__bnvs_A_AZ_Z_regex { ####1 } {
2916                 \__bnvs_match_pop:
2917                 \__bnvs_match_pop:cc az
2918                 \bnvs_tl_if_empty:cT a {
2919                   \__bnvs_tl_set:cn a 1
2920                 }
2921               }
2922             }
2923           }
2924         }
2925       }
2926     }
2927   }
2928 }

```



```

2921         \STOP \BNVS_range:nn { \l__bnvs_a_tl } { \l__bnvs_z_tl } ?
2922         \__bnvs_tl_put_right:cx { VS } {
2923             \l__bnvs_a_tl - \l__bnvs_z_tl
2924         }
2925     } {
2926         \__bnvs_tl_put_right:cn { VS } { ####1 }
2927     }
2928 }
2929 }
2930 } {
2931     \__bnvs_tl_if_empty:cF { VS } {
2932         \__bnvs_tl_put_right:cn { VS } { , }
2933     }
2934     \__bnvs_tl_put_right:cn { VS } { 1 }
2935 }
2936 }
2937 }
2938 \cs_set:Npn \BNVS_if_resolve_init:n ##1 {
2939     \__bnvs_tl_clear:c { VS }
2940     \tl_if_head_eq_meaning:nNTF { ##1 } \BNVS:n {
2941         ##1
2942     } {
2943         \BNVS:n { ##1 }
2944     }
2945 }
2946 \BNVS_tl_use:Nv \BNVS_if_resolve_init:n { VS }
2947 \BNVS_tl_use:nv { \regex_match:nnTF { [-,] } } { VS } {
2948     \__bnvs_gunset:nn { #1 } { #2 }
2949     \__bnvs_gset_unsolvable:nnn { #1 } { #2 } V
2950     \__bnvs_gset:nnnv { #1 } { #2 } S { VS }
2951     \BNVS_end:
2952     \prg_return_true:
2953 } {
2954     \__bnvs_tl_if_empty:cTF { VS } {
2955         \__bnvs_gunset:nnn { #1 } { #2 } V
2956         \__bnvs_gset:nnn { #1 } { #2 } S
2957         \BNVS_end:
2958         \prg_return_false:
2959     } {
2960         \__bnvs_gunset:nn { #1 } { #2 }
2961         \__bnvs_gset:nnnv { #1 } { #2 } V { VS }
2962         \__bnvs_gset_unsolvable:nnn { #1 } { #2 } S
2963         \BNVS_end:
2964         \prg_return_true:
2965     }
2966 }
2967 } {

```

❗ $\langle id \rangle!$ $\langle path \rangle$ is not known.

```

2968     \BNVS_end:
2969     \prg_return_false:
2970 }
2971 }
2972 \BNVS_undefine:c { if_resolve_init:nn }

```

6.15.3 V for value

```

__bnvs_if_resolve_V:nncTF __bnvs_if_resolve_V:nncTF {<id>}<path> <ans> {<yes:>} {<no:>}}
__bnvs_if_resolve_V:wvcTF __bnvs_if_append_V:nncTF {<id>}<path> <ans> {<yes:>} {<no:>}}
__bnvs_if_append_V:nncTF
__bnvs_if_append_V:wxcTF
__bnvs_if_append_V:wvcTF

```

Resolve the content of the $\langle id \rangle!$ $\langle path \rangle$ value counter into the $\langle ans \rangle$ t1 variable or append this value to the right of this variable. Execute $\langle yes: \rangle$ when there is a $\langle value \rangle$, $\langle no: \rangle$ otherwise. Inside the $\langle no: \rangle$ branch, the content of the $\langle ans \rangle$ t1 variable is undefined. Implementation detail: in $\langle ans \rangle$ we return the first in the cache for subkey V and in the general prop for subkey V (once resolved). Once we have found a value, we feed the previous items such that the next search stops at the first item. The cache contains an integer which is the computed value from the general prop. A local group is created while appending but not while resolving.

If a range is associated to $\langle id \rangle!$ $\langle path \rangle$, $\langle no: \rangle$ is executed.

```

2973 \BNVS_new_conditional:cpnn { if_resolve_V:nnc } #1 #2 #3 { T, F, TF } {
2974   __bnvs_if_get:nnncTF { #1 } { #2 } V { #3 } {
2975     __bnvs_is_will_gset:cTF { #3 } {
2976       \BNVS_error:n { Circular~definition:~#1!#2/V (Error~recovery~1) }
2977       __bnvs_gunset:nn { #1 } { #2 }
2978       __bnvs_gset:nnnn { #1 } { #2 } 1 1
2979       __bnvs_gset:nnnn { #1 } { #2 } V 1
2980       __bnvs_tl_set:cn { #3 } 1
2981       \prg_return_true:

```

Circular definition call during resolution.

```

    __bnvs_ans_tl
2982   } {
    l__bnvs_ans_tll__bnvs_ans_tl
2983     \prg_return_true:
2984   }
2985 } {
2986   __bnvs_is_provided:nnnTF { #1 } { #2 } R {
2987     __bnvs_set_unresolvable:nnn { #1 } { #2 } V
2988     \prg_return_false:
2989   } {
2990     __bnvs_if_resolve_init:nnTF { #1 } { #2 } {
2991       __bnvs_if_resolve_V:nncTF { #1 } { #2 } { #3 } {
2992         \prg_return_true:
2993       } {
2994         \prg_return_false:
2995       }
2996     } {
2997       \prg_return_false:
2998     }
2999   }
3000 }
3001 }
3002 \BNVS_undefine:c { if_resolve_V:nnc }
3003 \BNVS_new_conditional_vvc:cn { if_resolve_V } { T, F, TF }

3004 \BNVS_new_conditional:cpnn { if_append_V:nnc } #1 #2 #3 { T, F, TF } {
3005   \BNVS_begin:

```